ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

CENG 408

FINAL REPORT

TARAS: Tarım Analizi Sistemi

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Literature & Technology Review

1.1. Introduction

Modern agriculture is increasingly affected by climate variability, declining freshwater resources and sustainability challenges[1]. Many farmers still make irrigation and crop-management decisions based on intuition rather than systematic measurements, which reduces decision accuracy and leads to inefficient water use [2]. Research also shows that improper irrigation practices can cause water loss and yield reduction, making data-driven decision support essential for future agricultural productivity[3].

Recent advances in digital agriculture offer viable solutions. IoT-based sensing systems provide continuous measurements of soil and environmental conditions [4], while ML algorithms can analyze these data to generate accurate irrigation recommendations [4]. Digital twin technologies enable virtual representations of agricultural fields for real-time simulation and visualization [5] and later studies show that digital twins can significantly enhance field monitoring under dynamic conditions [6]. In parallel, computer-vision models have demonstrated strong performance in diagnosing plant diseases directly from leaf images [7].

TARAS platform brings these technologies together into a unified, farmer-oriented system that offers real-time field monitoring, irrigation prediction, disease assessment, and digital-twin visualization. The project aligns with Türkiye's Twelfth Development Plan, which emphasizes AI-supported digital agriculture [8], and also contributes to global sustainability objectives such as SDG 12, SDG 13, and SDG 15 [9].

1.1.1. Background

Agriculture has entered a new digital transformation phase often described as Agriculture 4.0, in which technologies such as IoT, artificial intelligence, robotics, and data-driven systems are expected to transform agricultural practices [10]. Precision Agriculture, a key component of this era, relies on monitoring and responding to spatial and temporal field variability to optimize resource use [4]. IoT sensors enable real-time environmental monitoring, digital twins provide interactive virtual models of fields for prediction and analysis [5], and computer-vision systems support early disease recognition through image-based diagnosis [7]. TARAS builds on this technological landscape to create a unified platform tailored for real agricultural environments.

1.1.2. Problem Definition

Traditional Farming Limitations: One of the major challenges in agriculture is that farmers struggle to make reliable, data-based decisions. Many still rely on intuition rather than systematic and evidence-based methods, which limits the accuracy of field management [1]. Although intuition has supported farming for generations, it is becoming less effective as climate patterns shift and soil

conditions change. As a result, crops often fail to reach their full yield potential when decisions depend solely on historical experience.

Resource Scarcity: Water scarcity is one of the most critical constraints in agriculture today. Traditional irrigation methods frequently lead to over-watering, which wastes water and damages root systems or under-watering, which stunts plant growth. Studies show that inefficient or poorly timed irrigation can cause significant water loss and reduce productivity [2], [3]. Because soil moisture varies across fields and changes over time, static rule-based irrigation schedules cannot meet crop-specific requirements, leading to substantial inefficiencies in both water and energy use.

Disease Management Challenges: Farmers commonly notice unusual visual changes in plants but cannot determine whether these differences indicate disease or harmless variation. This uncertainty can delay intervention or lead to unnecessary pesticide use. When early symptoms are subtle or unfamiliar, problems may be ignored until they become severe, or pesticides may be applied without need. Both outcomes increase production costs and negatively affect food safety and environmental health. Image-based decision-support systems can help address this uncertainty by allowing farmers to upload leaf images and receive automated analysis, improving early assessment and supporting timely, informed action [7].

Data Quality & System Reliability Issues: Even when farmers adopt technological tools, field data are often incomplete or inconsistent due to sensor drift, hardware faults, or communication failures. Soil-moisture probes may lose calibration over time, wireless connections may drop, and environmental fluctuations can distort sensor readings. Without proper validation, calibration, and anomaly-detection mechanisms, inaccurate data can mislead decision-making and become as harmful as intuition-based irrigation.

Lack of Accessible, Integrated Decision Support: Many farms rely on fragmented digital tools, such as separate systems for sensors, irrigation controllers, weather forecasts, and crop health information. Because these systems do not communicate with each other, farmers, especially small-scale producers, struggle to benefit from advanced analytics. This fragmentation highlights the need for an integrated platform that combines real-time field monitoring, intelligent recommendations, visual explanations, and sustainability metrics within a single accessible system.

1.1.3. Scope of the Study

The scope of TARAS covers the development of an end-to-end precision-agriculture platform. The system includes IoT-based soil-moisture and temperature sensing using battery-powered ESP32-C6 sensor nodes that relay measurements over ESP-NOW to a mains-powered ESP32 gateway, which forwards them to the cloud over WiFi. It incorporates a secure cloud database for storing real-time and historical records, a calibrated rule-based irrigation engine that applies FAO-inspired crop coefficients together with a simplified evapotranspiration estimate, and computer-vision algorithms for disease identification.

The project further includes the design of a mobile application that provides farmers with moisture visualization through a digital twin, image-based disease assessment, and an AI-assisted decision interface. The system focuses on decision support rather than full automation, ensuring that farmers remain central to the decision-making process. It is intended to be low-cost, portable and scalable across various field configurations.

1.1.4. Aim and Contributions

The aim of TARAS is to provide farmers with an intelligent, real-time agricultural assistant that can analyze environmental conditions, predict irrigation requirements, detect plant diseases from leaf images and visualize moisture distribution through digital twin. By bringing together IoT, ML, computer vision and cloud computing, the system offers a highly adaptive and user-centered approach that supports precision farming and sustainable resource management.

TARAS aims to optimize irrigation efficiency, reduce water and energy waste, enhance crop health monitoring, improve sustainability through carbon footprint tracking and provide farmers with actionable and explainable insights.

Contributions: Recent advancements in smart agriculture highlight the value of combining IoT-based sensing with ML-driven decision systems to improve irrigation efficiency and field management. Studies show that data-driven decision systems using soil moisture, temperature, humidity and soil-type data can generate precise irrigation recommendations. Rather than relying on data-hungry predictive models, TARAS deliberately adopts a transparent, calibrated rule-based irrigation engine, favouring interpretability and robustness under intermittent connectivity, and pairs it with real-time digital-twin visualizations, enabling farmers to interpret moisture patterns through intuitive heatmaps, observe changes over time and identify areas requiring intervention more clearly.

Beyond irrigation, TARAS incorporates computer-vision-based disease detection that allows farmers to upload leaf images through the mobile app.

These images are processed using CNN models that classify disease symptoms and provide actionable assessments, reducing the need for expert diagnosis. The platform also addresses sustainability by integrating carbon-footprint metrics, estimating emissions based on pump energy usage, fertilizer inputs and fuel.

1.2. IoT Sensor Node Architecture

The foundation of any smart farming system is the ability to collect accurate environmental data from the field [11]. The literature emphasizes that hardware in agriculture must be low-power, durable, and cost-effective.

1.2.1. Microcontroller Selection: Fully Wireless ESP32

In the realm of agricultural IoT, the primary engineering challenge is maximizing the operational lifespan of battery-powered nodes. Zhang and Meng (2021) highlight that energy efficiency is the most critical factor for the economic viability of precision agriculture, as frequent battery replacements increase labor costs significantly [12], and using wired technology introduces complexity most fields are not suited to deal with.

While earlier projects successfully utilized standard ESP32 or ESP8266 modules, recent literature indicates a shift toward more specialized architectures. Recent studies in wireless sensor networks have begun favoring efficient architectures for their superior power-to-performance ratio. For example, a study by Irga et al. (2023) implemented agricultural monitoring nodes noting a significant reduction in standby power consumption compared to traditional controllers [13].

Building on these findings, TARAS adopts a two-tier wireless architecture rather than connecting every node to WiFi. Battery-powered ESP32-C6 sensor nodes spend almost all of their time in deep sleep and communicate over ESP-NOW (Espressif's low-power, connectionless 2.4 GHz protocol) to a single mains-powered ESP32 gateway, which is the only device that maintains a WiFi connection and forwards readings to the cloud over HTTPS. Because the nodes never associate with an access point, they avoid the substantial energy cost of the WiFi handshake and TLS setup, extending battery life to months while keeping each node simple and inexpensive. This directly addresses the energy-efficiency imperative that Mohd Khalid et al. (2024) identify as critical for continuous year-round field monitoring [14].

Another advantage of using ESP32 is its support of the I²C (Inter-Integrated Circuit) Communication Interface, which is a widely used two-wire communication interface designed for low-power embedded systems. I²C enables multiple digital sensors to communicate with the ESP32 over a shared SDA (data) and SCL (clock) bus, reducing wiring complexity and supporting scalable node designs. Its address-based structure allows several

sensors to operate simultaneously, making it particularly suitable for multi-parameter agricultural monitoring. The protocol's low latency and efficient power consumption make it ideal for continuous field measurements where reliability and simplicity are critical[15].

1.2.2. Sensing Technologies: Capacitive vs. Resistive

Accurate irrigation depends entirely on accurate soil moisture readings. Historic literature often cited "resistive" sensors, which measure moisture by passing an electric current through the soil [16]. However, engineering studies have documented a major limitation of resistive sensors: the DC current flowing through the electrodes can cause electrolysis and electrode wear/corrosion, reducing long-term measurement reliability [17].

To address this limitation, modern soil-moisture monitoring systems often use capacitive soil moisture sensors. These sensors estimate soil water content by measuring changes in the dielectric properties of the soil rather than relying on direct electrical conduction through metal probes. Since capacitive designs avoid the electrode-wear problem associated with resistive probes, they are more suitable for automated and long-term soil moisture monitoring when properly calibrated [17], [18]. Similarly, for air metrics, digital sensors like the SHT series sensors are preferred over analog thermistors because they provide calibrated digital output, eliminating errors caused by electrical noise in the wires [19].

1.2.3. Sensor Calibration

Proper calibration of soil moisture sensors is essential for accurate readings. TARAS employs a simple min-max calibration procedure that can be performed in the field. Using two reference samples, completely dry soil and fully saturated (muddy) soil, the sensor's raw analog values are mapped to the 0-100% moisture range. This calibration data is stored and applied to all subsequent readings. This approach ensures that sensors remain accurate across different soil types.

1.3. Cloud Computing and Data Management

Handling the influx of data from the field requires a scalable and organized backend system.

1.3.1. Structured Database Design (PostgreSQL)

Agricultural data is primarily "Time-Series Data" (e.g., specific moisture values recorded at specific times). While flexible NoSQL databases are popular for web apps, scientific apps usually prefer Relational Databases like **PostgreSQL** for their strict structure and reliability. It is ACID-compliant, meaning it guarantees data validity even in the event of errors or power failures [20]. Sensor telemetry in particular is stored as an append-only time-series log: once a reading is written it is never edited, preserving an immutable record of field conditions that is

valuable for later analysis. Operational entities such as zones, tracking folders and irrigation jobs use conventional relational updates, with soft-deletion, a deleted flag and timestamp rather than a hard delete, so that historical context is retained even when a record is removed from the active view.

1.4. Irrigation Recommendation

Determining when to irrigate crops is primarily a decision-making problem that depends on accurately interpreting soil-moisture dynamics and environmental conditions. Traditional irrigation controllers often rely on fixed schedules or simple threshold-based rules, which can lead to inefficient water use when field conditions vary over time.

In this system, irrigation decisions are generated using a calibrated rule-based approach that evaluates time-series soil-moisture trends together with environmental conditions and field-specific parameters. By analyzing moisture-level thresholds and rate-of-change behavior, the method adapts irrigation timing and duration/amount without relying on large historical datasets or complex predictive models.

Compared to ML-based solutions, this rule-based method offers improved transparency, lower computational complexity and greater robustness in environments with limited data availability or intermittent connectivity.

1.5. Artificial Intelligence and Machine Learning

Collecting data is only useful if it leads to better decisions. AI is used to process raw numbers into actionable advice.

1.5.1. Computer Vision for Disease Detection (CNNs)

Identifying plant diseases early can prevent significant yield losses; however, expert agronomists are not always readily available [7]. To address this challenge, we employ CNNs, which are deep learning models specifically designed for image analysis. By training a CNN on large-scale datasets containing images of both healthy and diseased plant leaves, the model learns to identify discriminative visual patterns such as color variations, texture irregularities, and lesion structures associated with different plant diseases.

In this study, disease classification is performed by an ensemble of three complementary backbones evaluated server-side: a ConvNeXtV2 convolutional network [21] and two Swin Transformer models of differing capacity [22]. Convolutional and attention-based architectures capture different visual cues, local texture and lesion structure versus longer-range spatial context, so averaging their softmax outputs yields a more robust diagnosis than any single model. This ensemble produces the authoritative result returned to the farmer, across 14 disease classes spanning seven crops. Because running three large models is infeasible on a phone, TARAS also ships a single compact

EfficientNet-B0 on the device through knowledge distillation [4]: the ensemble acts as a "teacher" whose soft predictions train the smaller "student" to approximate its accuracy at a fraction of the size. EfficientNet's compound scaling strategy, which uniformly balances depth, width and resolution, makes the student highly efficient for on-device inference [24]. This edge–cloud split gives farmers instant feedback while framing a leaf, with the server ensemble providing the final, higher-accuracy assessment.

1.5.2. Large Language Models

To translate complex analytical outputs into actionable guidance for farmers, TARAS integrates an LLM within an interactive chatbot interface. This component acts as the system's communication layer, interpreting sensor data, historical irrigation records and computer vision findings to generate clear and context-aware explanations.

Recent research highlights that LLMs can support agricultural decision-making by translating complex agronomic information into natural-language explanations and advisory responses that are easier for farmers to understand [25]. Instead of presenting users with raw numerical values, the LLM-equipped assistant provides meaningful insights. Rather than relying on model fine-tuning, the assistant is implemented as an agentic, tool-calling system. When a farmer asks a question, the underlying model autonomously decides which backend tools to invoke, like retrieving live sensor readings, irrigation history, disease-detection results, carbon summaries and active alerts, and grounds its natural-language answer in the data those tools return. This tool-use loop keeps responses accurate and current with the live state of the field without any retraining, while a cached system prompt supplies agronomic context. In this way the assistant turns raw numerical data into clear, context-aware explanations, transforming TARAS into a transparent, reliable, and user-friendly agricultural advisor.

1.6. Visualization and Digital Twins

The interface between the human and the machine is just as important as the technology itself.

1.6.1. Reducing Cognitive Load with Digital Twins

"Cognitive Load" refers to how much mental effort is needed to understand information. Research indicates that standard spreadsheets impose a high cognitive load, whereas visual maps reduce this load significantly [26]. The concept of the digital twin involves creating a 3D virtual replica of the physical field. Research suggests that visualizing data spatially, such as a 3D heat map where dry areas glow red, allows farmers to identify problem zones much faster than reading lists of numbers [27]. To make this accessible, React Native is used

to build a cross-platform mobile app, ensuring the tool works on the smartphones farmers already own.

1.7. Mobile Application

Mobile apps have emerged as one of the most widely adopted interfaces in smart agriculture systems. Studies highlight that mobile platforms make complex IoT and AI infrastructures accessible by presenting field information directly to farmers in real time. They support continuous monitoring, rapid decision making, and convenient data entry, which makes them a practical tool for field operations where desktop interfaces are rarely available [28].

1.7.1. Mobile Applications in Agriculture

Recent smart-agriculture research emphasizes the role of mobile apps as the primary interaction layer between farmers and digital farming systems. Mobile tools frequently act as gateways for sensor networks, cloud platforms and ML models. Literature also notes that mobile interfaces significantly increase technology adoption among farmers, especially when the design is simple and the recommendations are easy to interpret [28].

1.7.2. Mobile App in TARAS

Within this literature landscape, the TARAS mobile app follows the same principles identified in prior studies. It functions as the primary interface through which farmers receive sensor readings, irrigation recommendations, and disease analysis results. The app also includes a 3D digital-twin visualization that represents moisture distribution spatially. By combining monitoring, prediction, disease assessment and spatial visualization into a single platform, TARAS addresses the fragmentation noted in current literature and provides a unified decision-support tool aligned with modern precision-agriculture practices.

1.7.3. Technology Stack

The technological choices used in the TARAS mobile app reflect practices recommended in mobile and IoT literature. **React Native** is widely adopted for agricultural applications because it enables cross-platform deployment on Android and iOS, which is essential given the diversity of devices in rural areas [29]. Expo simplifies development and maintenance processes by managing native modules and supporting incremental updates, which is useful for systems deployed in field environments with limited technical infrastructure [30].

For visualization, **React Three Fiber** allows efficient rendering of 3D elements on mobile devices[31], supporting interactive digital-twin displays similar to those explored in recent spatial agriculture studies [27]. The data exchange between the mobile client and the cloud backend uses a simple

request–response API model, which is frequently adopted in smart farming architectures for real-time synchronization.

1.8. Ecological Impact

Technology must also serve environmental goals. Modern agriculture is under pressure to reduce its environmental impact, in line with UN Sustainable Development Goals 12 and 13 [9]. Literature suggests that IoT systems are uniquely positioned to automate sustainability tracking. By monitoring exactly how long water pumps run and how much resource is consumed, the system can calculate a real-time **Carbon Footprint**. This moves sustainability from a vague concept to a measurable metric [32].

1.8.1. Water Consumption

The TARAS system supports responsible resource use by ensuring irrigation is applied only when and where necessary, aligning with Goal 12. By replacing uniform, schedule-based watering with sensor-driven and algorithm-optimized recommendations, TARAS reduces water waste, nutrient runoff, and the environmental footprint of agriculture. This precision also lowers pumping energy needs, reducing operational emissions and supporting Goal 13. Through Digital Twin visualizations and adaptive decision support, farmers can monitor consumption, identify inefficiencies, and adopt more sustainable long-term practices, strengthening resource responsibility and climate resilience.

1.8.2. Carbon Footprint

Agriculture is one of the most significant contributors to global greenhouse gas (GHG) emissions, accounting for approximately one-fifth of total anthropogenic emissions [33]. The main sources include fertilizer production and application, energy consumption for irrigation and machinery, livestock management, and soil-based emissions such as nitrous oxide (N_2O) from nitrogen fertilizers and methane (CH_4) from organic decomposition. These emissions, when expressed as carbon dioxide equivalents (CO_2e), constitute the carbon footprint of agricultural production. Understanding and reducing this footprint has become a major focus in achieving the United Nations Sustainable Development Goals (SDGs), particularly Goal 12 (Responsible Consumption and Production) and Goal 13 (Climate Action).

Recent studies indicate that digital and IoT-based farming systems can play a transformative role in mitigating agricultural greenhouse gas emissions. Precision irrigation systems that utilize real-time soil-moisture and climate sensing enable farmers to reduce unnecessary water use and the associated energy required for pumping, thereby decreasing CO_2 emissions [34]. Similarly, IoT-supported fertilization and nutrient-management approaches help align nitrogen application with actual crop demand, preventing overuse and limiting nitrous oxide (N_2O) emissions, one of the most potent agricultural greenhouse

gases [35]. Overall, these digital and sensor-driven strategies improve resource efficiency and contribute to more sustainable agricultural production [36].

By integrating this functionality into a mobile app, farmers can view their estimated emissions and identify the main contributing factors. Within TARAS, this allows users to enter their input data and instantly calculate their farm's carbon footprint. The module supports sustainability awareness and provides actionable insights for emission reduction strategies, reinforcing the system's contribution to climate action (Goal 13) and responsible production (Goal 12).

1.9. Related Work

To design the TARAS platform, we analyzed several existing systems and research projects. While many solutions exist, they typically solve only one part of the problem. This section discusses related works and how they influenced our design choices.

1.9.1. Single-Purpose Smart Irrigation Systems

A significant amount of research focuses solely on automating water pumps.

- **The Study:** A prominent study by Nawandar & Satpute (2019) developed a low-cost IoT system using simple neural networks to schedule irrigation based on soil data [37].
- **Limitation:** While effective at saving water, this system operates in a vacuum. It assumes the plant is healthy. If a plant is attacked by a fungus, it might need less water or a chemical treatment, not just standard irrigation. The system had no way of "seeing" the plant's health.
- **Effect on TARAS:** This influenced us to build a hybrid system. TARAS combines soil sensors with camera-based disease detection so decisions are made based on the whole picture, not just the soil moisture level.

1.9.2. Standalone Disease Detection Applications

On the other side of the spectrum, there is extensive research on using AI to spot diseases.

- **The Study:** The most famous work in this field is by Mohanty et al. (2016), who trained Deep Learning models on the PlantVillage dataset [7].
- **Limitation:** While scientifically impressive, this approach was purely diagnostic. It tells the farmer "You have Apple Scab," but it doesn't link that information to the farm's environmental history. Was the disease caused by over-watering? The app couldn't say because it didn't have access to soil data.
- **Effect on TARAS:** This taught us that context matters. In TARAS, when a disease is detected, the system can cross-reference it with historical humidity logs to help explain why the outbreak happened.

1.9.3. High-End Digital Twins

Finally, we looked at existing "Digital Twin" concepts.

- **The Study:** Pylianidis et al. (2021) introduced the concept of digital twins in agriculture and discussed how digital representations of agricultural systems can support monitoring, simulation, and decision-making[38].
- **Limitation:** These systems are extremely complex, expensive, and require powerful desktop computers. They are designed for industrial agronomists, not for a typical farmer standing in a field with a phone.
- **Effect on TARAS:** This defined our Mobile-First strategy. We chose technologies like React Native and low-poly 3D rendering to ensure our Digital Twin works smoothly on a standard smartphone, making the technology accessible to regular farmers.

1.10. Conclusion and Research Gap

However, a clear gap remains: most current solutions work in isolation, focusing either on irrigation control or disease detection, while often lacking user-friendly visualization for small-scale farmers. Few also track sustainability metrics such as carbon footprints alongside agronomic data. Therefore, a unified, low-cost mobile platform combining real-time sensing, AI-driven predictions, and visual analytics is needed. TARAS aims to fill this gap with an end-to-end solution tailored to these needs.

2. Software Requirements Specification

2.1. Introduction

2.1.1. Purpose

The purpose of this Software Requirements Specification is to define the objectives, functions, and boundaries of the TARAS system. TARAS supports farmers in making accurate, data-driven decisions about irrigation, plant health and field management by using IoT sensors, rule-based zone-specific irrigation recommendations and computer-vision-based disease assessment. This document focuses on the TARAS mobile app, which serves as the main interface for real-time digital twin visualization, personalized notifications, decision support, and sustainability tools such as carbon-footprint estimation. It defines the software's goals, intended functions and constraints to ensure a shared understanding among all stakeholders.

2.1.2. Scope

The project scope includes developing an end-to-end precision-agriculture system that collects soil and environmental data through IoT sensor nodes and

sends them to a gateway via ESP-NOW, and the gateway forwards telemetry to the cloud backend via Wi-Fi/HTTPS. The data are stored in a cloud database and processed through algorithmic irrigation calculations, computer-vision-based disease detection, and an LLM for decision support. A mobile app provides farmers with digital-twin soil moisture visualization, real-time alerts, recommendations and carbon-footprint estimation.

2.2. General Description

2.2.1. Glossary

Term	Definition
Farmer	The primary user of TARAS who manages fields, monitors irrigation, reviews alerts, uploads plant images, and uses the mobile application for decision support.
Stakeholder	A read-only external user such as a cooperative, advisory body, or environmental institution that views aggregated TARAS outputs for monitoring and reporting.
Field	The agricultural area managed by a farmer and monitored through TARAS.
Field Zone	A logical or spatial subdivision of a field used for monitoring, visualization, and irrigation decisions.
Digital Twin	A virtual representation of the field that combines sensor data, spatial layout, and visualization to support field monitoring and decision-making.
IoT Sensor Node	A distributed ESP32-based device that collects soil moisture, temperature, humidity, and diagnostic data, then sends telemetry to the gateway.
Gateway	A central device that receives sensor-node data through ESP-NOW and forwards validated telemetry to the backend via Wi-Fi/HTTPS.
ESP32 / ESP32-C6	A low-power microcontroller used in TARAS sensor nodes and gateway components for sensing, communication, and embedded control.
SHT Sensor	A digital temperature and humidity sensor series that communicates through I²C and provides environmental measurements.
Capacitive Soil Moisture Sensor	A sensor that measures soil moisture through capacitance changes, offering better durability than resistive sensors.
I²C	A low-power serial communication protocol that connects microcontrollers with sensors using SDA and SCL lines.

ADC	A hardware unit that converts analog sensor signals, such as soil-moisture voltage, into digital values.
Telemetry	Sensor and diagnostic data collected from field devices and transmitted to the backend for storage and analysis.
Telemetry Packet	A structured data unit containing sensor readings, timestamps, device identifiers, and diagnostic information.
Time-Series Data	Data collected sequentially over time, such as soil moisture or temperature readings.
Store-and-Forward	A reliability strategy where data is stored locally during connectivity loss and transmitted when connection is restored.
Deep Sleep Mode	A low-power microcontroller state used to conserve battery between sensing and transmission cycles.
Backend	The server-side part of TARAS responsible for APIs, data storage, authentication, analytics, and integration with system modules.
Frontend / Client	The user-facing mobile application that displays field data, recommendations, alerts, reports, and system interactions.
REST API	A HTTP-based interface that enables structured communication between the mobile app, backend, and other services.
Cloud Services	Internet-based computing resources used for hosting backend services, databases, storage, and scalable processing.
PostgreSQL	The relational database system used to store structured TARAS data such as users, farms, fields, sensor readings, and analysis results.
TimescaleDB	A PostgreSQL extension optimized for storing and querying time-series sensor data.
PostGIS	A PostgreSQL extension used for storing and processing geospatial data such as field boundaries and zone polygons.
Storage Bucket	A cloud storage container used for files such as uploaded plant images and related assets.
JWT (JSON Web Token)	A secure token format used for authentication and authorization between the client and backend.
Irrigation Recommendation	A rule-based component that uses sensor readings, crop growth stage, field type, thresholds, and calibration parameters to generate irrigation

Engine	suggestions.
Evapotranspiration (ET_0)	A standardized measure of atmospheric water demand used to estimate water loss under reference conditions.
Crop Coefficient (Kc)	A coefficient used to adapt ET_0 to a specific crop and growth stage.
DAP (Days After Planting)	The number of days elapsed since planting, used to determine crop growth stage and related irrigation parameters.
Threshold	A predefined limit that triggers alerts, warnings, or irrigation decisions when crossed.
Disease Detection Module	A computer-vision component that analyzes leaf images and returns a disease-or-health label with a confidence score.
Machine Learning (ML)	Computational methods that learn patterns from data to make predictions or classifications, used in TARAS mainly for disease detection and advisory support.
ML Model	A trained model that generates predictions or classifications from input data, such as disease likelihood from leaf images.
Classification	A machine-learning task where input data is assigned to one of several predefined classes.
Inference	The process of applying a trained model to new input data to generate an output.
Computer Vision (CV) Model	A model designed to analyze visual data such as images, used in TARAS for plant disease detection.
CNN (Convolutional Neural Network)	A deep-learning architecture commonly used for image classification and visual feature extraction.

2.2.2. User Characteristics

The system is designed with two key user groups in mind: the primary user (farmer), and external institutional stakeholders (such as cooperatives or environmental reporting bodies). Each group has distinct abilities, responsibilities and interaction needs, which shape the system's interface and functionality.

2.2.2.1. Primary User: Farmer

Profile: Small-to-medium-scale farmers who rely on their experience for regular irrigation and crop-health decisions. They interact with the system mostly in outdoor field conditions.

Age Range: Typically 20–65, though many may be older and accustomed to traditional farming practices.

Technical Skills: Low-to-moderate comfort with digital tools. Interfaces shall be simple, intuitive and optimized for mobile use, with minimal steps and clear visual cues.

Work Environment: Outdoor, field-based conditions with limited connectivity and variable lighting.

Key Characteristics:

- Relies heavily on intuitive decision-making
- Faces challenges like water scarcity, plant disease uncertainty
- Needs clear, non-technical explanations rather than complex analytics
- Requires solutions that are low-cost, reliable and field-tolerant

2.2.2.2. External Party: Stakeholders

Profile: Stakeholders are organizations with highly restricted, read-only access to the system. They do not take part in daily decision-making or sensor management; their role is limited to viewing summaries and sustainability metrics for monitoring or reporting purposes.

Examples:

- Agricultural cooperatives
- Sustainability and environmental monitoring institutions
- Government programs that track water usage, efficiency, or carbon impact

Interaction Model: Stakeholders use a summary-focused dashboard to view high-level insights such as water consumption trends, irrigation efficiency metrics and carbon-footprint estimates. They cannot upload data, change settings or use field-level features like disease detection or digital-twin visualizations. Their role is passive and observational.

2.2.3. Overview of Functional Requirements

TARAS provides integrated functional capabilities for data-driven smart agriculture. The system continuously collects soil and environmental data from IoT sensor nodes, sends them to a gateway via ESP-NOW, and the gateway forwards telemetry to the cloud backend via Wi-Fi/HTTPS, where they are stored as time-series records for analysis, visualization, and decision support. Using these data, TARAS generates rule-based irrigation recommendations and supports computer-vision-based plant disease detection. The mobile app allows users to view real-time and historical field conditions through a digital twin, receive recommendations, upload plant images, and use advisory features. It also supports sustainability functions such as carbon-footprint calculation based on farmer input. Together, these requirements define an end-to-end system combining sensing, communication, data processing, analytics, visualization, and user interaction to provide reliable agricultural decision support.

2.2.4. General Constraints and Assumptions

2.2.4.1. Constraints

- Hardware Constraints:
  The sensor nodes operate on ESP32 microcontrollers with limited RAM and processing capability, which restricts complex on-device computation. As a result, most data processing, analytics, and decision-making tasks are offloaded to the gateway and cloud backend through ESP-NOW and Wi-Fi/HTTPS communication.

  All soil-moisture, temperature, and humidity sensors must operate within strict voltage ranges because of the ESP32's output limitations (typically 3.3 V), limiting hardware choices[39]. Since all sensor nodes are battery-powered, energy consumption must remain minimal to ensure long-term field deployment.

- Communication Constraints:
  The system uses ESP-NOW communication between ESP32-C6 sensor nodes and an ESP32-based gateway. Sensor nodes transmit low-power telemetry packets to the paired gateway, while the gateway uses Wi-Fi connectivity to forward validated telemetry to the cloud backend over secure HTTP/HTTPS communication. This architecture reduces the need for each battery-powered sensor node to maintain a direct internet connection and supports more efficient field-level data collection.

  Because agricultural environments may have unstable wireless conditions, both ESP-NOW communication and gateway internet connectivity may experience temporary interruptions. To maintain data continuity, sensor nodes use local buffering when

gateway acknowledgments are not received, and the gateway stores unsent telemetry when Wi-Fi or backend connectivity is unavailable. Buffered data are retransmitted once communication is restored, supporting reliable synchronization with the cloud backend.

- Environmental Constraints:
  Devices that are all field-deployed must withstand harsh outdoor conditions such as humidity, temperature extremes, rainfall, dust, and direct sunlight. Variations in soil composition and microclimatic differences introduce noise into sensor readings, potentially affecting predictive model accuracy.

- Software & Data Constraints:
  Available training data for ML models is limited and may not represent all crop types or environmental contexts, reducing early-stage model accuracy[40].

  Real-time inference must meet latency requirements appropriate for on-field decision-making, which imposes limits on model size and computational complexity.

- Operational Constraints:
  Continuous internet connectivity cannot be guaranteed for either the gateway or the user's mobile device, requiring offline-tolerant system behavior. The mobile app must support a wide range of user devices, specifically Android versions 7.0–16.0 and iOS versions 15.1–26.1.

- Budget & Resource Constraints:
  High-quality agricultural sensors and reliable power-management components represent a significant portion of the system cost. In addition, ensuring continuous Wi-Fi or internet connectivity in agricultural environments may require auxiliary networking equipment, such as mobile hotspots or cellular modems, which can increase deployment and operational expenses.

  Real-world field access for testing may be limited, which may reduce the frequency and duration of validation cycles.

2.2.4.2. Assumptions

- Sensor Deployment Assumptions:
  Sensors are assumed to be properly installed, calibrated, and positioned in pre-determined areas of the field. Battery-powered sensor nodes are assumed to maintain sufficient charge throughout their scheduled data collection intervals.

- Communication Assumptions:
  The sensor nodes are assumed to operate within ESP-NOW communication range of the paired gateway. The gateway is assumed to have intermittent but recurring Wi-Fi or internet access for backend synchronization. Temporary connectivity outages are expected in agricultural environments; however, it is assumed that retry and buffering mechanisms will allow buffered data to be transmitted successfully once connectivity is restored, ensuring eventual synchronization with the cloud backend without permanent data loss.

- User Behavior Assumptions:
  Farmers are assumed to regularly monitor the mobile application and take appropriate actions based on irrigation recommendations or disease analysis results. External stakeholders are assumed to require only high-level, read-only access for monitoring and reporting purposes.

- System Assumptions (Irrigation & Disease Detection):
  Soil-moisture, temperature, and humidity sensor readings are assumed to accurately represent current field conditions for irrigation decision-making. The rule-based irrigation logic assumes that moisture trends and threshold crossings reliably indicate crop water stress under normal operating conditions.

  Images uploaded for disease detection are assumed to be clear, correctly captured, and representative of the plant's current health state.

  Environmental anomalies or abnormal conditions (e.g., extreme weather, sensor faults, or equipment malfunctions) are assumed to be detectable through irregular sensor behavior or inconsistent image analysis results.

  Calibration parameters and decision rules are assumed to remain valid for the defined crop type and field conditions unless manually updated.

- Mobile Application Assumptions:
  The mobile app assumes intermittent but recurring Internet connectivity to retrieve sensor data, digital twin updates, and rule-based irrigation recommendations.

2.3. Specific Requirements

2.3.1. Interface Requirements

2.3.1.1. Hardware Interface Requirements

The TARAS system interacts with a distributed network of ESP32-C6 sensor nodes and an ESP32-based gateway. Each sensor node incorporates an ESP32 microcontroller, which provides the necessary GPIO pins, I²C bus capabilities and low-power operational modes required for continuous monitoring. The hardware interface must support stable connections to capacitive soil-moisture sensors, SHT31 temperature-humidity units and additional digital sensors through standardized I²C communication. All sensors must operate within the microcontroller's voltage range (3.0–3.6 V) and support uninterrupted periodic sampling[39].

Sensor nodes communicate with the gateway using ESP-NOW to reduce power consumption and avoid requiring each node to maintain direct internet connectivity. The gateway uses Wi-Fi and HTTPS to forward validated telemetry to the backend. Power interfaces are also essential: each sensor node must operate reliably with a 3.7 V Li-Ion or Li-Po battery, support deep-sleep operation, and remain physically robust and weather-resistant for field or greenhouse use.

2.3.1.2. Communication Requirements

The TARAS communication infrastructure is designed as a scalable and reliable architecture for continuous IoT telemetry, real-time ML inference for disease detection and mobile app.

At the field level, distributed ESP32-C6 sensor nodes collect soil moisture and environmental data, then transmit telemetry packets to the paired ESP32-based gateway using ESP-NOW. The gateway validates and aggregates incoming packets, then forwards telemetry to the cloud backend through Wi-Fi over HTTPS. This architecture reduces the need for each battery-powered sensor node to maintain a direct internet connection while supporting reliable field-level data collection.

The system's data infrastructure is built around a cloud-hosted PostgreSQL database, deployed on a cloud platform such as AWS. This backend architecture supports reliable data storage, efficient querying of historical records and real-time access for analytical services[41].

TARAS's backend architecture consists of two layers: the cloud-based main backend and the service layer that manages mobile clients. The main backend consists of API services running on a

commercial cloud hosting service such as AWS. Sensor data from the nodes first reaches the API layer, where it undergoes formatting before being transferred to time series tables. The backend service provides all the necessary API endpoints for users for them to view the digital twin map, receive irrigation recommendations and upload images for disease detection. This layer will include “caching” strategies to ensure devices can operate in intermittent internet conditions by keeping any recent data that has failed to transmit. Authentication is handled through JWT-based backend authentication. The data model includes a structured schema consisting of sensor nodes, time-series telemetry records, field regions, digital twin snapshots, irrigation recommendation processing history, disease detection outputs and user profiles. All tables are designed to maintain ACID integrity and relationship constraints, identification mechanisms and data validation rules will be enforced. Image files used for disease detection will be stored in cloud storage, only the file path and classification outputs will be stored in the database.

The system relies on ESP-NOW for node-to-gateway communication and Wi-Fi/HTTPS for gateway-to-backend communication. Each sensor node collects soil-moisture and environmental telemetry and transmits the data to a dedicated ESP32-based gateway. The gateway acts as an intermediate communication unit between the distributed sensor nodes and the cloud backend, aggregating incoming field data and forwarding it via Wi-Fi. This design improves communication organization, reduces the need for every node to connect directly to the cloud and supports reliable periodic sensor updates and system monitoring.

To ensure continuity and data integrity in rural agricultural environments, the system uses device-level retry and buffering mechanisms. Sensor nodes temporarily store measurements when transmission to the ESP32-based gateway fails and resend buffered data once the connection is restored. The gateway then forwards validated telemetry to the cloud backend over secure HTTP APIs via Wi-Fi, using encrypted channels such as HTTPS to protect data confidentiality and integrity. On the cloud side, received data are validated, stored, and made available to analytics, ML, and visualization services. This architecture provides a secure, reliable, and scalable data flow from field-level sensors to cloud intelligence.

2.3.1.3. System and Backend Requirements

The system’s core database will be implemented using PostgreSQL, providing a robust relational foundation for structured agricultural data management[41].

To efficiently handle sensor telemetry, the architecture supports the use of the TimescaleDB extension, which may be enabled to improve performance for time-based queries, historical analysis and aggregation-heavy workloads[42]. When utilized, this extension facilitates efficient storage, partitioning and compression of sensor data. When spatial field analysis is required, the PostGIS extension can be enabled to support geospatial queries and region-based operations, ensuring seamless integration with the digital twin component[43].

The TARAS backend architecture will utilize Node.js as the server-side runtime environment for managing RESTful APIs, processing incoming sensor data and coordinating interactions between the mobile app, database services and ML modules. The backend exposes RESTful API endpoints that allow users to access digital twin maps, receive irrigation recommendations, upload plant images for disease detection and receive system notifications.

2.3.1.4. Software Interface Requirements

The TARAS software architecture uses coordinated interfaces between the backend, mobile app, database, and ML components to support irrigation recommendation, disease detection, and LLM-based advisory functions. These interfaces rely on defined API contracts for exchanging structured sensor data, user inputs, images, and model outputs.

The irrigation module accesses time-series sensor data, environmental conditions and past irrigation records from the cloud database to generate recommendation objects that include irrigation need, timing, duration and explainability indicators. The disease-detection module receives compressed JPEG/PNG leaf images from the mobile app through the backend and returns a classification label, confidence score and explanation map when available.

The LLM advisory module uses recent sensor readings, irrigation recommendations, disease results and user-submitted information to produce natural-language explanations, warnings or scenario-based advice. The mobile app communicates with these services through designated API endpoints, while all interfaces remain compatible with the PostgreSQL-backed data layer and support reliable synchronization between sensor nodes, gateway, cloud, and mobile app.

2.3.1.5. User Interface Requirements

The TARAS mobile app is the main interaction layer between farmers and the system's sensing, recommendation, and visualization components. Its interface must be responsive across different screen sizes, visually

consistent, and capable of presenting model outputs as clear, actionable insights.

The app uses a persistent navigation bar for primary screens such as Overview and Irrigation Suggestion, Timetable, Disease Detection, Carbon Footprint Calculation and Account. A global floating button provides access to the AI assistant, allowing users to ask natural-language questions. Access is managed through role-based login, which adjusts permissions and data visibility for farmers and stakeholders.

The central dashboard functions as a digital twin, displaying interactive 2D/3D field visualizations with real-time soil moisture and temperature data mapped to color-coded zones. Users can rotate the view, select specific zones and receive visual alerts for conditions such as drought stress or over-irrigation. The disease detection module supports camera capture and image upload for plant disease classification. The timetable screen shows irrigation and fertilization tasks and allows users to enter fuel, energy, and fertilizer data for carbon-footprint calculations. Navigation must remain simple and consistent across Android and iOS devices.

2.3.2. Detailed Description of Functional Requirements

2.3.2.1. Microcontroller - Sensor Interface

Name	Microcontroller - Sensor Interface
Purpose / Description	The sensor interface enables reliable acquisition of soil moisture, temperature, humidity and environmental data from IoT sensor nodes. Each ESP32-C6 node reads SHT31 sensors via I²C and the soil-moisture probe via ADC. The interface ensures proper electrical levels (3.0–3.6 V), periodic sampling and noise-resistant communication.
Inputs	● I²C readings from SHT31 temperature-humidity sensor ● ADC readings from capacitive soil-moisture sensor ● I²C/ADC configuration parameters (address, bus speed, ADC resolution, sampling rate) ● Regulated 3.0–3.6 V power input
Processing	1. Initialize I²C bus and ADC at boot 2. Perform periodic sampling of all sensors 3. Verify sensor presence via I²C acknowledgment 4. Apply calibration and scaling to raw readings 5. Filter noisy values using smoothing/averaging

Name	Microcontroller - Sensor Interface
	6. Package processed data into ESP-NOW packet payload 7. Attach sampling-cycle or relative time information for backend timestamp reconstruction
Outputs	● Validated sensor data (soil moisture, temperature and humidity) ● Structured telemetry payloads for ESP-NOW transmission to the gateway ● Sensor status indicators (active, missing, calibrated)
Error Handling	● Retry I²C communication on missing acknowledgment ● Mark and suppress physically invalid readings ● Reset I²C bus after collisions or noise ● Suspend sampling under low-voltage conditions ● Log diagnostic flags locally for cloud-side monitoring and debugging

2.3.2.2. Gateway Communication and Management Interface Functional Requirement

Name	Gateway Communication and Management Interface
Purpose / Description	The gateway hardware interface enables reliable communication between ESP32-C6 sensor nodes and the cloud backend. The gateway acts as a bridge between low-power sensor nodes using ESP-NOW and the backend services using Wi-Fi. It receives signed sensor packets from paired nodes, verifies communication integrity, forwards telemetry to the backend over HTTPS, supports temporary local buffering during connectivity loss, and manages node pairing, configuration updates, diagnostics, and OTA firmware updates.
Inputs	● ESP-NOW packets from paired ESP32-C6 sensor nodes ● Sensor telemetry packets containing soil moisture, temperature, humidity and diagnostic data ● Gateway Wi-Fi credentials and backend API key ● Node pairing requests and approval/rejection commands from the backend ● Node configuration parameters such as sleep interval, buffer cycles and sensor thresholds ● OTA firmware update commands and firmware download URL ● System time and gateway runtime status ● Local SPIFFS storage for temporary buffering

Name	Gateway Communication and Management Interface
Processing	1. Initialize Wi-Fi, ESP-NOW, BLE provisioning and local storage at startup 2. Receive telemetry and diagnostic packets from paired sensor nodes via ESP-NOW 3. Verify packet authenticity using HMAC-based validation and replay protection counters 4. Acknowledge valid sensor packets to prevent unnecessary retransmission 5. Convert raw sensor packets into backend-compatible JSON payloads 6. Forward telemetry to the backend through HTTPS using the device key 7. Maintain a secure Socket.IO connection with the backend for gateway commands 8. Buffer unsent telemetry locally when Wi-Fi or backend connectivity is unavailable 9. Flush buffered telemetry once connectivity is restored 10. Support BLE-based gateway provisioning for Wi-Fi credentials and API key setup 11. Manage sensor node pairing through backend-controlled approve/reject workflow 12. Push updated node configuration to paired sensor nodes 13. Send heartbeat and diagnostic status to the backend 14. Process OTA firmware update commands and install validated firmware updates
Outputs	● Backend-delivered sensor telemetry ● ESP-NOW acknowledgments sent to sensor nodes ● Gateway heartbeat and online/offline status ● Node pairing status and paired device keys ● Buffered telemetry synchronization results ● Diagnostic logs and gateway health indicators ● OTA update status
Error Handling	● Retry Wi-Fi connection when network connectivity is lost ● Buffer telemetry in local storage when backend transmission fails ● Flush buffered records after reconnection ● Reject invalid or unsigned ESP-NOW packets ● Ignore duplicate or replayed packets using packet counters ● Report gateway heartbeat, packet statistics and diagnostic information to the backend ● Handle BLE provisioning failures by returning status codes to the

Name	Gateway Communication and Management Interface
	mobile app ● Abort or report OTA update failure if firmware download or installation fails ● Reset or restart gateway operation when unrecoverable communication errors occur

2.3.2.3. Node-to-Gateway Communication – Functional Requirement

Name	Node-to-Gateway Communication
Purpose / Description	Enables secure and reliable transmission of sensor telemetry from ESP32-C6 sensor nodes to the gateway using ESP-NOW. Sensor nodes collect soil moisture, temperature, humidity, and diagnostic data, package the readings into structured packets, and transmit them to the paired gateway. The gateway acknowledges valid packets, allowing the node to minimize unnecessary retransmissions and preserve battery life.
Inputs	● Validated sensor readings generated by the ESP32-C6 node ● Node identifier and pairing credentials ● Packet counter or sequence number ● Sensor diagnostics such as battery level and runtime status ● ESP-NOW communication status ● System timestamp or local sampling cycle information
Processing	1. Initialize sensor interfaces and ESP-NOW communication at startup 2. Collect soil moisture, temperature, humidity, and diagnostic data 3. Validate sensor readings before transmission 4. Format telemetry into a structured ESP-NOW packet 5. Attach node identifier, timestamp or cycle number, and packet counter 6. Sign or authenticate the packet before transmission 7. Send telemetry packets to the paired gateway using ESP-NOW 8. Wait for gateway acknowledgment after transmission 9. Retry transmission if acknowledgment is not received 10. Temporarily store unsent readings when communication fails 11. Enter low-power or deep-sleep mode after successful transmission or retry completion
Outputs	● ESP-NOW telemetry packets delivered to the gateway

Name	Node-to-Gateway Communication
	● Gateway acknowledgment received by the sensor node ● Updated local transmission status ● Buffered telemetry records waiting for retransmission ● Sensor node diagnostic information sent to the gateway
Error Handling	● Retry ESP-NOW transmission when acknowledgment is not received ● Buffer unsent telemetry when gateway communication fails ● Discard or flag invalid sensor readings before transmission ● Prevent duplicate packet processing using packet counters ● Resume transmission in the next sampling cycle if communication remains unavailable ● Report low battery or sensor fault status through diagnostic packets

2.3.2.4. Power Interface Functional Requirement

Name	Power Interface
Purpose / Description	The power interface ensures stable and efficient energy delivery to ESP32 sensor nodes and connected sensors using a 3.7 V Li-Ion/Li-Po battery regulated to safe operating levels. It supports low-power modes, deep-sleep operation and voltage stability to maximize battery lifetime. The interface also prevents undervoltage, brownout condition, and power-related data loss.
Inputs	● 3.7 V Li-Ion / Li-Po battery input ● Regulated 3.0–3.6 V output for ESP32-C6 and sensors ● Power-management configuration (sleep intervals, wake-up triggers) ● Voltage and current readings from onboard regulators
Processing	1. Regulate battery voltage to component-safe operating levels 2. Manage deep-sleep and low-power operational modes of the ESP32 3. Monitor voltage stability to detect undervoltage and brownout conditions 4. Allocate power to sensors and communication subsystems based on sensing and ESP-NOW transmission cycles 5. Log power-related events (sleep, wake-up, low-voltage warnings)

Name	Power Interface
Outputs	● Stable power delivery to the microcontroller and sensors ● Power-state indicators (active, sleep, low voltage) ● Logged power-performance data for system diagnostics
Error Handling	● Enter safe low-power mode under undervoltage conditions ● Suspend sensing and wireless transmission when power becomes unstable ● Trigger automatic system restart after a detected brownout event ● Log critical power faults for cloud-side monitoring and analysis

2.3.2.5. Environmental / Enclosure Interface Functional Requirement

Name	Environmental / Enclosure
Purpose / Description	The environmental and enclosure interface defines the physical protection requirements for TARAS sensor nodes and gateway devices deployed in field or greenhouse conditions. The enclosure shall protect internal electronics from dust, moisture, accidental contact, cable strain and moderate outdoor exposure, while allowing sensors to collect accurate environmental and soil measurements.
Inputs	● Outdoor or greenhouse environmental exposure such as dust, humidity, rain splash, sunlight and temperature variation ● Mechanical stress caused by handling, mounting or field maintenance ● Cable entry points for soil-moisture probes, power lines and sensor connections ● Thermal load caused by sunlight or enclosed operation ● Sensor placement requirements for accurate air temperature, humidity and soil-moisture readings
Processing	1. Use a weather-resistant enclosure suitable for outdoor or greenhouse deployment 2. Protect electronic components from direct water ingress and dust exposure 3. Use sealed or strain-relieved cable openings for sensors and power connections 4. Position air temperature and humidity sensors so they are protected from direct water contact while still exposed to representative airflow

Name	Environmental / Enclosure
	5. Keep soil-moisture probes externally accessible for insertion into soil or pots 6. Provide physical separation between power, microcontroller and sensor wiring to reduce accidental short circuits 7. Support ventilation or passive heat dissipation where needed to avoid overheating 8. Use corrosion-resistant connectors or protective coating where long-term humidity exposure is expected
Outputs	● Protected ESP32 sensor node and gateway electronics ● Stable physical mounting of sensor and gateway units ● Reliable sensor placement for field or greenhouse measurements ● Reduced risk of dust, moisture, cable strain and accidental contact damage
Error Handling	● If enclosure damage, water ingress or connector corrosion is detected during maintenance, the device shall be marked for inspection or replacement ● If abnormal sensor readings suggest environmental damage or disconnection, the system shall report the affected sensor as faulty ● If overheating or moisture exposure is suspected, the device shall be removed from operation until maintenance is completed

2.3.2.6. Store-and-Forward Communication Reliability Functional Requirement

Name	Store-and-Forward Communication Reliability
Purpose / Description	Ensures reliable telemetry delivery in environments with intermittent ESP-NOW, Wi-Fi, or backend connectivity by using buffering at both the ESP32-C6 sensor node and gateway levels. Sensor nodes temporarily store unsent readings when gateway acknowledgments are missing, while the gateway stores backend-bound payloads when Wi-Fi or cloud access is unavailable.
Inputs	● Sensor data awaiting transmission ● ESP-NOW acknowledgment status between sensor node and gateway

Name	Store-and-Forward Communication Reliability
	● Gateway Wi-Fi connectivity status ● Cloud backend availability status ● Sensor node RTC memory and gateway SPIFFS storage resources ● Packet metadata (timestamps, sequence numbers)
Processing	1. Detect missing ESP-NOW acknowledgments between sensor nodes and the gateway. 2. Store unsent telemetry in sensor node RTC memory when gateway delivery fails. 3. Detect failed Wi-Fi or backend transmission attempts at the gateway. 4. Store unsent backend payloads in gateway SPIFFS storage. 5. Retransmit buffered sensor-node packets in the next sampling cycle. 6. Flush gateway-buffered telemetry once backend connectivity is restored. 7. Verify successful backend acknowledgment before deleting buffered records. 8. Filter duplicate packets using timestamps or sequence identifiers.
Outputs	● Reliable, lossless delivery of sensor telemetry to the cloud backend ● Updated synchronization logs indicating queued, transmitted, and retried packets ● Continuous data flow despite connectivity interruptions
Error Handling	● Apply retry and backoff strategies when repeated transmission failures occur ● Manage buffer overflow by discarding or overwriting the oldest data based on policy ● Log cache write/read errors for diagnostics ● Validate buffered packets before retransmission to prevent data corruption

2.3.2.7. Sensor Telemetry Storage and Management Functional Requirement

Name	Sensor Telemetry Storage and Management
Purpose / Description	Defines the system's responsibility to receive, validate, store and provide access to sensor telemetry data collected from TARAS sensor nodes through the gateway and backend services. The system stores soil moisture, temperature, humidity, battery and diagnostic-related measurements in PostgreSQL and exposes them through backend APIs for dashboard visualization, irrigation recommendation and historical analysis.
Inputs	● Incoming telemetry packets from gateway or registered sensor devices: ● soil moisture raw value ● calibrated soil moisture percentage ● temperature ● humidity ● battery voltage or battery level ● node_id or device key ● timestamp or relative sampling time ● sensor error flags, when available
Processing	1. Validate incoming telemetry structure and required authentication headers. 2. Authenticate the device using its registered device key. 3. Convert relative sampling times into backend timestamps. 4. Apply soil-moisture calibration using stored dry/wet calibration values when available. 5. Store validated readings in the sensor_readings table. 6. Associate each reading with the correct sensor node, zone, field and farm hierarchy. 7. Index telemetry by node identifier and timestamp for efficient latest-reading and history queries. 8. Provide APIs for: ○ latest readings by zone, ○ historical readings by zone, ○ historical readings by node, ○ field-level sensor history. 9. Forward latest readings to connected clients using real-time socket updates. 10. Use stored telemetry as input for irrigation recommendation and dashboard modules.

Name	Sensor Telemetry Storage and Management
Outputs	● Stored telemetry records in PostgreSQL. ● Latest sensor readings available for dashboard display. ● Historical sensor datasets available through REST APIs. ● Zone and field-level telemetry history for visualization and analysis. ● Real-time sensor updates emitted to the mobile application. ● Telemetry input available for irrigation recommendation logic.
Error Handling	● If the device key is missing or invalid, reject the telemetry request. ● If the readings array is missing or empty, reject the request with a validation error. ● If a reading contains sensor error flags, store valid fields and report sensor failure through alerts. ● If soil-moisture calibration values are missing, store raw soil-moisture values and leave calibrated percentage unavailable. ● If the database write operation fails, return an error and rely on gateway or node buffering for retransmission. ● If telemetry belongs to an unknown node, reject or ignore the record and log the event.

2.3.2.8. Disease Detection Database Recording Functional Requirement

Name	Disease Detection Database Recording
Purpose / Description	Stores the output of the disease detection module, including image storage key, predicted disease class, confidence score, prediction distribution, processing status and timestamp, inside the disease_detections table in PostgreSQL.
Inputs	● Classification result from CNN model ● Cloud storage URL of user-uploaded image ● User ID (farmer) ● Device and field metadata
Processing	1. Validate disease result schema. 2. Insert record into disease_detections table. 3. Store prediction status and confidence information. 4. Make disease history accessible through backend APIs.
Outputs	● Database record created in disease_detections.

Name	Disease Detection Database Recording
	● Disease history accessible via API. ● Classification status available to the mobile application.
Error Handling	● If the database write operation fails, the system performs an automatic retry before reporting an error. ● If the classification output is missing, the system records the entry with the label “unidentified.”

2.3.2.9. Digital Twin Data Assembly Functional Requirement

Name	Digital Twin Data Assembly
Purpose / Description	Supports the TARAS Digital Twin by storing field geometry, zone definitions and sensor-node placement data, then assembling these records with the latest telemetry for visualization in the mobile application. The system enables users to create greenhouse or pot-area field layouts, define zones, associate sensor nodes with zones and render a 2D/3D digital representation of the agricultural area.
Inputs	● Field boundary polygon generated by the mobile field creation flow ● Zone polygons or pot-area zone definitions ● Sensor node placement coordinates ● Field type, such as greenhouse or pot area ● Crop and planting metadata ● Latest soil moisture, temperature and humidity readings ● Sensor node status and battery level
Processing	1. Store field boundary and zone geometry as structured polygon data. 2. Associate each field with a farm and each zone with a field. 3. Associate sensor nodes with their assigned zones. 4. Store sensor node display coordinates for digital twin visualization. 5. Retrieve the latest telemetry for each sensor node. 6. Assemble field, zone, node and telemetry data into a structured dashboard dataset. 7. Calculate zone-level display values using the latest available sensor readings. 8. Prepare data for 2D/3D field rendering and heatmap-like moisture visualization in the mobile interface. 9. Support greenhouse layouts with polygon-based zones.

Name	Digital Twin Data Assembly
	10. Support pot-area layouts with individual pot or virtual-node based representations.
Outputs	● Structured digital twin dataset for the mobile dashboard. ● Field boundary and zone layout data. ● Sensor node positions and status information. ● Latest moisture, temperature and humidity values per node or zone. ● Visual data used for 2D/3D field rendering. ● Zone-level overview for dashboard and irrigation screens.
Error Handling	● If a field polygon has fewer than three points, reject the field creation request. ● If required field or zone geometry is missing, return a validation error. ● If a sensor node has no assigned zone, display it as unassigned or exclude it from zone-level visualization. ● If latest telemetry is unavailable, show the sensor node with an empty or stale-data state. ● If a user requests a field that does not belong to them, deny access. ● If geometry or dashboard data cannot be assembled, return an error and keep the previous valid UI state when available.

2.3.2.10. Disease Detection Functional Requirement

Name	Disease Detection Functional Requirement
Purpose / Description	Enables users to upload plant leaf images and receive disease classification using the system's CNN-based image-processing module. The module analyzes images, detects disease type, generates confidence scores and provides recommendations.
Inputs	● User-uploaded plant leaf image. ● Optional metadata: crop type, field zone, timestamp. ● Pretrained CNN weights and preprocessing parameters. ● Stored disease classes in the database.
Processing	1. Validate image size, format, and resolution. 2. Preprocess image (resize, normalize, color correction). 3. Run CNN inference to classify disease type. 4. Store classification output into PostgreSQL disease_detections table with image path and prediction metadata. 5. Forward result to mobile application for visualization.

Name	Disease Detection Functional Requirement
Outputs	● Disease label ● Confidence value. ● API response to the mobile client. ● Database entry containing: ○ image_url ○ classification ○ confidence ○ timestamp
Error Handling	● If the image format is invalid, the system returns an “unsupported file format” error. ● If the image file is corrupted, the system rejects the upload and records it in the logs. ● If the inference engine is unavailable, the system attempts a retry and then returns a error if the issue persists. ● If inserting the result into the database fails, the system logs the failure and returns a partial-success response to the client. ● If metadata is missing, the system proceeds using default assumptions defined by the backend.

2.3.2.11. Irrigation Recommendation Request & Response Handling Functional Requirement

Name	Irrigation Recommendation Request & Response
Purpose / Description	Generates irrigation recommendations using a rule-based decision engine that evaluates recent sensor readings, crop growth stage, field type, irrigation mode and calibrated soil-moisture thresholds. The system produces zone-level irrigation decisions for greenhouse or pot-area environments and stores recommendation results for later tracking and follow-up analysis.
Inputs	● Latest soil moisture readings from sensor nodes ● Temperature and humidity readings ● Zone, field and farm metadata ● Field type, such as greenhouse or pot area ● Irrigation mode, such as manual or duration-based irrigation ● Crop type and planting date ● Crop growth stage or calculated growth stage from planting date ● Configured soil-moisture thresholds ● Calibration parameters such as:

Name	Irrigation Recommendation Request & Response
	● ml_per_sm_percent ● sm_percent_per_100ml ● irrigation_gain_mm_per_100ml ● irrigation_gain_mm_per_10_min ● Historical irrigation jobs and follow-up results
Processing	1. Retrieve the latest valid sensor readings for the selected zone. 2. Calculate average soil moisture, temperature and humidity from available sensor nodes. 3. Determine the active planting and crop growth stage. 4. Select rule thresholds according to crop growth stage. 5. Evaluate current soil moisture against recommended and critical thresholds. 6. Determine whether irrigation is required. 7. For pot-area fields, calculate recommended water amount in milliliters. 8. For greenhouse fields, calculate recommended irrigation duration when applicable. 9. Apply safety limits to prevent excessive irrigation recommendations. 10. Use calibration data and historical follow-up results to improve recommendation accuracy. 11. Create and store an irrigation job with recommendation status, reasoning and calculated values. 12. Prevent duplicate active recommendations by skipping or replacing older pending jobs when a newer recommendation is generated. 13. Track actual irrigation outcomes submitted by the user. 14. Schedule follow-up checks after irrigation to compare predicted and observed soil-moisture change. 15. Store follow-up analysis for future calibration.
Outputs	● JSON recommendation object including: ● irrigation decision ● recommended water amount or duration ● current soil moisture ● target soil moisture ● soil-moisture deficit ● urgency level ● reasoning text ● suggested follow-up check time ● Stored irrigation job record. ● Updated irrigation history for the selected zone.

Name	Irrigation Recommendation Request & Response
	● Follow-up records comparing predicted and actual soil-moisture changes.
Error Handling	● If no sensor node is found for the selected zone, return an unavailable recommendation error. ● If no valid sensor reading is available, return an unavailable recommendation error. ● If crop, field type, irrigation mode or growth stage information is missing, mark the irrigation job as failed and store the reason. ● If unsupported field type or irrigation mode is provided, reject the recommendation request. ● If required calibration parameters are missing, return an error explaining the missing configuration. ● If a previous executed irrigation job is still waiting for follow-up, delay generating a new recommendation. ● If database write or recommendation generation fails, log the error and create a failed irrigation job record when possible.

2.3.2.12. Role-Based Access Control & UI Authorization Functional Requirement

Name	Role-Based Access Control & UI Authorization
Purpose/ Description	Defines role-based access rules for the TARAS mobile interfaceTARAS mobile and web interfaces. The system controls which screens, features and data are visible or accessible to users based on their assigned role.
Inputs	● Authenticated User Session ● User Role ● Requested UI Screen or Feature ● Navigation Events
Processing	1. Role Identification: Determine the user's role after successful authentication. 2. Access Evaluation: Check requested screen or feature against role-based permission rules. 3. UI Rendering: Display only the interface components permitted for the user's role. 4. Navigation Filtering: Hide or disable navigation items that are not allowed for the current role. 5. Session Enforcement: Enforce access rules during navigation and

Name	Role-Based Access Control & UI Authorization
	screen transitions.
Outputs	● Role-appropriate UI layout. ● Accessible navigation menu based on user role. ● Authorized access to data, visualizations and actions.
Error Handling	● If a user attempts to access a restricted screen, redirect to an authorized default screen. ● If role information is missing or invalid, terminate the session and require re-authentication. ● If permission rules fail to load, restrict access to read-only or safe default views.

2.3.2.13. Accessibility & Inclusive UI Support Functional Requirement

Name	Accessibility & Inclusive UI Support
Purpose / Description	Defines accessibility and usability support for the TARAS mobile interface to improve readability, touch interaction and visual clarity for farmers using the application in field or greenhouse conditions. The interface shall support light and dark themes, responsive sizing and clear feedback states to provide a usable experience across different devices and environments.
Inputs	● System theme preference ● User-selected theme preference ● Device screen size and safe-area constraints ● User interaction events ● UI rendering context
Processing	1. Apply light, dark or system-based theme selection across the mobile application. 2. Use responsive spacing and sizing utilities to adapt layouts to different screen sizes. 3. Maintain readable text hierarchy for dashboard, disease detection, irrigation, settings and carbon footprint screens. 4. Use visually clear buttons, icons and navigation items for primary actions. 5. Provide immediate visual feedback for user interactions, including pressed, loading, selected, empty and error states. 6. Use safe-area handling to prevent important controls from

Name	Accessibility & Inclusive UI Support
	overlapping with device notches, home indicators or system UI. 7. Provide localized interface text in Turkish and English where translations are available. 8. Preserve usability when live data is unavailable by showing loading, empty or stale-data states.
Outputs	● Readable and theme-aware mobile UI. ● Responsive layouts for supported mobile screen sizes. ● Clear navigation and action controls. ● Visible loading, empty, error and selected states. ● Localized UI text for supported languages.
Error Handling	● If the selected theme cannot be loaded, fall back to the system or default theme. ● If localized text is unavailable, fall back to the default language. ● If a screen cannot load live data, display a safe empty, loading or cached-data state. ● If rendering fails in a complex visual component, show a fallback UI instead of crashing the application.

2.3.2.14. Mobile Application Framework & Navigation Functional Requirement

Name	Mobile Application Framework & Navigation
Purpose / Description	Defines the initialization, localization and navigation structure of the TARAS application. It establishes the primary user interface shell, ensuring consistent navigation and language support across Android and iOS devices.
Inputs	● System Language Settings (Locale) ● System Theme Preference (Light/Dark) ● User Credentials (Login/Skip) ● Touch Interaction Events
Processing	1. Localization: Apply Turkish (TR) as the default language and allow English (EN) when explicitly selected by the user. 2. Theme Application: Apply the color schema (Light/Dark) based on system settings or user override. 3. Authentication Routing: Direct user to Login Screen or load main dashboard if a valid session token exists.

Name	Mobile Application Framework & Navigation
	4. Navigation Mounting: Render the persistent bottom navigation bar including carbon footprint, timetable, home dashboard, disease detection and settings screens. 5. Overlay Initialization: Attach the Global Floating Action Button (FAB) for the AI Assistant to the root view.
Outputs	● Localized Main UI Shell. ● Active Navigation Stack. ● Visual Theme (Colors/Fonts).
Error Handling	● If the system language is not supported, automatically fallback to Turkish. ● If a screen fails to mount, reset the navigation stack to the dashboard.

2.3.2.15. Carbon Footprint Calculation & Display

Name	Carbon Footprint Calculator
Purpose / Description	The system shall provide a carbon footprint tracking feature that calculates agricultural CO2e emissions for a farm based on user-entered activity records and predefined emission factors. The calculated emissions shall be stored as carbon logs, associated with the selected farm, and displayed to the user through the mobile application as total and category-based summaries.
Inputs	● Selected farm identifier ● Activity type selected by the user ● Activity category, such as fuel, fertilizer or electricity ● Activity amount ● Activity unit ● Activity date ● Optional notes ● Emission factor associated with the selected activity type and date range
Processing	1. Display a Carbon Footprint section in the mobile application. 2. Retrieve active activity types grouped by category. 3. Allow the user to select an activity category and activity type. 4. Collect activity amount, date and optional notes.

Name	Carbon Footprint Calculator
	5. Validate that the selected farm, activity type and activity amount are provided. 6. Reject invalid activity amounts such as zero, negative or non-numeric values. 7. Retrieve the applicable emission factor for the selected activity type and activity date. 8. Calculate emission amount using: a. emission_amount = activity_amount × emission_factor 9. Store the result in the carbon_logs table with: a. Farm_id b. Activity_type_id c. Activity_date d. Activity_amount e. Emission_amount f. Notes g. created_at 10. Provide farm-level carbon summaries by aggregating stored carbon logs. 11. Group summary results by activity category. 12. Allow users to view recent carbon activity logs. 13. Allow users to delete carbon logs that belong to their farm. 14. Support admin-side management of emission factors where applicable.
Outputs	● Stored carbon log record. ● Calculated emission amount for the submitted activity. ● Total farm carbon emission summary. ● Category-based emission breakdown. ● Recent carbon activity log list. ● Yearly total CO2e value when requested. ● Mobile UI display of carbon footprint totals and logs.
Error Handling	● If the selected farm does not belong to the authenticated user, deny

Name	Carbon Footprint Calculator
	access. ● If the activity type is missing or invalid, reject the request. ● If the activity amount is missing, zero, negative or non-numeric, reject the request. ● If no emission factor exists for the selected activity type and date, return an error. ● If database write fails, return an error and do not mark the activity as saved. ● If carbon logs cannot be loaded, display an empty or error state in the mobile application. ● If a user attempts to delete a carbon log outside their farm, reject the request.

2.3.3. Data Preprocessing

2.3.3.1. Irrigation Data Preprocessing

2.3.3.1.1. Purpose and Data Sources

The irrigation recommendation module uses consistent and time-aligned sensor, crop, irrigation, and weather-related data to estimate crop water demand for both greenhouse zones and pot-based cultivation units. In greenhouse cultivation, each zone is evaluated separately according to its sensor readings and crop metadata. In pot-based cultivation, each pot is treated as an individual zone, allowing irrigation needs to be calculated independently.

When ET_0 and Kc data are available, the system calculates crop evapotranspiration as $ETc = ET_0 \times Kc$, where ET_0 represents atmospheric water demand and Kc reflects crop type and growth stage. Based on ETc-derived water demand and observed soil moisture dynamics, the system estimates when irrigation should begin and how long it should continue by converting required water into irrigation duration using calibration and irrigation gain parameters.

Before irrigation calculations, the system shall process the following inputs:

Weather / Environmental Data: temperature, relative humidity, wind speed, or other available ET_0-related variables.

Sensor Data: soil moisture, air temperature, and humidity from greenhouse zones or individual pots.

Crop Metadata: crop type, planting date, growth stage, and Kc profile parameters.

Irrigation Records: previous irrigation events, applied duration, water amount and calibration data.

2.3.3.1.2. Timestamp Standardization and Time Alignment

The system shall convert all timestamps to a unified timezone and format. Sensor, weather, and irrigation records shall be aligned on a common timeline using a fixed interval, such as hourly or daily, so that greenhouse-zone and pot-level data can be processed consistently.

2.3.3.1.3. Sensor Quality Control

The system shall apply quality controls to reduce noise, spikes, and long-term drift in soil moisture measurements. Unrealistic spikes shall be excluded, smoothing shall reduce noise without hiding real irrigation effects, and baseline drift shall be monitored to generate sensor health indicators. These controls shall apply to both greenhouse sensors and pot-level sensors before missing-value handling.

2.3.3.1.4. Missing Data Analysis and Handling

The system shall calculate missing-value ratios for each input and generate a missing-data report. Short gaps in weather or sensor data shall be interpolated, while extended gaps in ET_0-critical variables shall suspend ETc-based calculations. For soil moisture data, short gaps shall be interpolated, and long gaps shall reduce recommendation confidence or trigger fallback estimation based on available environmental data.

2.3.3.1.5. Context-Aware Outlier Detection

The system shall use context-aware outlier detection instead of generic IQR filtering. Meteorological outliers shall be evaluated statistically, while soil moisture outliers shall be checked using irrigation-aware rules. This allows the system to distinguish real moisture increases caused by irrigation from sensor anomalies in both greenhouse zones and pots.

2.3.3.1.6. Crop Coefficient and Growth Profile Preparation

The system shall generate a continuous Kc time series by loading crop-specific Kc profiles and mapping planting dates to a day-after-planting index. For greenhouse zones, Kc values shall be applied according to the active crop in each zone. For pot-based cultivation, Kc values shall be applied individually based on the crop assigned to each pot. If calibrated Kc parameters are available, versioned calibrated values shall be used consistently.

2.3.3.1.7. Derived Features for Irrigation Logic

The system shall compute ETc as $ET_0 \times Kc$ when the required data are available and derive cumulative ETc over a configurable rolling window. In addition, soil moisture trends, moisture deficit, irrigation history, and post-irrigation sensor changes shall be used to support recommendation decisions. These derived features enable zone-specific irrigation recommendations for greenhouses and pot-specific recommendations for individual cultivation units.

2.3.3.2. Image Data Preprocessing for Disease Detection

The disease detection module relies on accurate, noise-reduced, and consistently formatted image data to ensure model stability, high inference accuracy and reproducible visual diagnostics. To support both training and inference, the system shall apply a structured preprocessing pipeline to all plant leaf images captured/uploaded through the TARAS app. This pipeline ensures that the CNN-based disease classifier operates on standardized, clean and meaningful visual inputs.

2.3.3.2.1. Raw Image Validation

The system shall validate the integrity and format of each incoming image before performing any analytical operation. Validation includes checking file format (JPEG/PNG), verifying that file size does not exceed predefined limits, ensuring that the image is not corrupted, confirming minimum acceptable resolution for classification. Images failing any of these checks shall be rejected and logged for quality assurance.

2.3.3.2.2. Image Resizing and Aspect-Ratio Normalization

In order to maintain consistency across the CNN input pipeline, the system shall resize all images to the model's expected input

dimensions, preserve aspect ratio when possible and apply center-cropping/padding to prevent distortion. This ensures that all images conform to the numerical structure required by the model.

2.3.3.2.3. Color Space Standardization

To ensure consistent lighting interpretation and feature extraction, the system shall convert all images to a standardized color space. Operations include converting raw images to RGB and normalizing pixel intensity ranges to 0–1 (depending on model specification), This enables reproducible visual feature extraction across devices.

2.3.3.2.4. Noise Reduction

Mobile field images often contain shadows, glare or compression artifacts. The preprocessing pipeline shall apply optional filters such as Gaussian blur (low-intensity smoothing), median filtering for salt-and-pepper noise and contrast normalization for uneven lighting conditions. These operations reduce noise while preserving disease-relevant texture patterns.

2.3.3.2.5. Leaf Region Isolation

The system performs leaf segmentation to isolate disease-relevant regions. Methods include color-threshold masking (green-channel analysis), background removal and contour detection. Isolating the leaf region reduces false positives caused by soil, sky, or human-made background elements.

2.3.3.2.6. Data Augmentation for Model Generalization

To improve model robustness and prevent overfitting, the training pipeline shall apply controlled augmentation techniques, such as random rotations and flips, controlled brightness/contrast adjustments, and slight zoom or crop transformations. Augmentation must not distort disease-critical features.

2.3.3.2.7. Image Quality Assessment Checks

Before inference or training, the system shall evaluate image quality using criteria such as blur detection (Laplacian variance), lighting adequacy, obstruction analysis (e.g., partially visible leaves). Images failing minimum quality thresholds shall be flagged or optionally rejected.

2.3.3.2.8. Dataset Class Distribution Documentation

To support fairness and balanced classification accuracy, the system shall compute and store the distribution of disease classes

in the annotated dataset. This documentation includes number of images per disease category, identification of underrepresented diseases, recommendations for synthetic augmentation or targeted data collection.

2.3.3.2.9. Normalization for Inference

During inference, the system shall apply the same normalization parameters used during training, specifically mean subtraction, standard deviation division, reordering to the model's expected tensor structure. Consistency between training and inference pipelines is mandatory for reliable performance.

2.3.4. Non-Functional Requirements

2.3.4.1. Performance Requirements

Under normal operating conditions, sensor data is expected to become available to the backend and mobile application within 30 seconds to 1 minute after collection. The mobile application is expected to respond to user interactions such as tab navigation, data loading and dashboard updates within 1–2 seconds. Irrigation analysis and recommendation generation is expected to complete within a few seconds after relevant data becomes available. Disease-detection results are expected to be delivered within 5–10 seconds following image upload, depending on network conditions. In cases of limited or unstable connectivity, the system is expected to immediately display cached or last-known data to maintain usability.

2.3.4.2. Reliability Requirements

The system is expected to work reliably in real agricultural environments where network interruptions and temporary component issues may occur. Under normal conditions, sensor data is expected to be successfully delivered for around 95–98% of the time and any missing data being limited to short gaps. If communication is temporarily unavailable, the system is expected to continue using the last known data for up to 10–30 minutes until new measurements arrive. Once connectivity is restored, data synchronization between field devices and the backend is expected to resume automatically within a few minutes. Any data loss is expected to be minimal and occasional rather than continuous. In case of temporary failures in sensor nodes, gateways or backend services, the system is expected to keep operating in a reduced but stable state and return to normal operation automatically when the issue is resolved.

2.3.4.3. Availability Requirements

The system is expected to remain available for regular use throughout the agricultural season. Backend services are expected to achieve an overall

availability of around 97–99% under normal conditions. The application is expected to be accessible whenever the user's device is operational and core features remain available even during temporary outages. Planned maintenance or short service interruptions are expected to be infrequent and limited in duration, usually not exceeding a few minutes at a time. In the event of connectivity loss, the system is expected to continue providing access to previously synchronized data. Once connectivity is restored, the system is expected to resume normal operation automatically without requiring user intervention.

2.3.4.4. Scalability Requirements

The system shall scale gradually as the number of fields and deployed sensor nodes increases. Initial deployments shall begin with a limited prototype setup using two sensor nodes, while the total number of sensors may be adjusted according to collected data, system performance, and recommendation quality. Data storage and processing components shall support continuous growth across multiple growing seasons. Overall data volume is expected to increase steadily as measurements and field records accumulate, rather than through sudden large-scale expansion.

2.3.4.5. Security Requirements

The system is expected to protect user accounts and field related data from unauthorized access. Under normal operating conditions, only authenticated users shall be able to access the system. And each user is supposed to only see the information that they are allowed. User login information and other sensitive data are expected to be protected during both transmission and storage. Data related to fields, sensor outputs and analysis results are expected to be available only to the authorized users. Images and inputs uploaded by the users and inputs are expected to be secure in order to prevent data exposure. As the number of users increase, the system is expected to maintain this level of security without affecting usability and performance.

2.3.4.6. Usability Requirements

The system shall be usable by farmers with limited technical experience. First-time users should be able to perform basic tasks, such as viewing field data and checking irrigation recommendations, under normal conditions. Most mobile app actions shall be accessible within 2–4 steps. Visual elements shall remain readable outdoors, and key information should be distinguishable within 3–5 seconds. New users should become familiar with the main features after at most two short tutorial sessions. As the system evolves, common tasks shall remain within these usability limits.

2.3.4.7. Maintainability Requirements

The system shall be maintainable with limited effort. Under normal conditions, configuration changes, minor bug fixes, and small updates shall be completed within 1–2 maintenance sessions, with a maximum downtime of 10–30 minutes. Software updates shall be deployable remotely and require no more than one technician. In case of a fault, the root cause should be identified within 30–60 minutes. Minor issues shall be resolved within one workday, while complex issues shall be resolved within 2–5 workdays.

2.3.4.8. Portability Requirements

The system is supposed to be portable across commonly used platforms and environments. Under normal operating conditions, the mobile app is expected to run on both iOS and Android devices without platform specific changes. Backend components are expected to be deployable across different server and cloud environments with one configuration adjustment. System migration to a new environment is expected to be completed in one work day without affecting stored data or functionality.

2.4. Analysis - UML

2.4.1. Use Cases

2.4.1.1. Use Case Diagram

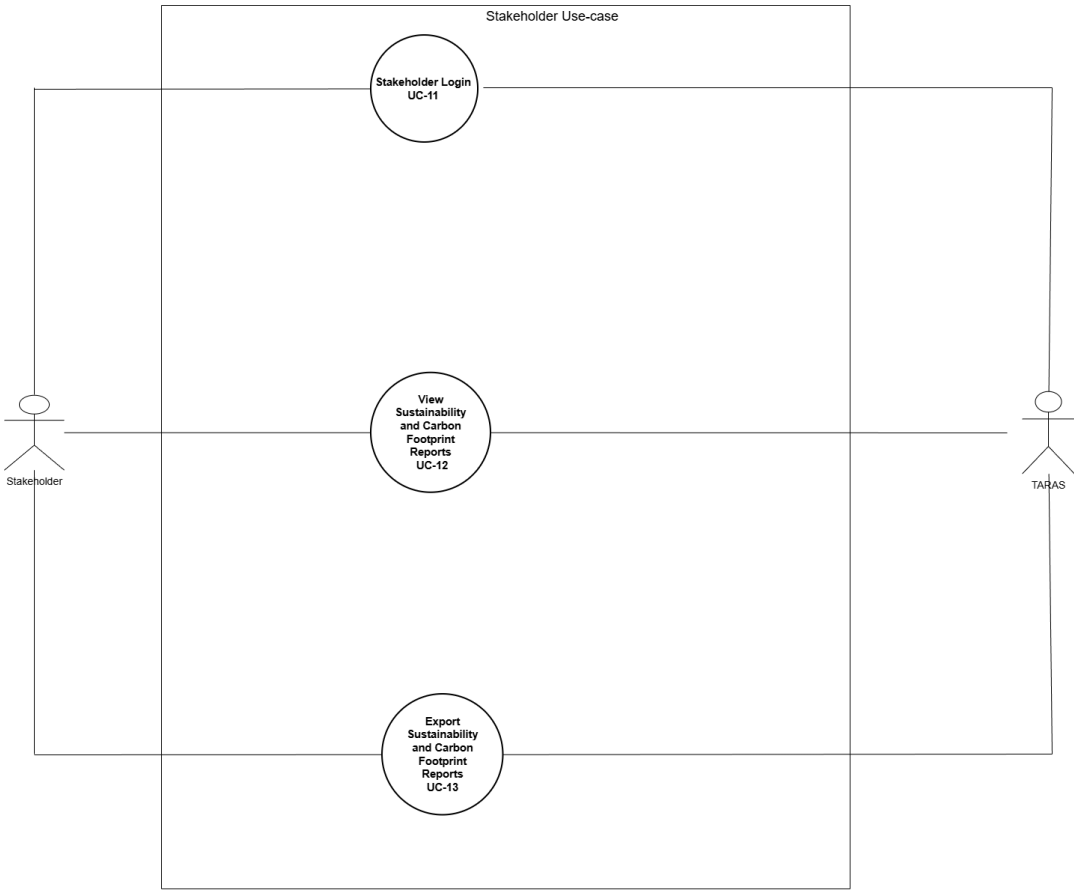


Diagram 1: Stakeholder Use Case Diagram

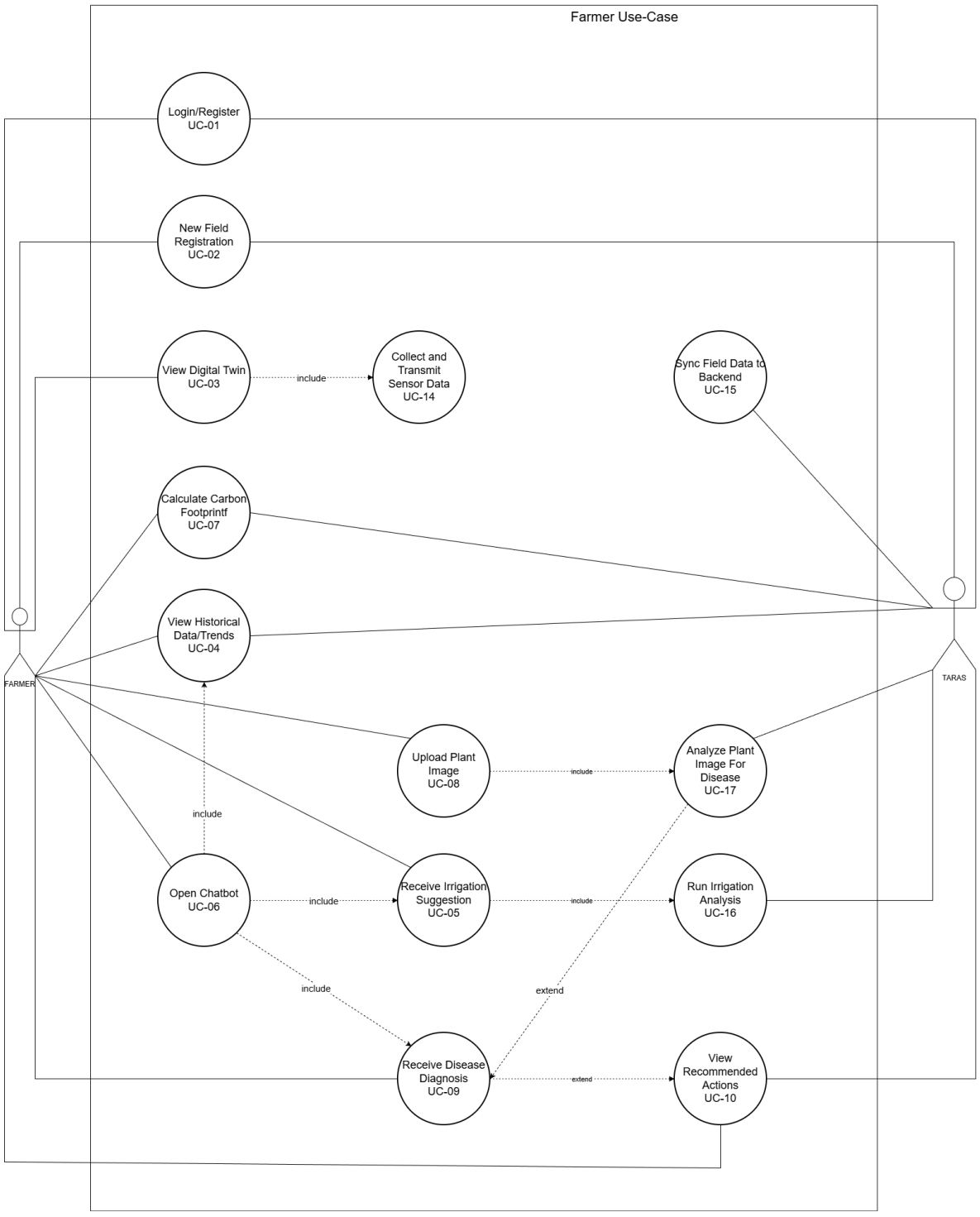


Diagram 2: Farmer Use Case Diagram

2.4.1.2. Description of Use Cases

Use Case Name	Login/Register
Use Case #	UC-01
Actors	Farmer
Description	The system manages user login and registration and grants access to the TARAS mobile application according to the user's role.
Precondition	● The user has access to the TARAS application installed. ● The user has a stable internet connection for the login/registration process. ● For login, the user already has a registered account.

Scenario	1. The system displays a login/register screen.
	2. User chooses Login or Registration
	3. For login:
	  a. User inputs credentials
	  b. System verifies information
	4. For registration:
	  a. User inputs required information
	  b. The system validates the inputs and creates a new user account.
	5. Upon successful authentication, the system loads the appropriate dashboard

Postcondition	● Users are authenticated and granted access to the system according to their role. ● The user session is initialized and stored for further operations.
Exceptions	● Incorrect username/password = System displays an error message. ● Weak or missing registration information = System rejects input and prompts correction. ● Network failure = System cannot authenticate and inform the user. ● Suspended or disabled account = System denies access.
Related Use Cases	

Use Case Name	New Field Registration
Use Case #	UC-02
Actors	Farmer
Description	The system records a new field for a registered farmer, stores its field type and layout information, and prepares zones so that sensor nodes can later be assigned to the correct field or zone.
Precondition	● The farmer is already logged into the system. ● The farmer has created or selected a farm. ● Sensor nodes may be installed or paired later.
Scenario	● The farmer enters field information such as field name, crop type and field type. ● For greenhouse fields, the farmer draws the field boundary and zone polygons. ● For pot-area fields, the farmer defines pot or zone layout information. ● The system creates the field and associated zones. ● Sensor nodes can later be assigned to the appropriate zones through the hardware setup or pairing flow. ● The system confirms that the field has been registered.
Postcondition	● A field record and its related zone/layout information are created in the system. ● Sensor nodes can later be associated with the created field or zones through the hardware setup or pairing flow.
Exceptions	● Network issues = System cannot complete the registration. ● Invalid field geometry = System asks for correction. ● Missing required field information = System rejects input and prompts correction.
Related Use Cases	

Use Case Name	View Digital Twin
Use Case #	UC-03
Actors	Farmer
Description	Users view the real-time and historical moisture/temperature distribution of the field via digital twin.
Precondition	Sensor data exists in DB.
Scenario	1. The farmer opens the digital twin page. 2. The app sends requests for the latest sensor values. 3. Backend queries PostgreSQL sensor, field, zone and node records. 4. Latest telemetry and zone/node layout data are returned. 5. The mobile application renders the field layout with node-level moisture, temperature and humidity indicators.
Postcondition	Digital Twin updated with fresh data.
Exceptions	● No recent sensor data available. ● The backend cannot access DB. ● Device offline = cached data shown.
Related Use Cases	UC-14

Use Case Name	View Historical Data/Trends
Use Case #	UC-04
Actors	Farmer
Description	The system displays historical information for a selected field over a chosen time range, helping the farmer understand past field conditions and the effects of irrigation decisions.
Precondition	● Farmer is logged in. ● At least one field is registered and linked to sensor nodes. ● Historical data exists in the system for the selected field (or at least some past measurements are stored).
Scenario	1. Farmer chooses the history/trends option in the app.

	2. System displays a time-range selector 3. Farmer selects a time range and confirms. 4. The system retrieves historical sensor and/or irrigation data for that period. 5. The system displays graphs or charts (e.g., moisture vs. time, temperature vs. time, irrigation events on the timeline). 6. Farmers may scroll, zoom, or switch between variables (moisture, temperature, irrigation). 7. Farmer closes the view or returns to the field overview.
Postcondition	● The farmer has viewed historical trends for the selected field over the chosen period.
Exceptions	● No data available for the selected period = System shows an informative “No data for this period” message and suggests another range. ● Partial data gaps = System shows available data and may mark missing intervals. ● Network/connectivity issues = System cannot fetch new history and offers cached trends if available.
Related Use Cases	UC-06

Use Case Name	Receive Irrigation Suggestion
Use Case #	UC-05
Actors	Farmer
Description	The system provides an irrigation recommendation for the selected field based on previously computed irrigation analysis results, based on latest soil moisture, temperature, humidity, crop growth stage, field type and calibration parameters. The recommendation indicates whether irrigation is needed and, depending on field type, the recommended water amount or irrigation duration, presented in a clear and farmer-friendly format.
Precondition	● The farmer is logged into the system. ● At least one field is registered and selected. ● Irrigation analysis results are available for the field.
Scenario	1. The farmer opens the field dashboard in the mobile application. 2. The system checks whether a valid and up-to-date irrigation analysis result is available for the selected field.

	3. If available, the system retrieves the most recent irrigation analysis outputs, including irrigation timing indicators and estimated irrigation duration. 4. The system interprets these outputs and converts them into a farmer-oriented recommendation format (e.g., textual guidance, highlighted indicators, or visual cues). 5. The recommendation is displayed on the dashboard, clearly indicating whether irrigation is needed and the recommended water amount or duration depending on field type. 6. The farmer reviews the recommendation and decides whether to act on it.
Postcondition	● The irrigation recommendation is presented to the farmer for decision-making. ● The recommendation remains accessible until a new analysis result replaces it.
Exceptions	● No irrigation analysis result available: The system informs the farmer that a recommendation cannot currently be provided. ● Network or backend failure: The system displays the last successfully retrieved recommendation, if available.
Related Use Cases	UC-06,UC-16

Use Case Name	AI Decision-Support Chatbot
Use Case #	UC-06
Actors	Farmer
Description	The farmer interacts with a conversational AI assistant that uses a Large Language Model (LLM) to interpret sensor data, irrigation recommendations, historical trends and disease-detection results. The chatbot provides natural-language explanations guidance, and actionable decision-support suggestions tailored to the field's current conditions.
Precondition	● The user is authenticated and has access to the chatbot interface. ● Recent sensor data, irrigation recommendations, or disease analysis results are available. ● The LLM service is online and reachable by the backend.
Scenario	1. The user opens the AI assistant interface within the mobile application. 2. The user enters a question, command or request for advice (e.g.,

Use Case Name	AI Decision-Support Chatbot
	"Should I irrigate today?" or *"What does this disease mean for my field?"*). 3. The system retrieves relevant contextual data: ● Latest sensor readings (soil moisture, temperature, humidity) ● Irrigation recommendations generated by the recommendation service. ● Disease-detection results, if available ● Weather and historical trends (optional) 4. The backend compiles this data into a structured context package. 5. The LLM processes the user's question together with the contextual data. 6. The LLM generates a natural-language response with explanations or recommended actions. 7. The system returns the response to the user through the chat interface. 8. The user reads or follows the provided guidance.
Postcondition	● The farmer receives a personalized, context-aware advisory message. ● The conversation may be logged for future analytics or model improvement.
Exceptions	● The LLM service is unreachable or times out. ● No relevant sensor or irrigation data is available for the query. ● User query is ambiguous or cannot be interpreted. ● Backend fails to compile context data. ● Response generation error occurs (model failure).
Related Use Cases	UC-04, UC-05, UC-09

Use Case Name	Calculate Carbon Footprint
Use Case #	UC-07
Actors	Farmer
Description	The farmer requests a carbon footprint calculation for a selected field by creating activity records such as fuel, fertilizer or electricity usage. The system uses this information together with existing field data to estimate the environmental impact and presents the result to the farmer [6].
Precondition	● Farmer is logged in.

	● At least one field is registered and selected. ● The farmer has access to the carbon footprint feature.
Scenario	1. Farmer opens the Carbon Footprint section. 2. The system displays activity categories such as fuel, fertilizer and electricity. 3. Farmer selects an activity type, enters amount, date and optional notes. 4. The system retrieves the applicable emission factor. 5. The system calculates emission_amount = activity_amount × emission_factor. 6. The system stores the result as a carbon log. 7. The system updates total and category-based carbon summaries.
Postcondition	● A carbon log is created for the selected farm and carbon summaries are updated. ● The result becomes available for viewing and comparison over time.
Exceptions	● Required inputs are missing = System asks the farmer to complete the information. ● Calculation cannot be completed = System informs the farmer that the result is temporarily unavailable.
Related Use Cases	UC-13

Use Case Name	Upload Plant Image
Use Case #	UC-08
Actors	Farmer
Description	The user selects or captures a plant leaf image and uploads it to the system for disease analysis.
Precondition	● The user is authenticated. ● The device has network connectivity/queued upload mode enabled.
Scenario	1. The user navigates to the disease detection page. 2. The user captures an image or selects an existing image. 3. The system validates the selected image type and size.

Use Case Name	Upload Plant Image
	4. The image is uploaded to the backend for further processing. 5. The system acknowledges successful upload.
Postcondition	The image is received and stored temporarily by the backend for analysis.
Exceptions	● Invalid or unsupported file format. ● Image corrupted or unreadable. ● Network failure during upload. ● Upload exceeds size limit.
Related Use Cases	UC-17

Use Case Name	Receive Disease Diagnosis
Use Case #	UC-09
Actors	Farmer
Description	After uploading an image, the user receives the disease classification, confidence score returned by the system.
Precondition	● A valid image has been uploaded. ● The inference engine is available.
Scenario	1. The system preprocesses the uploaded image. 2. The system runs the disease-detection model on the image. 3. The model generates a label and confidence score. 4. The backend sends the diagnosis result to the user's device. 5. The user views the diagnosis.
Postcondition	● Diagnosis is displayed to the user. ● The result is stored in the database
Exceptions	● Model service unavailable. ● Inference timeout. ● Disease cannot be identified with sufficient confidence. ● Backend failure returning the result.
Related Use Cases	UC-06,UC-10, UC-17

Use Case Name	View Recommended Actions
Use Case #	UC-10
Actors	Farmer
Description	The user views actionable recommendations generated based on the diagnosed disease (e.g., treatment steps, risk warnings).
Precondition	● A diagnosis has been successfully generated ● Recommendation rules or mappings exist for the detected disease.
Scenario	1. The system matches the diagnosed disease to predefined recommendation rules. 2. The system generates appropriate treatment or monitoring actions. 3. The system presents the recommended actions on the app. 4. The user reviews the suggested next steps.
Postcondition	● Users receive actionable treatment guidance.
Exceptions	● No recommendation exists for the specific disease. ● Backend fails to retrieve recommendation rules. ● User interface fails to load the recommendation section.
Related Use Cases	UC-09

Use Case Name	Stakeholder Login
Use Case #	UC-11
Actors	Stakeholder
Description	A stakeholder logs into the platform to access high-level summaries, sustainability metrics and environmental reports for the agricultural fields they have a stake in. Stakeholders can only read the existing data/analysis.
Precondition	● The stakeholder has a valid account and credentials. ● System authentication services are available.

Use Case Name	Stakeholder Login
Scenario	1. Stakeholder opens the dashboard login page. 2. Stakeholder enters email/username and password. 3. System verifies credentials. 4. The system loads the stakeholder dashboard with authorized field data. 5. Stakeholders gain access to summary and reporting tools.
Postcondition	● Stakeholders are successfully authenticated and connected to their dashboard.
Exceptions	● Invalid credentials. ● User account disabled or missing necessary permissions. ● Authentication service unavailable. ● Network failure.
Related Use Cases	

Use Case Name	View / Export Timetable
Use Case #	UC-12
Actors	Stakeholder, Farmer
Description	The farmer or stakeholder views and exports timetable information for a selected field, farm, or reporting period. Farmers use the timetable to monitor scheduled agricultural activities, irrigation plans, and field operations, while stakeholders use it for read-only monitoring and reporting.
Precondition	● The farmer or stakeholder is authenticated and logged into the system. ● Relevant timetable, field, or irrigation schedule data exists in the database.
Scenario	1. The farmer or stakeholder selects a field or farm. 2. The farmer or stakeholder selects a time period such as day, week, month, or custom range. 3. The system retrieves timetable data for the selected field and period. 4. The system displays the timetable using calendar views, tables, or timeline-based visuals. 5. The farmer or stakeholder optionally selects an export file format

Use Case Name	View / Export Timetable
	such as CSV or Excel. 6. The system generates the selected timetable file. 7. The system provides the file as a downloadable resource.
Postcondition	The selected timetable is successfully displayed and, if requested, exported for download.
Exceptions	● No timetable data is available for the selected period. ● Timetable data cannot be retrieved due to missing or incomplete records. ● Export generation fails. ● Backend query timeout or timetable rendering error occurs.
Related Use Cases	UC-13

Use Case Name	View / Export Carbon Footprint Reports
Use Case #	UC-13
Actors	Stakeholder, Farmer
Description	The farmer or stakeholder views and exports carbon footprint reports for a selected field, farm, or time period. The report supports environmental monitoring by presenting emission-related data and sustainability metrics.
Precondition	● The farmer or stakeholder is authenticated and logged into the system. ● Relevant historical field, irrigation, activity, and carbon footprint data exists in the database. ● Carbon footprint metrics can be calculated for the selected period.
Scenario	1. The farmer or stakeholder selects a field, farm, or reporting scope. 2. The farmer or stakeholder selects a time period such as month, quarter, year, or custom range. 3. The system retrieves and processes the relevant carbon footprint data. 4. The system displays carbon footprint results using charts, tables, and key performance indicators. 5. The farmer or stakeholder optionally selects an export file format such as CSV or Excel.

Use Case Name	View / Export Carbon Footprint Reports
	6. The system generates the requested carbon footprint report file. 7. The system provides the file as a downloadable resource.
Postcondition	The selected carbon footprint report is successfully displayed and, if requested, exported for download.
Exceptions	● No carbon footprint data is available for the selected period. ● Required metrics cannot be computed due to missing or incomplete data. ● Export generation fails. ● Backend analytics or query service failure occurs.
Related Use Cases	UC-12, UC-07

Use Case Name	Collect and Transmit Sensor Data
Use Case #	UC-14
Actors	System
Description	Sensor nodes deployed in the field automatically collect environmental measurements and transmit them to the gateway using ESP-NOW; the gateway forwards them to the TARAS backend via Wi-Fi/HTTPS. The received data are stored and used for field monitoring, digital-twin updates, and irrigation decision support.
Precondition	● The sensor node is powered on and calibrated. ● The node is correctly registered in the system and associated with a specific field. ● The node is paired with a gateway. ● Gateway internet connectivity is available or local buffering is enabled. ● The sampling interval is configured.
Scenario	1. The sensor node collects environmental measurements in the field. 2. The node prepares the measurements as a structured data payload. 3. The node transmits the data to the paired gateway using ESP-NOW. 4. The gateway verifies the packet and acknowledges valid data. 5. The gateway forwards the telemetry to the backend via HTTPS when

	internet connectivity is available. 6. The backend receives the data and verifies the associated field and node identity. 7. If valid, the backend stores the new measurements in the database. 8. The backend updates the field's latest status. 9. The system automatically triggers follow-up processes (digital-twin update, trend analysis, irrigation model input).
Postcondition	● New sensor data is stored in the database and available to farmer dashboards. ● The digital twin and field status are updated based on the latest measurements.
Exceptions	● If new data cannot be received due to connectivity issues, the system continues to display the most recent available data. ● The system may notify the farmer if sensor data has not been received for an unusually long period.
Related Use Cases	UC-03

Use Case Name	Sync Field Data to Backend
Use Case #	UC-15
Actors	System
Description	Field data collected from sensor nodes and the mobile application, including sensor readings and irrigation-related records, are synchronized with the cloud backend to ensure that all field information is safely stored and kept up to date.
Precondition	● Sensor nodes and/or the mobile application have collected new field data. ● At least one field is registered in the system. ● Gateway internet connectivity is available, or local buffering is enabled on the sensor node/gateway.
Scenario	1. New field data becomes available on a sensor node or the mobile application. 2. The system prepares the collected data for transmission to the

	backend. 3. When connectivity is available, sensor telemetry is transmitted through the gateway, while mobile-created records are synchronized through backend APIs. 4. The backend receives the data and stores it in the database. 5. The records associated with the relevant field are updated. 6. Updated information becomes available for farmer dashboards, digital-twin visualizations, historical analysis, and recommendations.
Postcondition	● Field data stored on sensor nodes or mobile devices is synchronized with the central TARAS system. ● The backend contains the latest version of field-related information.
Exceptions	● If no internet connection is available, the system temporarily stores the data locally and retries synchronization later. ● If synchronization fails repeatedly, the system may display a warning or status notification.
Related Use Cases	

Use Case Name	Run Irrigation Analysis
Use Case #	UC-16
Actors	System
Description	The system performs irrigation analysis for a registered field by processing sensor data, sensor readings, crop metadata, field type, irrigation mode, calibration parameters and irrigation records. The analysis produces irrigation-related indicators that are later used to generate irrigation recommendations and visual feedback.
Precondition	● A field is registered and linked to active sensor nodes. ● Sensor data and supporting inputs are available. ● Irrigation analysis services are operational.
Scenario	1. The arrival of new sensor data, updated weather information or a scheduled evaluation triggers the irrigation analysis process. 2. The system gathers the required field-related data from the database.

	3. Irrigation data preprocessing steps are executed to ensure temporal consistency and data quality. 4. The system performs rule-based irrigation analysis 5. Irrigation timing indicators and estimated irrigation duration parameters are derived. 6. The analysis results are stored in the backend and marked as available. 7. The system updates dependent modules.
Postcondition	● Updated irrigation analysis results are stored for the field. ● The results become available for recommendation generation and historical inspection.
Exceptions	● Insufficient or invalid data: The analysis is delayed or marked as unavailable. ● Processing error: The system logs the failure and follows the corresponding procedure.
Related Use Cases	UC-05

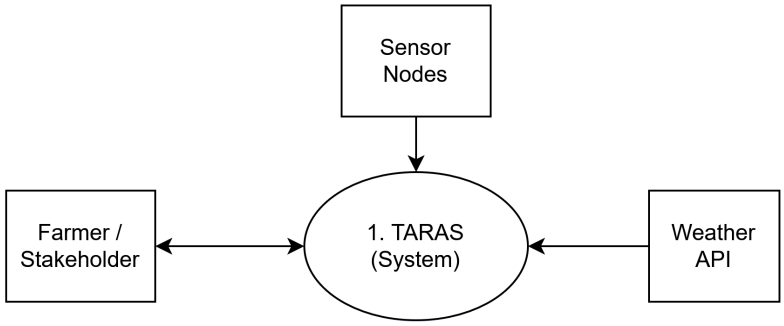
Use Case Name	Analyze Plant Image for Diseases
Use Case #	UC-17
Actors	System (Inference Engine, Backend Services)
Description	The system automatically processes an incoming plant image, runs the CNN model to identify potential diseases and stores the result for retrieval by the user.
Precondition	● A valid image upload request has been received from the mobile client. ● The CNN model is available and loaded on the server.
Scenario	1. The system receives the uploaded leaf image. 2. The system performs validation (format, resolution, corruption check). 3. The image is preprocessed (resize, normalization). 4. The system executes CNN inference to detect the disease class. 5. The system generates confidence scores and prediction details when available. 6. The system stores the classification result in the database.

Use Case Name	Analyze Plant Image for Diseases
	7. The system returns the structured result to the mobile application.
Postcondition	● The image analysis result is stored in the disease_detections table. ● The result becomes available for display in the disease detection module.
Exceptions	● Model service unavailable or inference timeout. ● Image preprocessing failure. ● Database insertion error. ● Unrecognized disease or insufficient confidence.
Related Use Cases	UC-08, UC-09

2.4.2. Functional Modelling(DFD)

2.4.2.1. DFD Diagrams

2.4.2.1.1. Level 0 - Context Diagram



2.4.2.1.2. Level 1 DFD

2.5. Conclusion

This SRS defines the architectural and functional scope of TARAS and provides a foundation for system implementation. It explains how the platform connects data analysis with practical agricultural decision-making for irrigation and plant health management. The proposed design integrates IoT sensing, machine-learning models, and digital-twin visualization within a unified cloud-based architecture, while addressing field constraints such as intermittent connectivity through buffering and synchronization.

Overall, this SRS establishes a shared technical understanding among stakeholders and serves as a blueprint for developing TARAS as a scalable, secure, and user-friendly precision agriculture system.

3. System Architecture and Methodology

3.1. Introduction

3.1.1. Purpose

This document defines the system architecture of the TARAS precision agriculture platform by presenting its core components, logical layers, and interactions. The architectural decisions are derived from the functional and non-functional requirements defined in the SRS.

In addition to the architectural design, the document explains the methodological approach for data acquisition, preprocessing, and irrigation decision logic, showing how agronomic principles and sensor-driven analytics are translated into system behavior. It also describes auxiliary AI-assisted advisory components that support explainability and user understanding without performing core decision-making.

Overall, this document serves as an architectural and methodological reference for implementation, integration, and future development, ensuring a consistent, maintainable realization of the TARAS platform.

3.2. System Overview

The TARAS (Tarım Analizi Sistemi) platform is a precision agriculture decision-support system designed to assist farmers in irrigation planning, crop health monitoring and sustainability assessment. The project initially explored a wide range of smart agriculture features, which were evaluated based on impact, feasibility, usability, cost and sustainability. Based on literature analysis and comparisons with existing solutions, the system scope was refined to focus on the most effective and implementable components. The final design centers on integrated field sensing, AI-driven analysis, digital twin visualization and a mobile application interface, ensuring a practical, scalable and user-oriented solution.

3.3. System Architecture

3.3.1. Architectural Design

The architectural design of the TARAS follows a hybrid architecture that combines Layered Architecture and Client–Server Architecture principles. This approach supports modular development, scalability and clear separation of responsibilities across system components.

3.3.1.1. Layered Architecture

3.3.1.1.1. Presentation Layer

Provides the mobile application interface through which farmers interact with the system. This layer presents sensor readings, irrigation recommendations, disease detection results, digital twin visualizations, sustainability metrics such as carbon footprint, and AI-assisted advisory explanations in an accessible and user-friendly manner. The advisory interface translates analytical outputs into clear, context-aware guidance, supporting user understanding without replacing farmer decision-making.

3.3.1.1.2. Application & Analytics Layer

Implements the core system logic, including data processing, irrigation prediction, disease analysis, decision-support workflows and user interaction management. ML and computer vision models operate within this layer to transform raw data into actionable insights.

3.3.1.1.3. Data & Cloud Services Layer

Handles data storage, retrieval and synchronization between field devices and the mobile application. This layer manages time-series sensor data, historical records, model outputs and user inputs using a centralized cloud-based backend.

3.3.1.1.4. Device & Sensing Layer

Includes IoT sensor nodes and gateways deployed in the field, responsible for collecting soil moisture, temperature and environmental data. This layer abstracts hardware-level interactions and ensures reliable data acquisition.

3.3.1.1.5. Communication Layer

Manages data transmission between sensor nodes, gateway devices and cloud services. Sensor nodes communicate with the field gateway over ESP-NOW, a low-power connectionless 2.4 GHz Wi-Fi protocol, while the gateway relays buffered readings to the cloud over Wi-Fi using HTTP and a Socket.IO real-time channel. The layer ensures reliable, low-power and fault-tolerant data flow, with on-device buffering on the gateway for resilience during connectivity loss.

3.3.1.2. Client–Server Architecture

The TARAS mobile app operates as a client that interacts with the cloud-based backend services. Field sensor data, user inputs, and

system outputs, such as irrigation recommendations, disease analysis results and sustainability metrics are exchanged through request–response communication mechanisms. This separation between client and server components enables loose coupling, supports scalability and simplifies maintenance and future extensions.

3.3.2. Decomposition Description

3.3.2.1. Field Sensing & Data Acquisition Subsystem

- Soil Moisture Sensing Module
- Environmental Data Collection (Temperature, Humidity)
- Sensor Calibration & Validation Module
- Gateway Communication Module

3.3.2.2. Analytics & Decision Support Subsystem

- Data Storage Module
- Irrigation Recommendation Module
- Disease Detection Module
- Sustainability & Carbon Footprint Calculation Module
- Data Processing & Model Management Module
- LLM advisory & Explainability Module

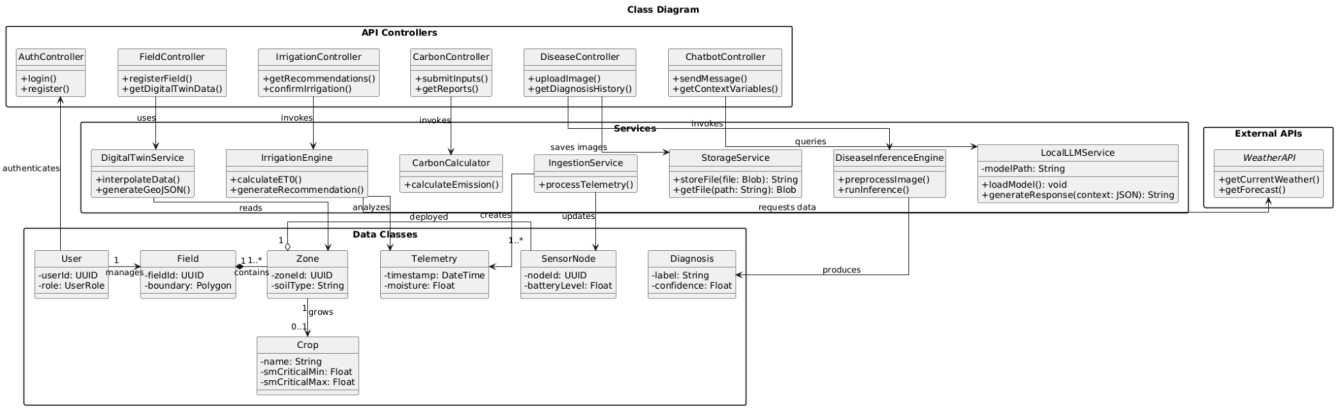
3.3.2.3. Visualization & User Interaction Subsystem

- Mobile Application Interface
- Digital Twin Visualization Module
- Recommendation & Alert Presentation
- User Input & Configuration Management

Each TARAS module communicates through clearly defined interfaces, hiding internal logic while exposing essential operations such as retrieving sensor data, generating irrigation recommendations, performing disease analysis, and updating visualization data. This supports abstraction, modular refinement, and reusability in line with software engineering principles.

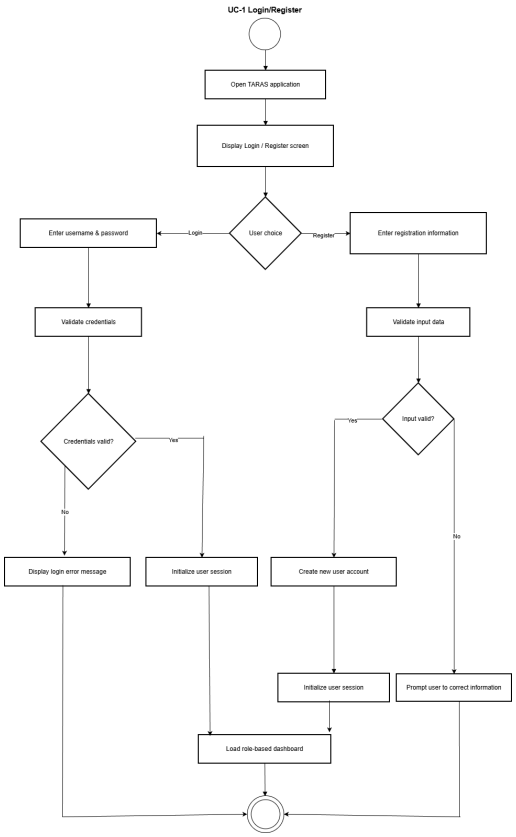
The modular structure also simplifies independent development and testing, while allowing future extensions such as new sensor types, improved prediction models, or expanded sustainability metrics without affecting the overall architecture.

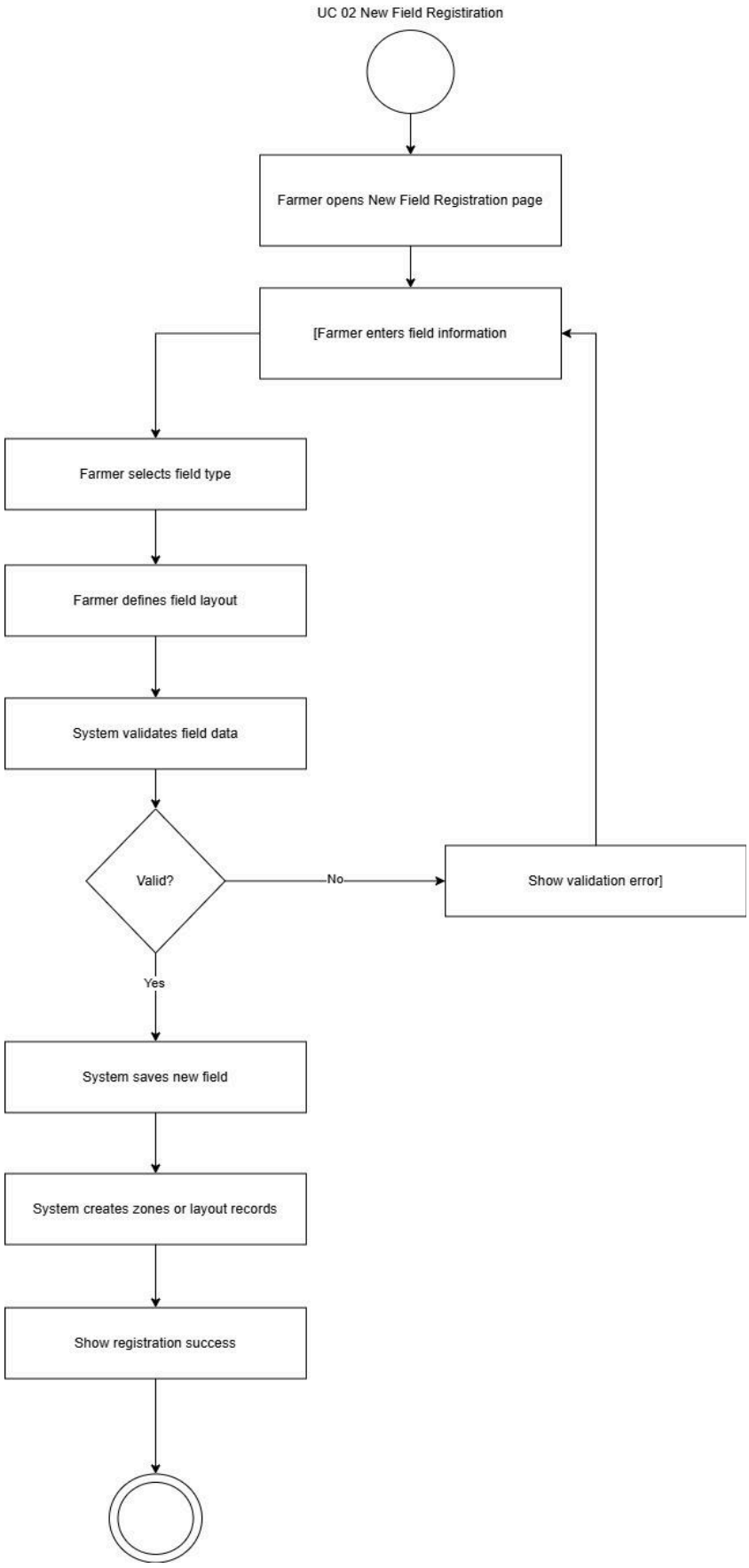
3.3.2.4. Class Diagram

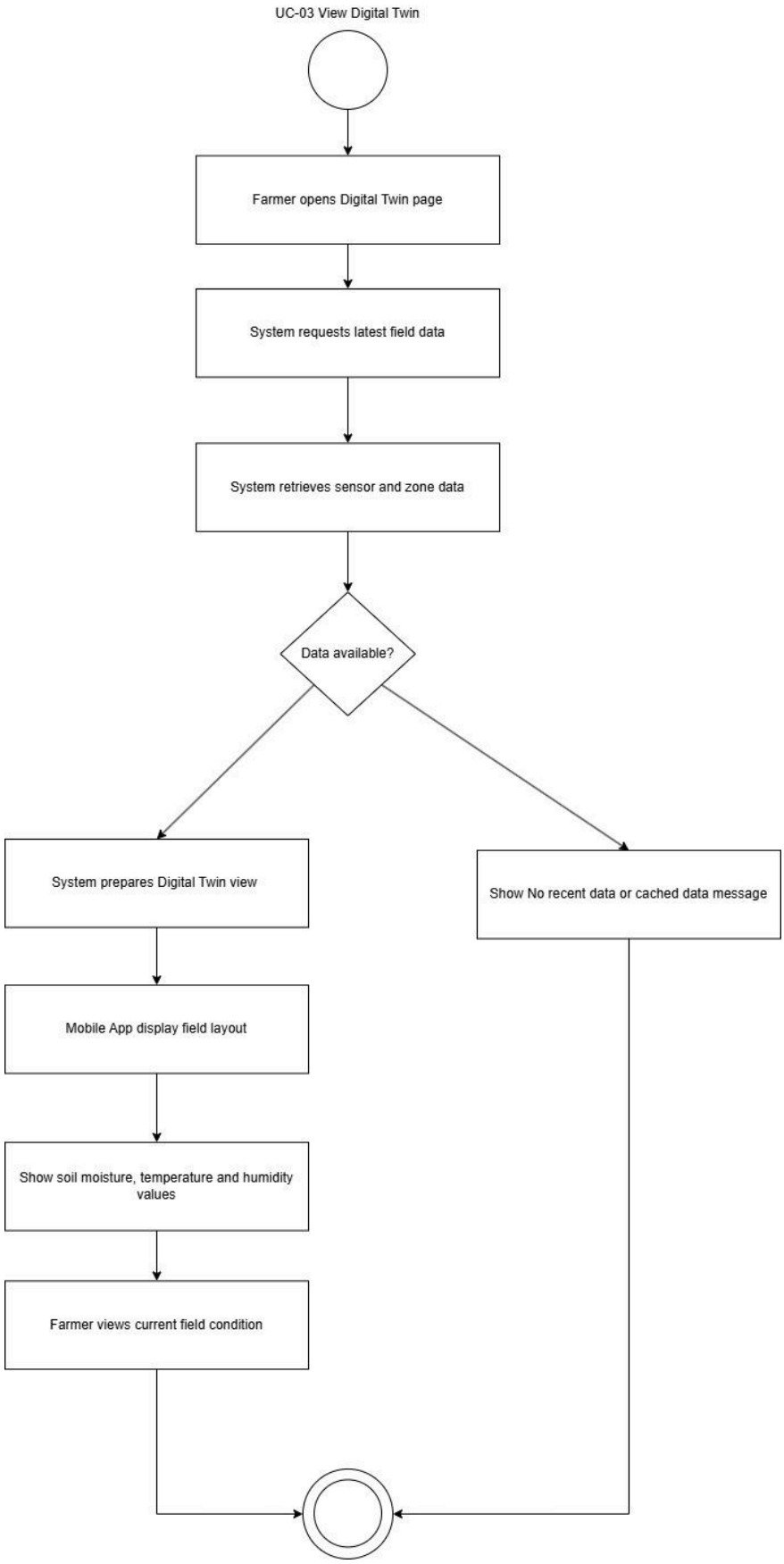


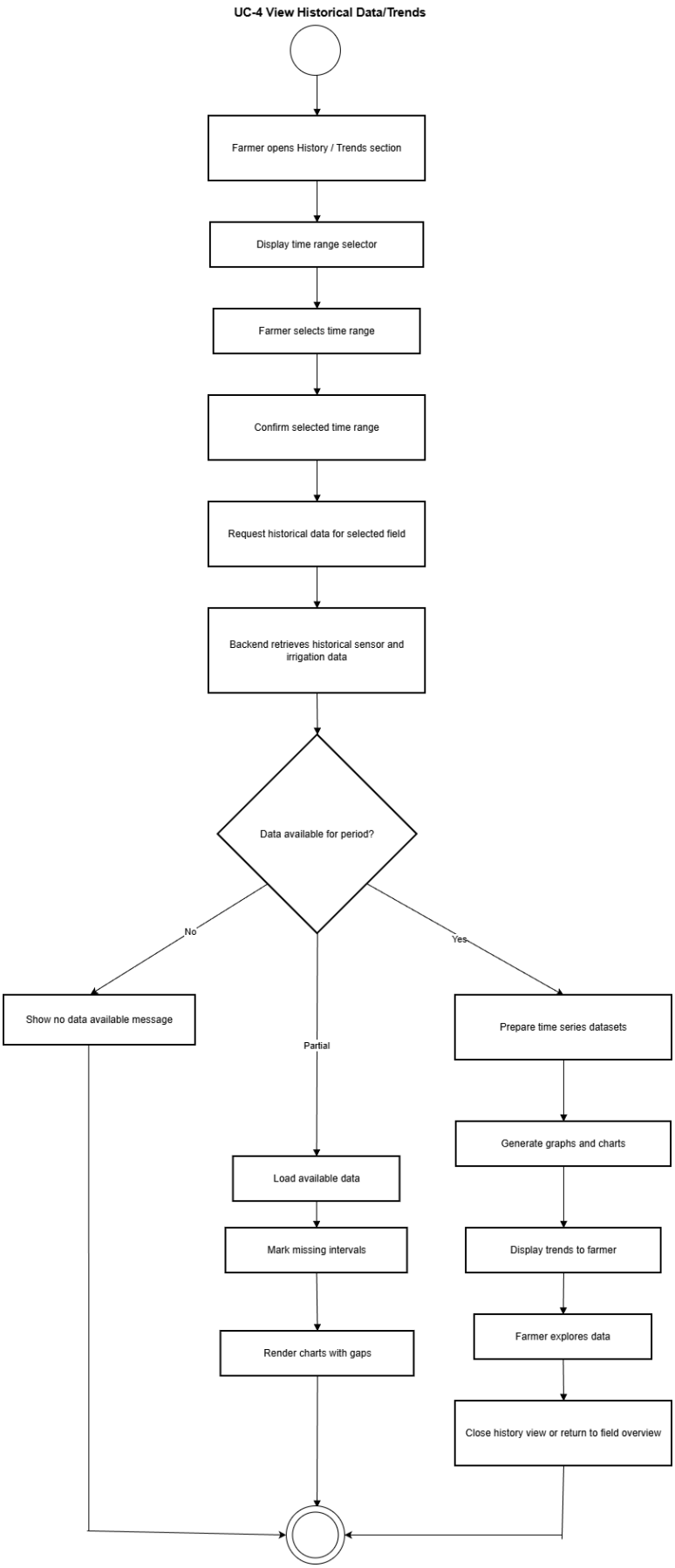
3.3.3. System Modeling

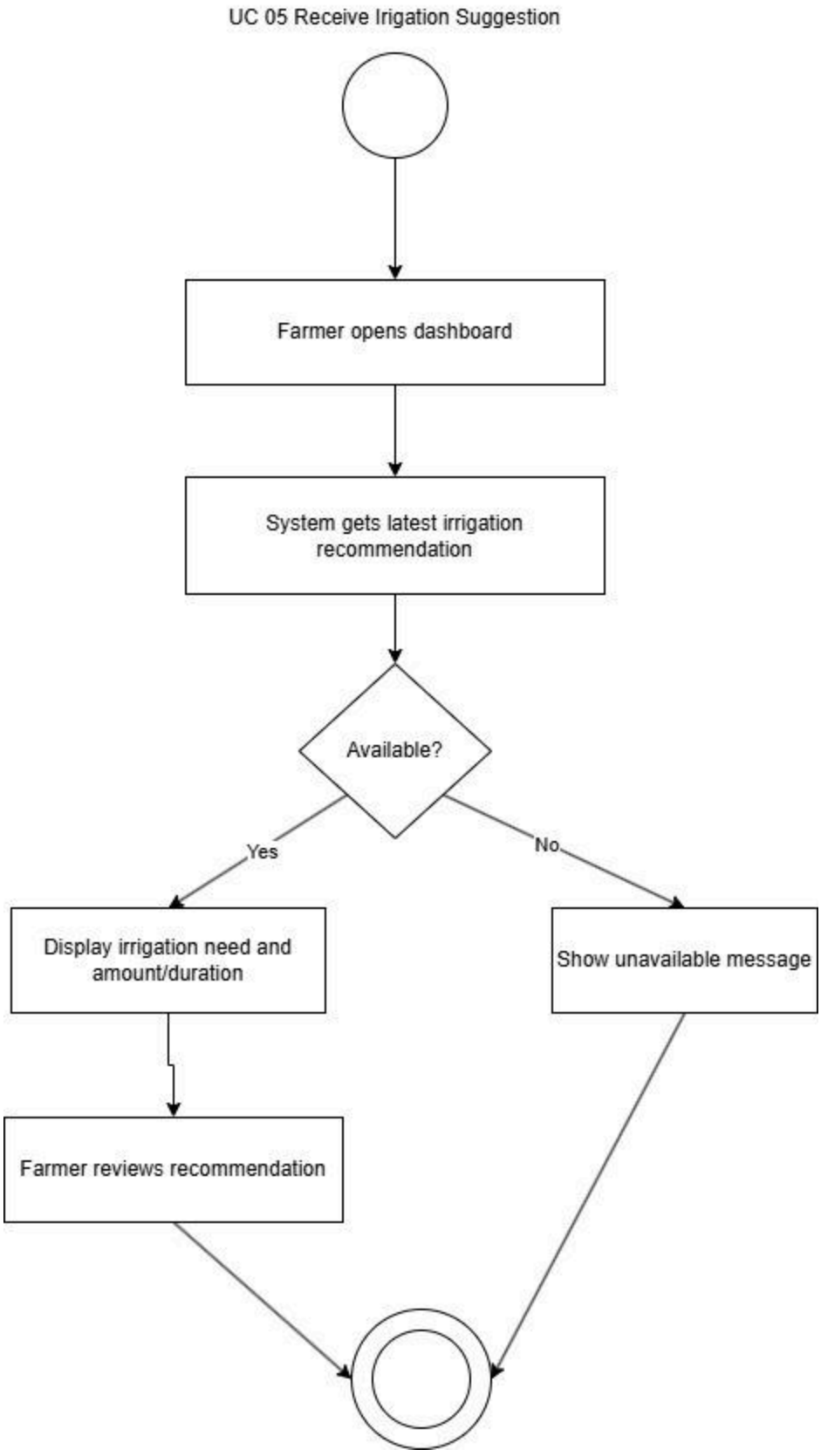
3.3.3.1. Activity Diagrams

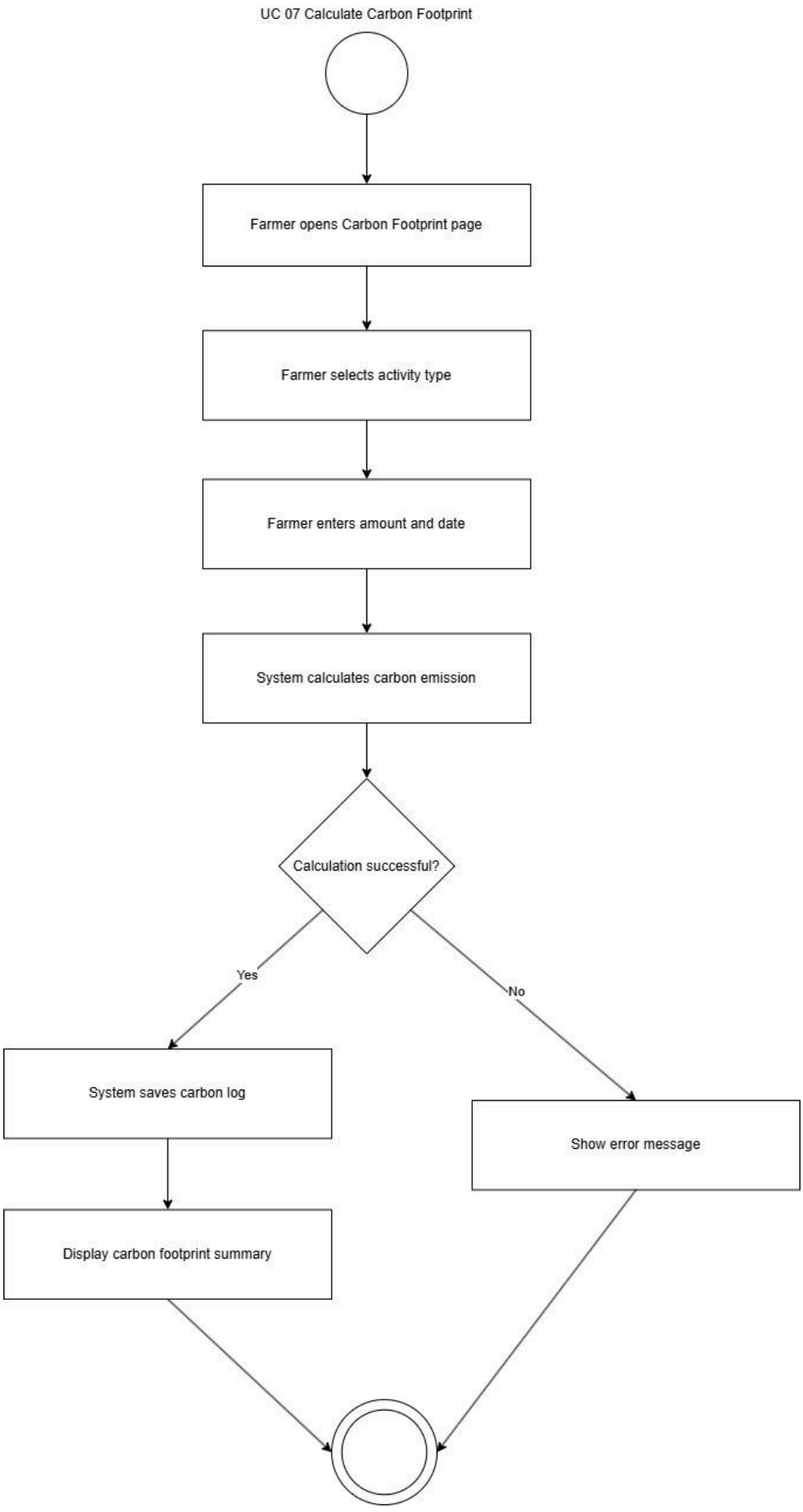


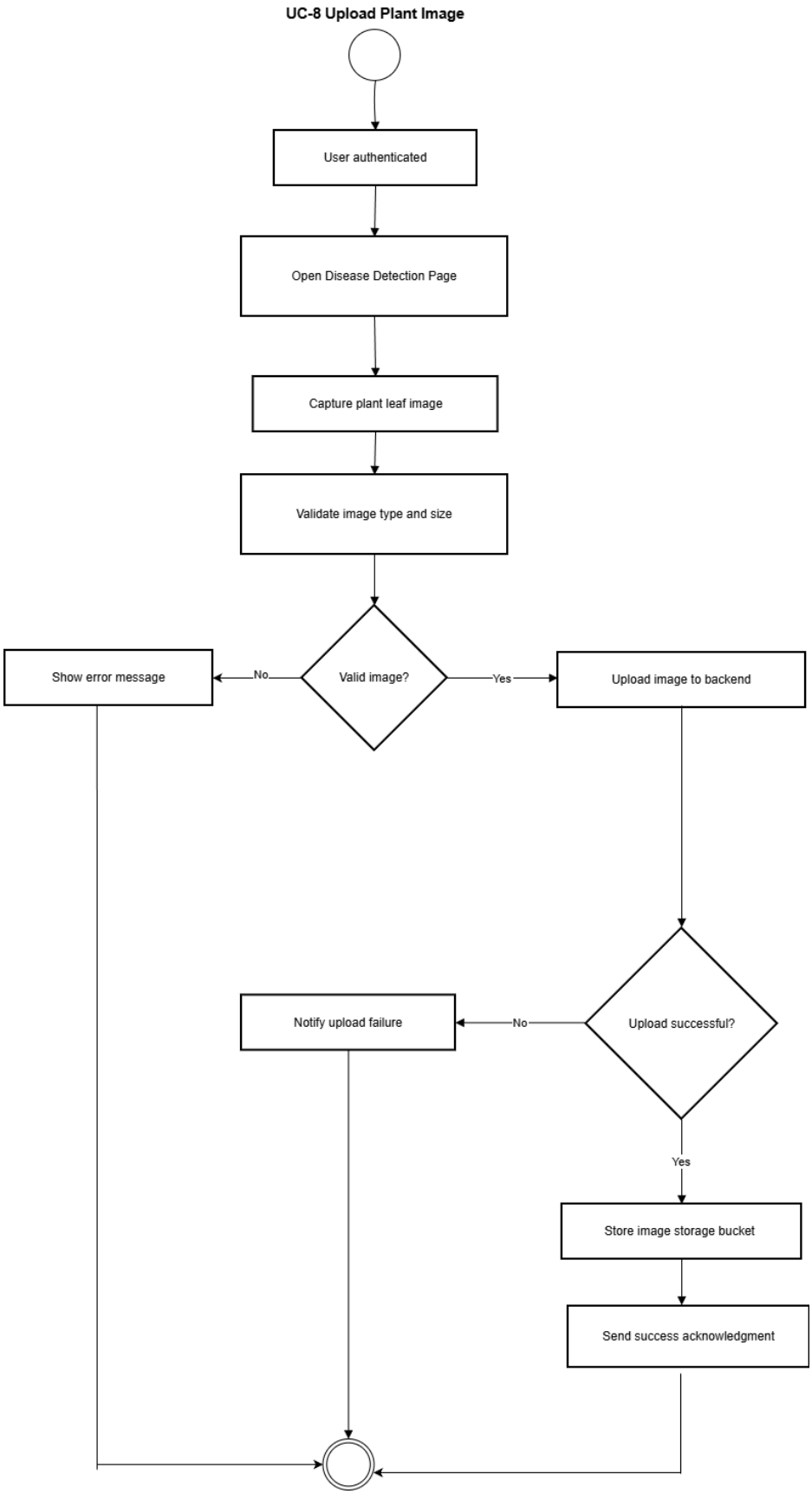

UC 02 New Field Registiration
Farmer opens New Field Registration page
[Farmer enters field information
Farmer selects field type
Farmer defines field layout
System validates field data
Valid?
No
Show validation error]
Yes
System saves new field
System creates zones or layout records
Show registration success

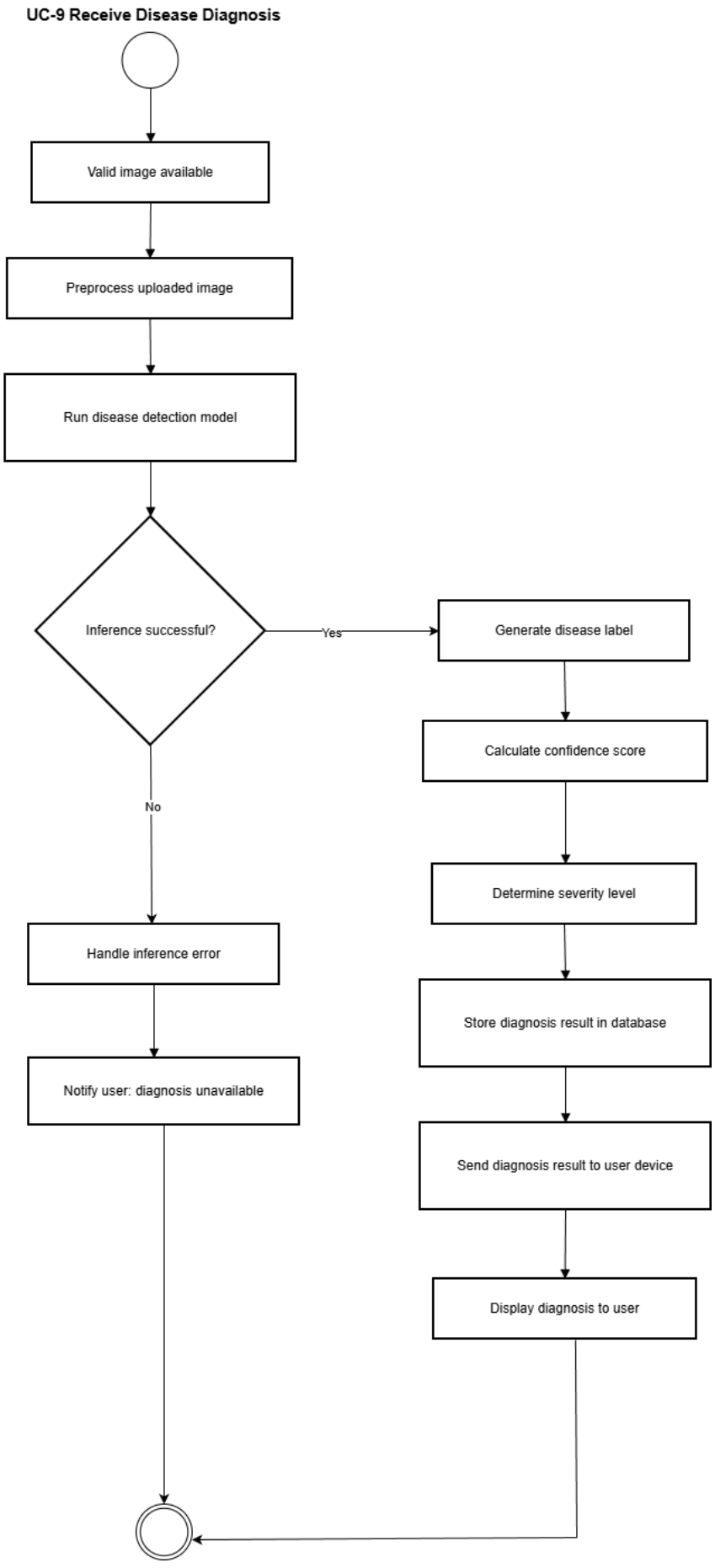

UC-03 View Digital Twin
Farmer opens Digital Twin page
System requests latest field data
System retrieves sensor and zone data
Data available?
System prepares Digital Twin view
Show No recent data or cached data message
Mobile App display field layout
Show soil moisture, temperature and humidity values
Farmer views current field condition

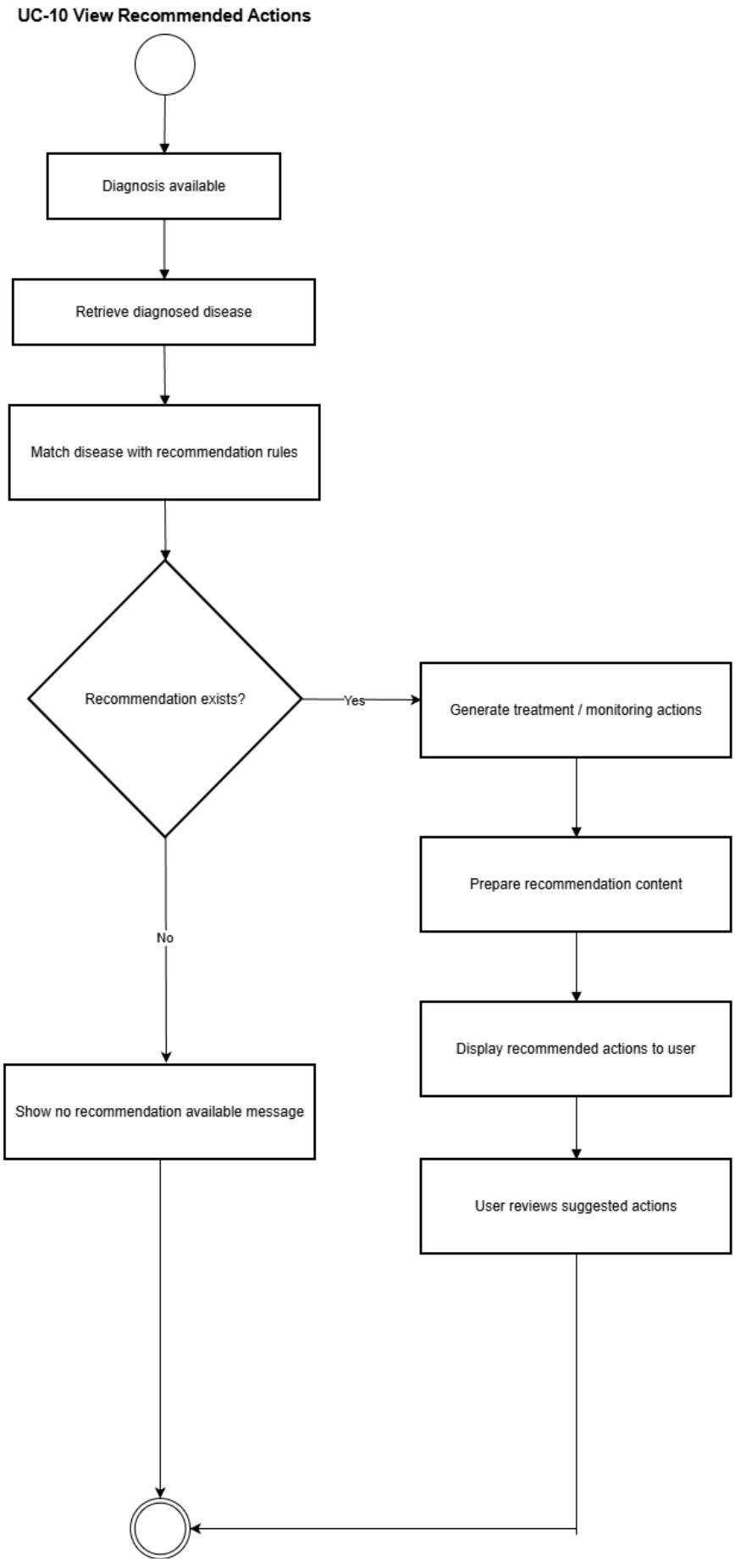

UC-4 View Historical Data/Trends
Farmer opens History / Trends section
Display time range selector
Farmer selects time range
Confirm selected time range
Request historical data for selected field
Backend retrieves historical sensor and irrigation data
Data available for period?
No
Partial
Yes
Show no data available message
Load available data
Mark missing intervals
Render charts with gaps
Prepare time series datasets
Generate graphs and charts
Display trends to farmer
Farmer explores data
Close history view or return to field overview

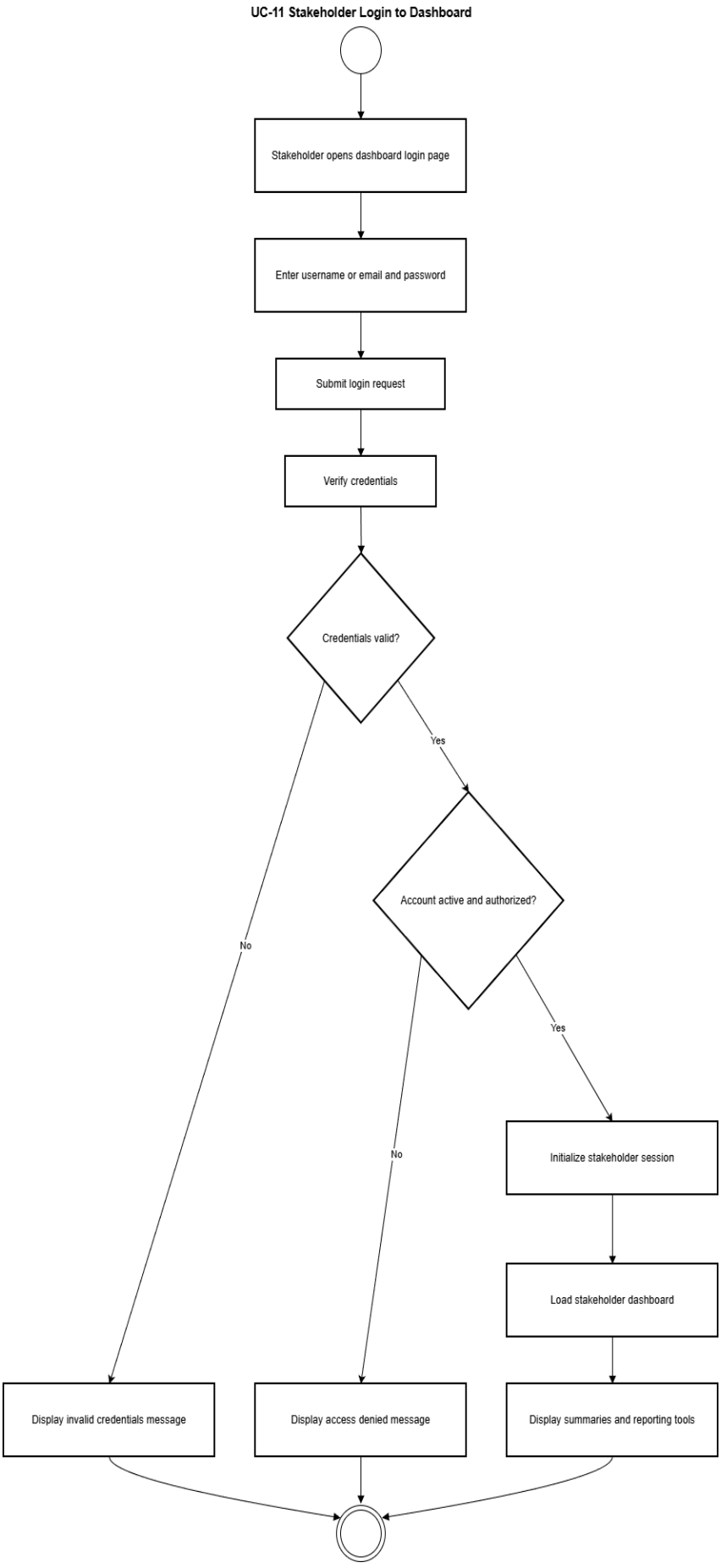

UC 05 Receive Irigation Suggestion
Farmer opens dashboard
System gets latest irrigation recommendation
Available?
Yes
No
Display irrigation need and amount/duration
Show unavailable message
Farmer reviews recommendation

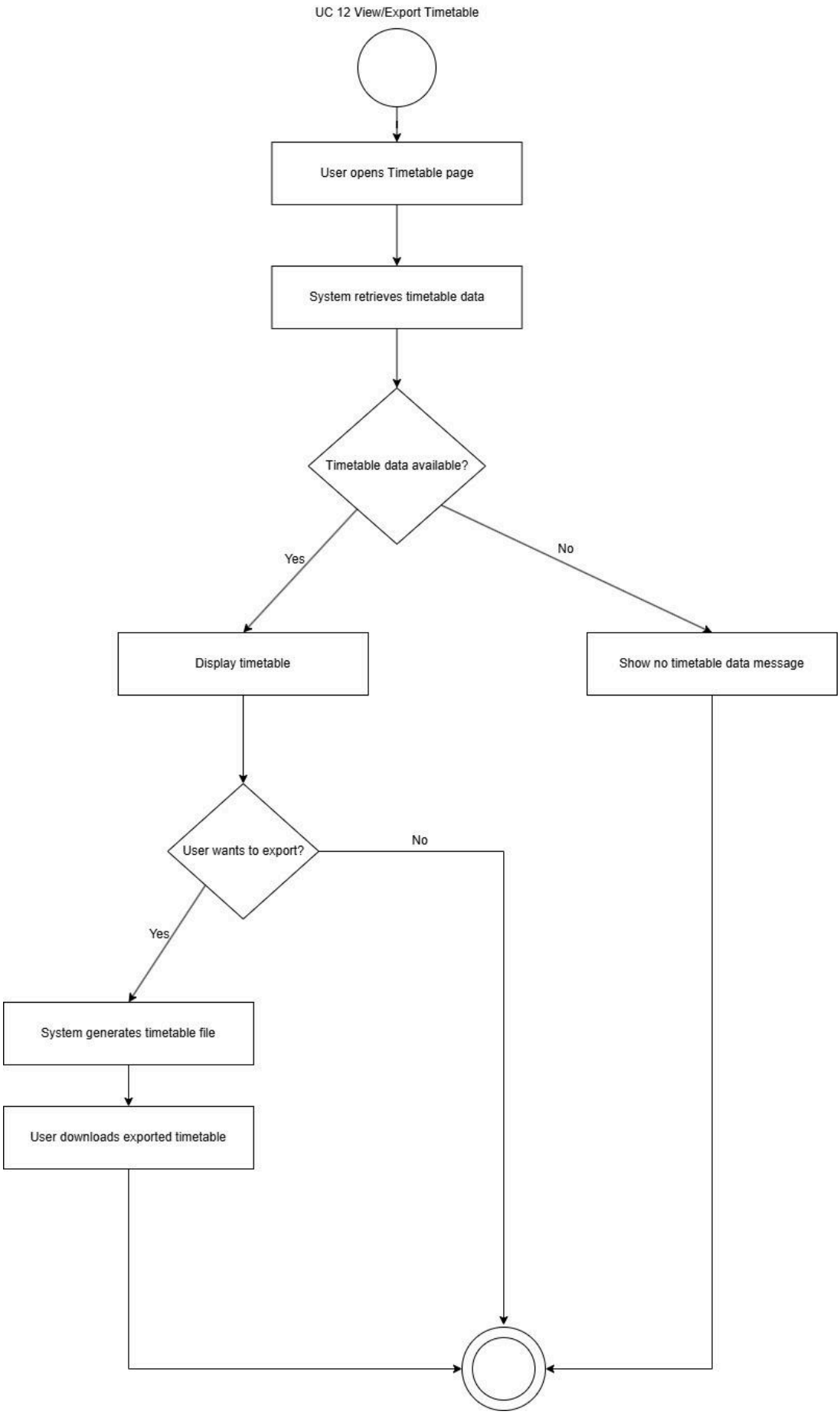

UC 07 Calculate Carbon Footprint
Farmer opens Carbon Footprint page
Farmer selects activity type
Farmer enters amount and date
System calculates carbon emission
Calculation successful?
Yes
No
System saves carbon log
Show error message
Display carbon footprint summary

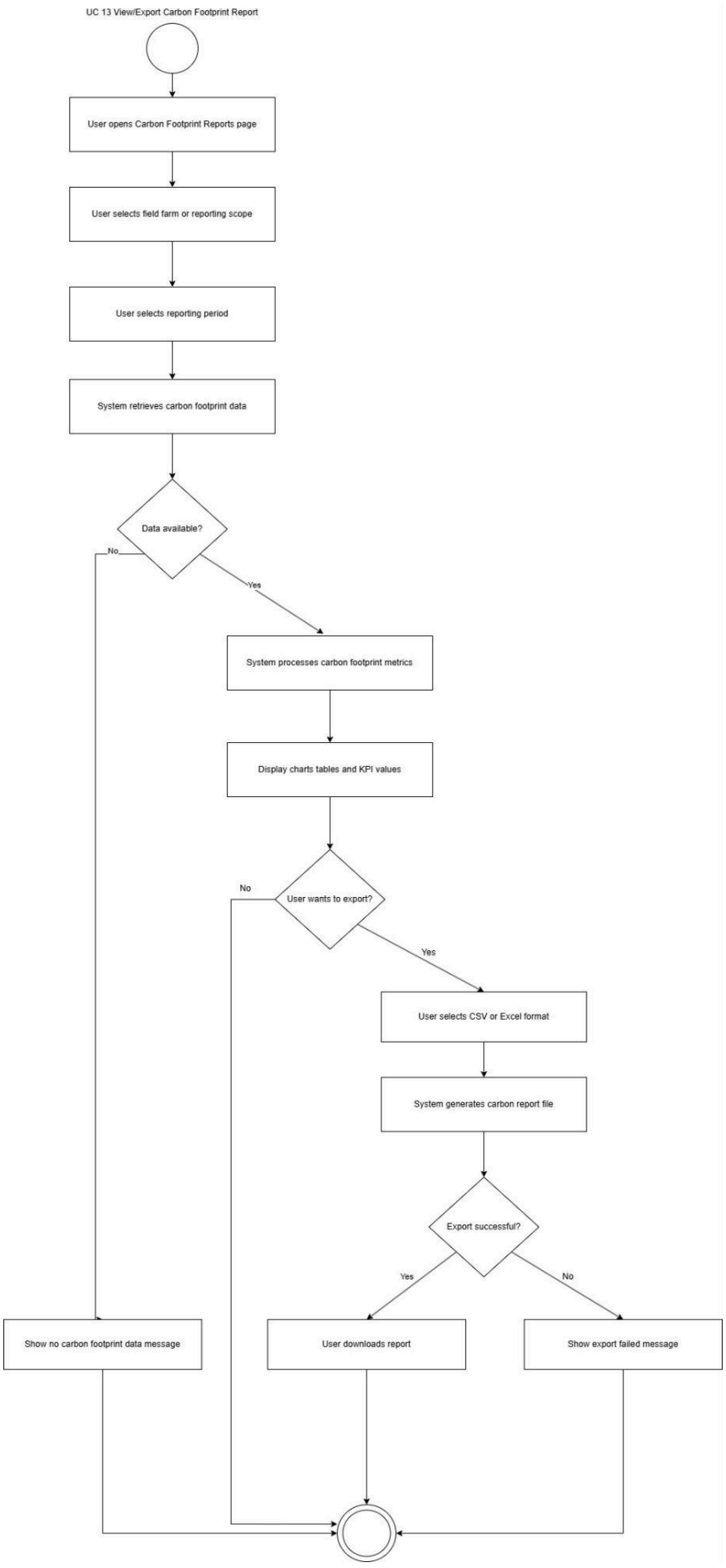

UC-8 Upload Plant Image
User authenticated
Open Disease Detection Page
Capture plant leaf image
Validate image type and size
Valid image?
No
Yes
Show error message
Upload image to backend
Upload successful?
No
Notify upload failure
Yes
Store image storage bucket
Send success acknowledgment

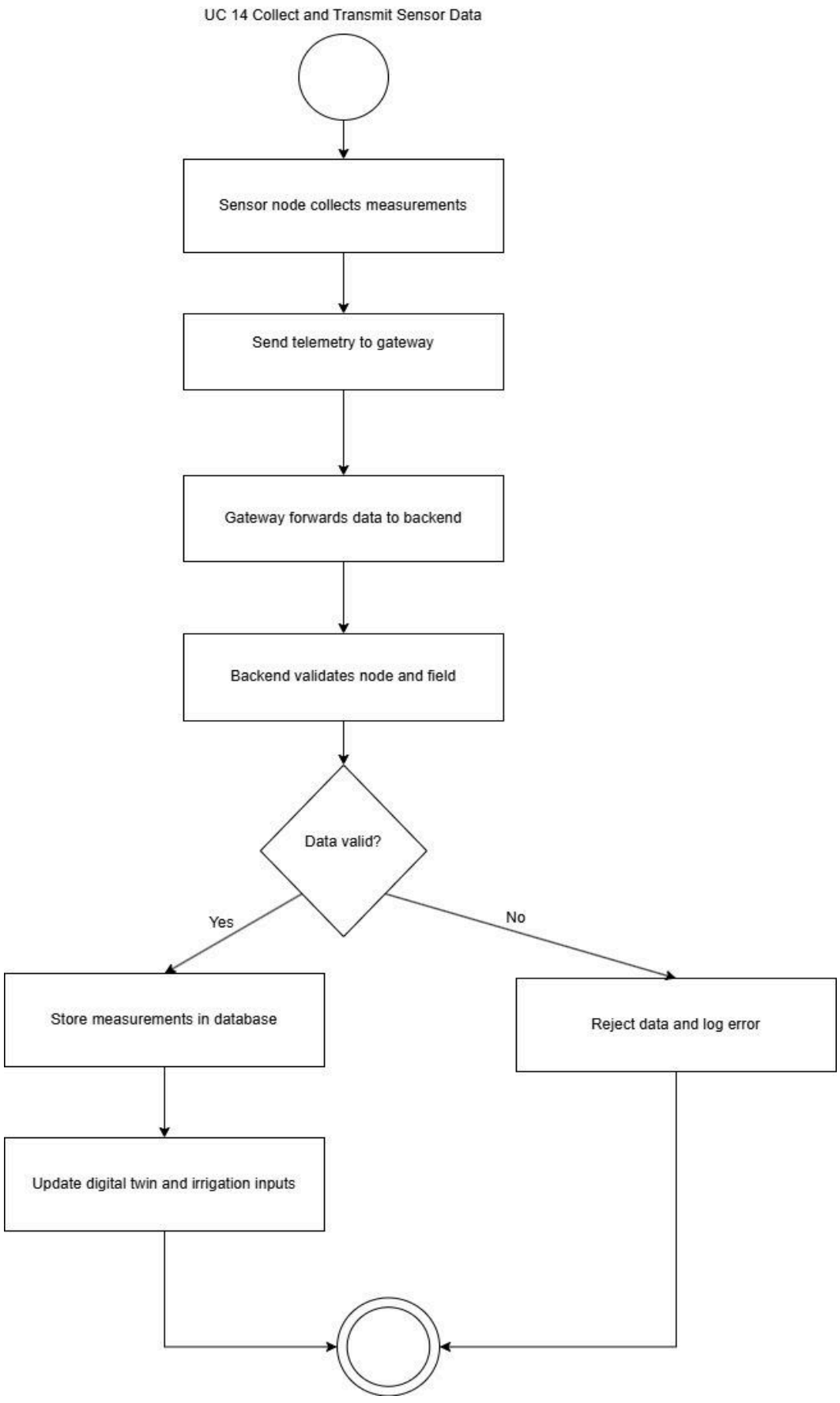

UC-9 Receive Disease Diagnosis
Valid image available
Preprocess uploaded image
Run disease detection model
Inference successful?
Yes
No
Generate disease label
Calculate confidence score
Determine severity level
Store diagnosis result in database
Send diagnosis result to user device
Display diagnosis to user
Handle inference error
Notify user: diagnosis unavailable


UC-10 View Recommended Actions
Diagnosis available
Retrieve diagnosed disease
Match disease with recommendation rules
Recommendation exists?
Yes
No
Generate treatment / monitoring actions
Prepare recommendation content
Display recommended actions to user
User reviews suggested actions
Show no recommendation available message

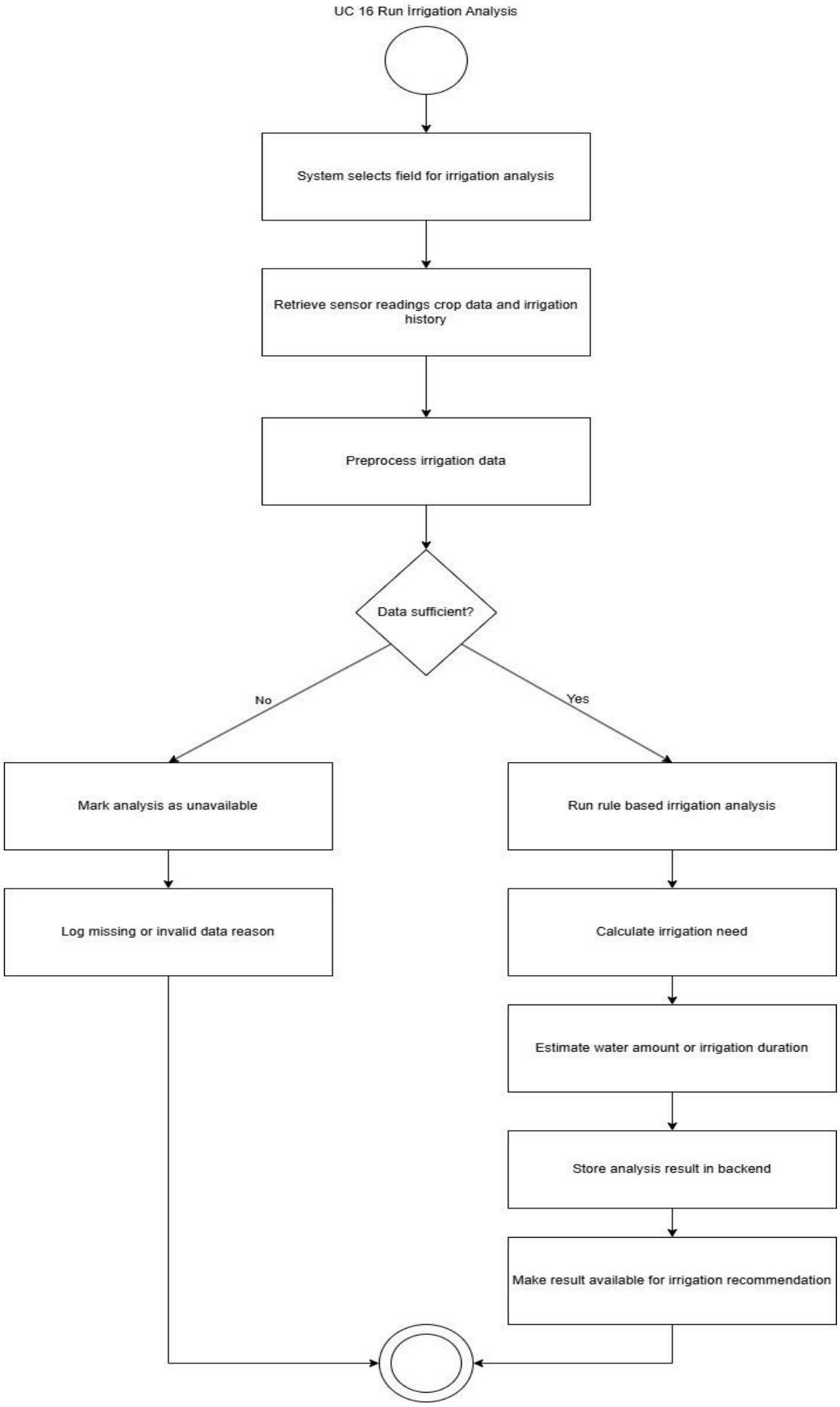

UC-11 Stakeholder Login to Dashboard
Stakeholder opens dashboard login page
Enter username or email and password
Submit login request
Verify credentials
Credentials valid?
Yes
No
Account active and authorized?
Yes
No
Initialize stakeholder session
Load stakeholder dashboard
Display invalid credentials message
Display access denied message
Display summaries and reporting tools

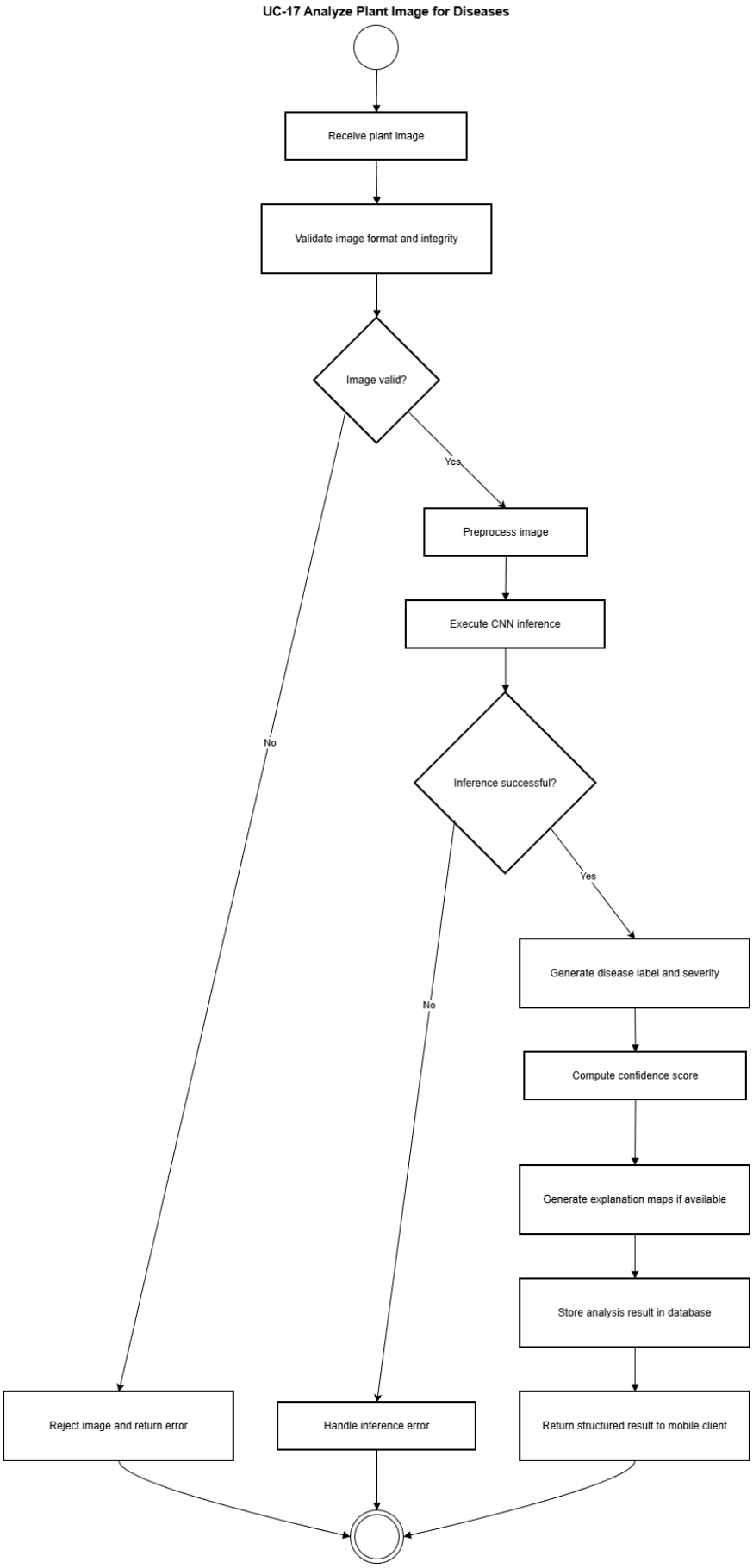

UC 12 View/Export Timetable
User opens Timetable page
System retrieves timetable data
Timetable data available?
Yes
No
Display timetable
Show no timetable data message
User wants to export?
No
Yes
System generates timetable file
User downloads exported timetable


UC 13 View/Export Carbon Footprint Report
User opens Carbon Footprint Reports page
User selects field farm or reporting scope
User selects reporting period
System retrieves carbon footprint data
Data available?
No
Yes
System processes carbon footprint metrics
Display charts tables and KPI values
User wants to export?
No
Yes
User selects CSV or Excel format
System generates carbon report file
Export successful?
Yes
No
Show no carbon footprint data message
User downloads report
Show export failed message

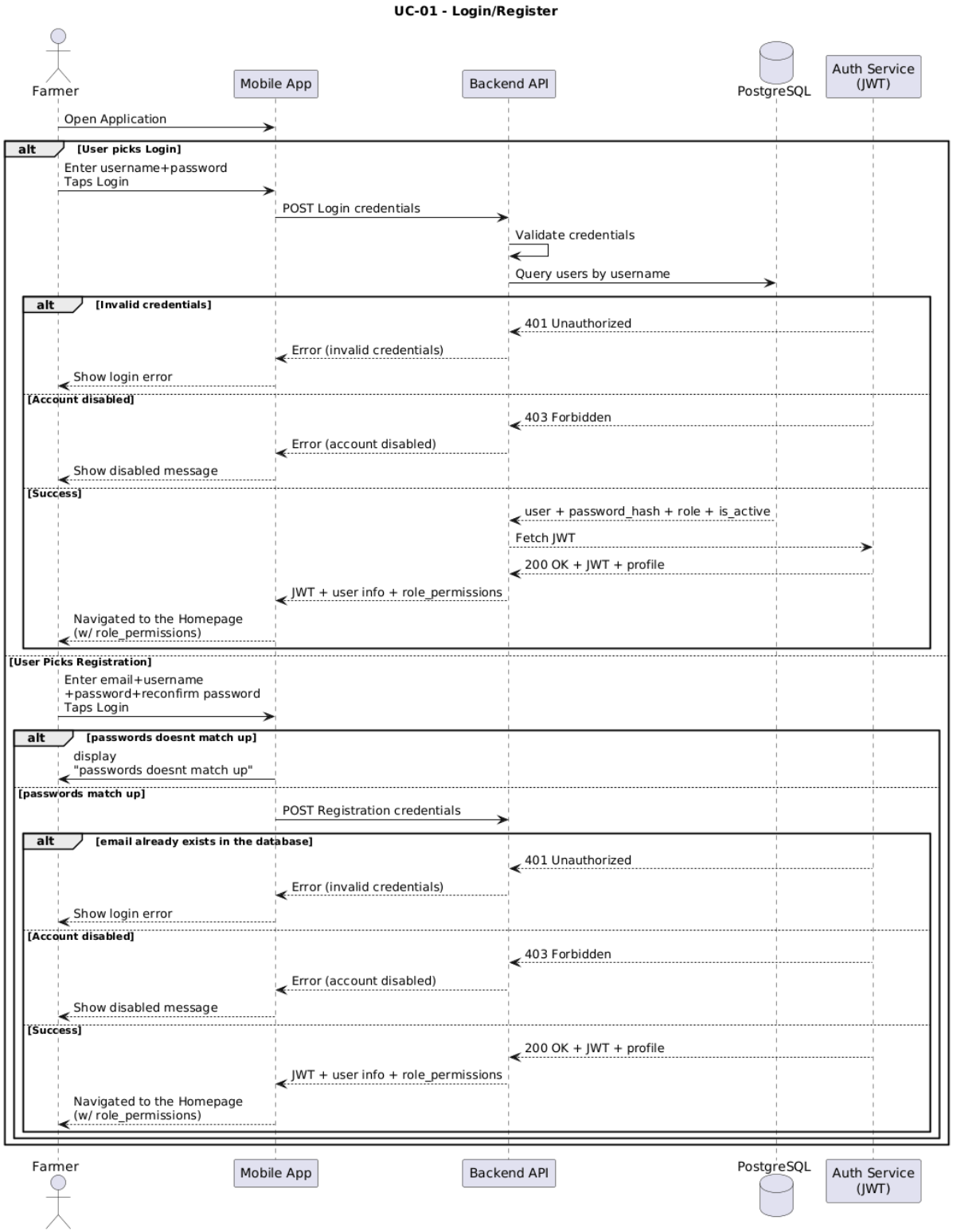

UC 14 Collect and Transmit Sensor Data
Sensor node collects measurements
Send telemetry to gateway
Gateway forwards data to backend
Backend validates node and field
Data valid?
Yes
No
Store measurements in database
Reject data and log error
Update digital twin and irrigation inputs

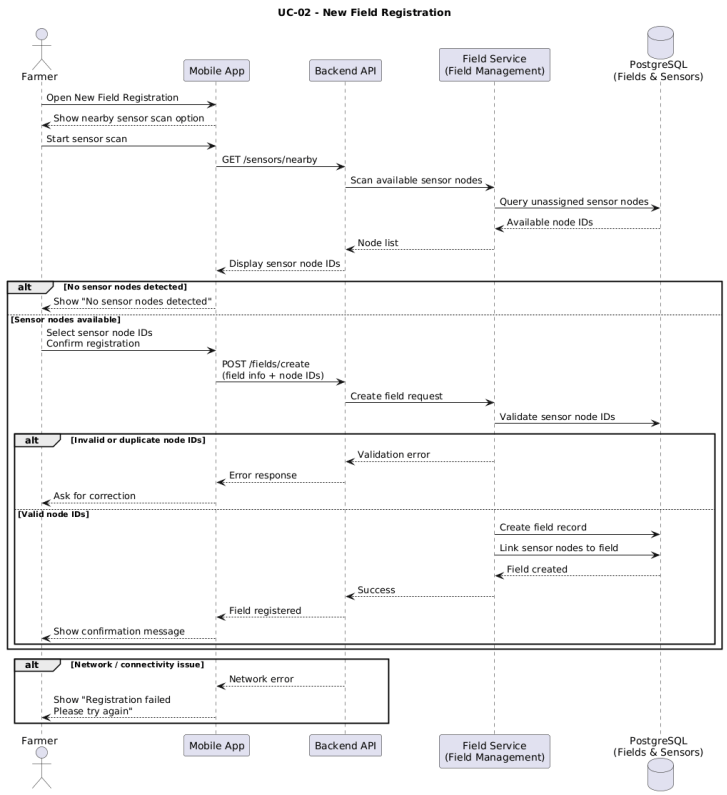

UC 15 Sync Field Data to Backend
New field data becomes available
System prepares data for backend sync
Connectivity available?
No
Yes
Store data locally
Retry synchronization later
Sync failed repeatedly?
No
Yes
Show sync warning or status notification
Send field data to backend
Backend stores data in database
Update related field records
Make updated data available for dashboard and analysis

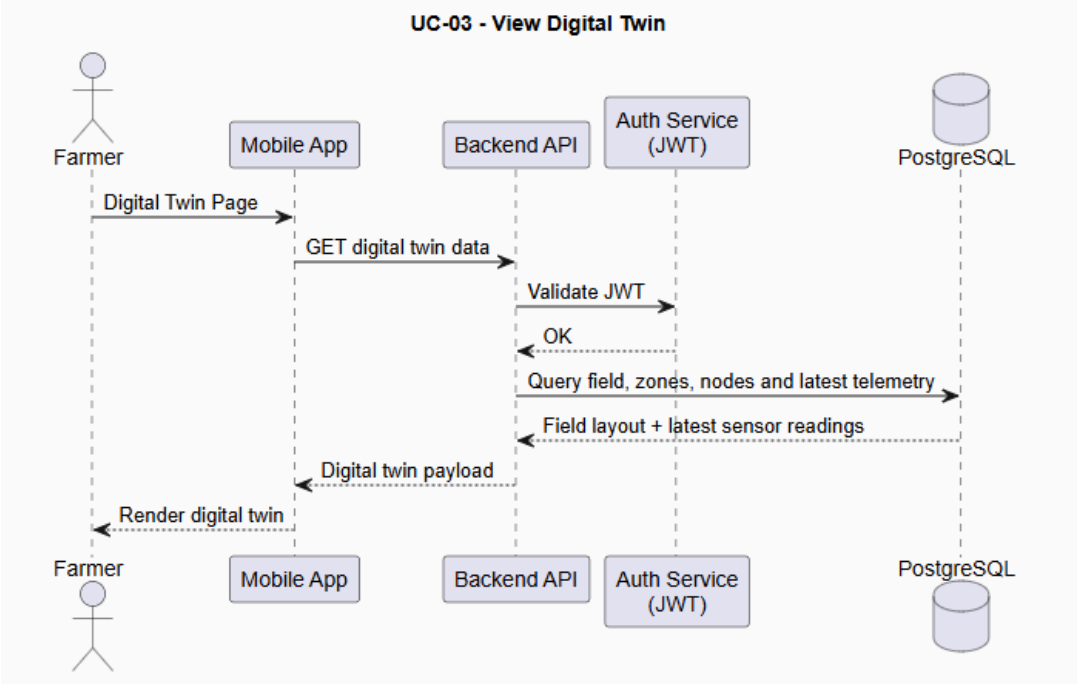

UC 16 Run İrrigation Analysis
System selects field for irrigation analysis
Retrieve sensor readings crop data and irrigation history
Preprocess irrigation data
Data sufficient?
No
Yes
Mark analysis as unavailable
Log missing or invalid data reason
Run rule based irrigation analysis
Calculate irrigation need
Estimate water amount or irrigation duration
Store analysis result in backend
Make result available for irrigation recommendation

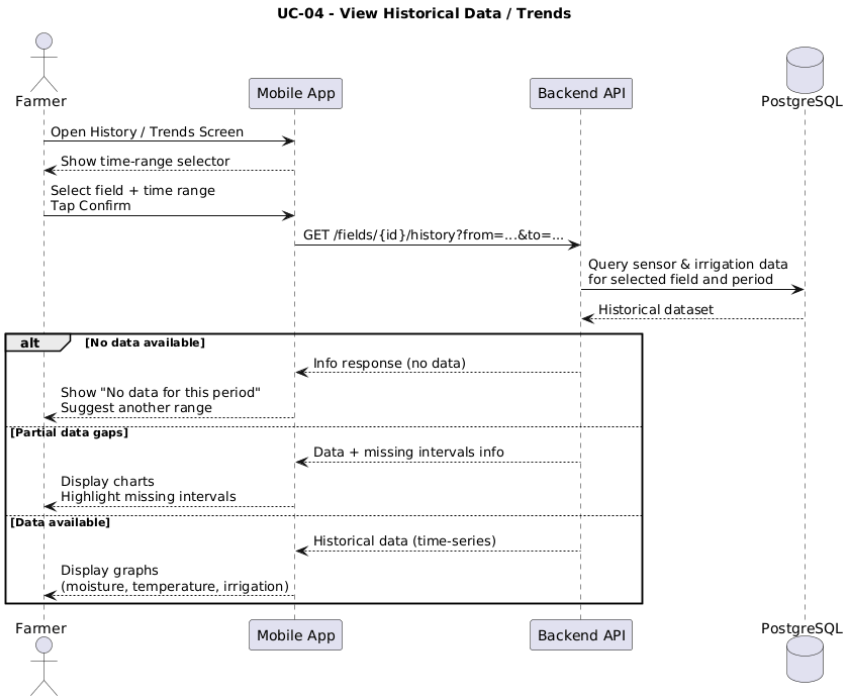

UC-17 Analyze Plant Image for Diseases
Receive plant image
Validate image format and integrity
Image valid?
Yes
No
Preprocess image
Execute CNN inference
Inference successful?
Yes
No
Generate disease label and severity
Compute confidence score
Generate explanation maps if available
Store analysis result in database
Return structured result to mobile client
Reject image and return error
Handle inference error

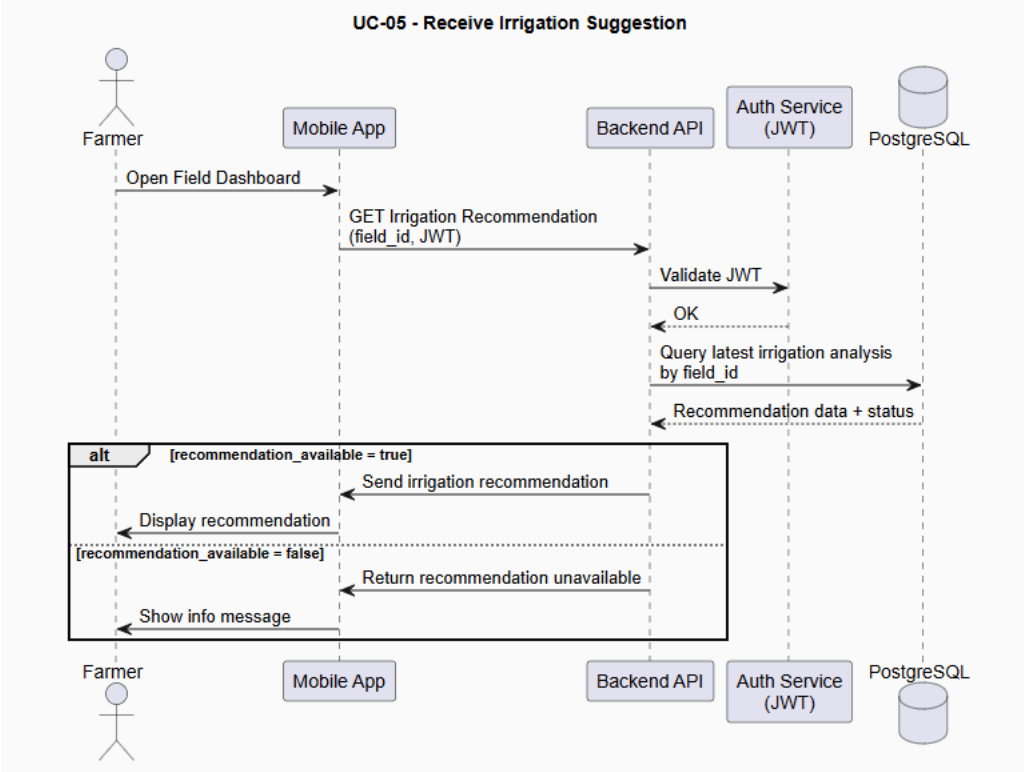
3.3.3.2. Sequence Diagrams

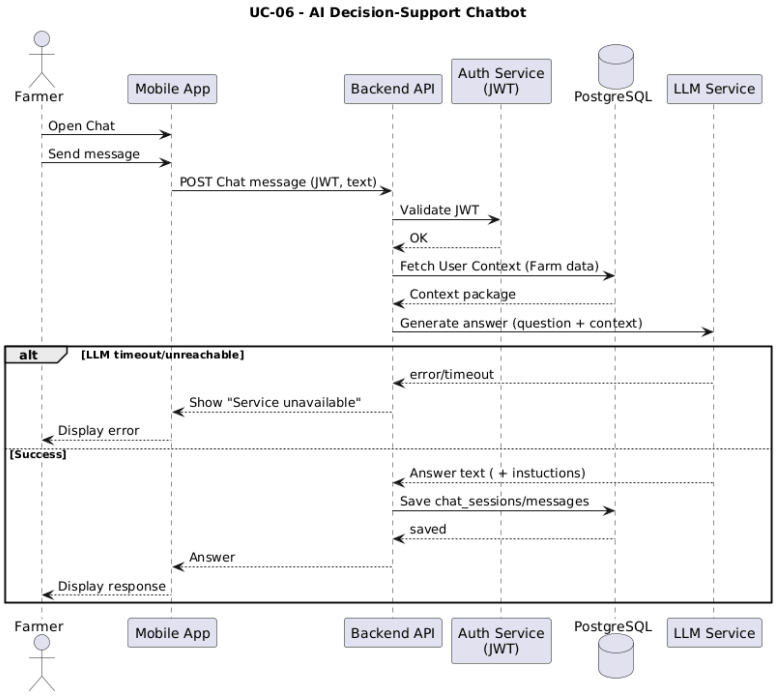


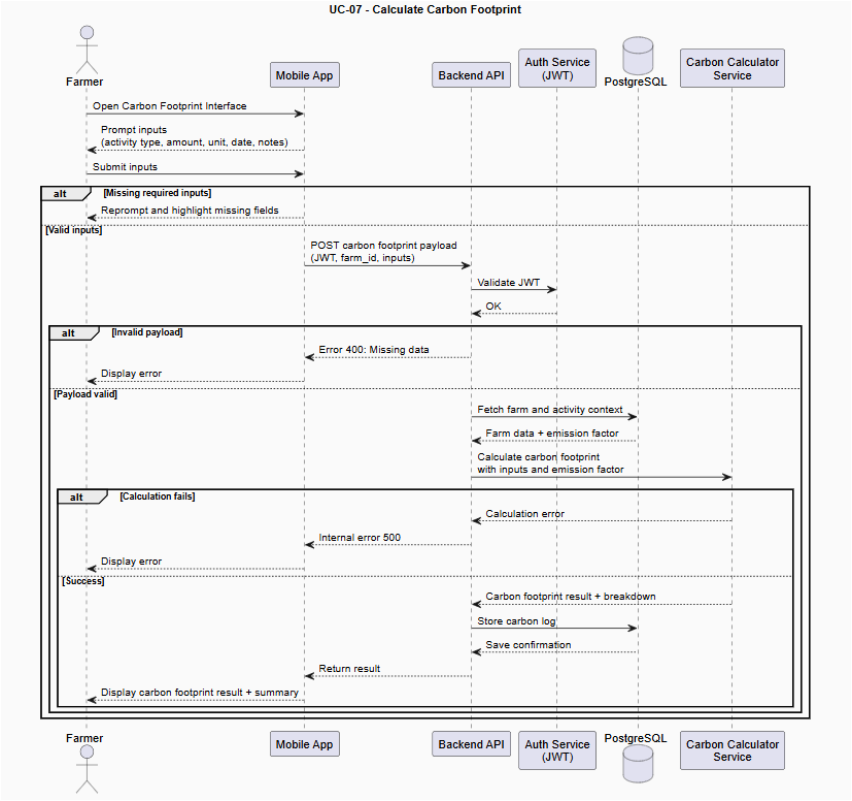

UC-02 - New Field Registration
Farmer
Mobile App
Backend API
Field Service (Field Management)
PostgreSQL (Fields & Sensors)
Open New Field Registration
Show nearby sensor scan option
Start sensor scan
GET /sensors/nearby
Scan available sensor nodes
Query unassigned sensor nodes
Available node IDs
Node list
Display sensor node IDs
alt
[No sensor nodes detected]
Show "No sensor nodes detected"
[Sensor nodes available]
Select sensor node IDs
Confirm registration
POST /fields/create (field info + node IDs)
Create field request
Validate sensor node IDs
alt
[Invalid or duplicate node IDs]
Validation error
Error response
Ask for correction
[Valid node IDs]
Create field record
Link sensor nodes to field
Field created
Success
Field registered
Show confirmation message
alt
[Network / connectivity issue]
Network error
Show "Registration failed Please try again"
Farmer
Mobile App
Backend API
Field Service (Field Management)
PostgreSQL (Fields & Sensors)


UC-03 - View Digital Twin
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
Digital Twin Page
GET digital twin data
Validate JWT
OK
Query field, zones, nodes and latest telemetry
Field layout + latest sensor readings
Digital twin payload
Render digital twin
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL

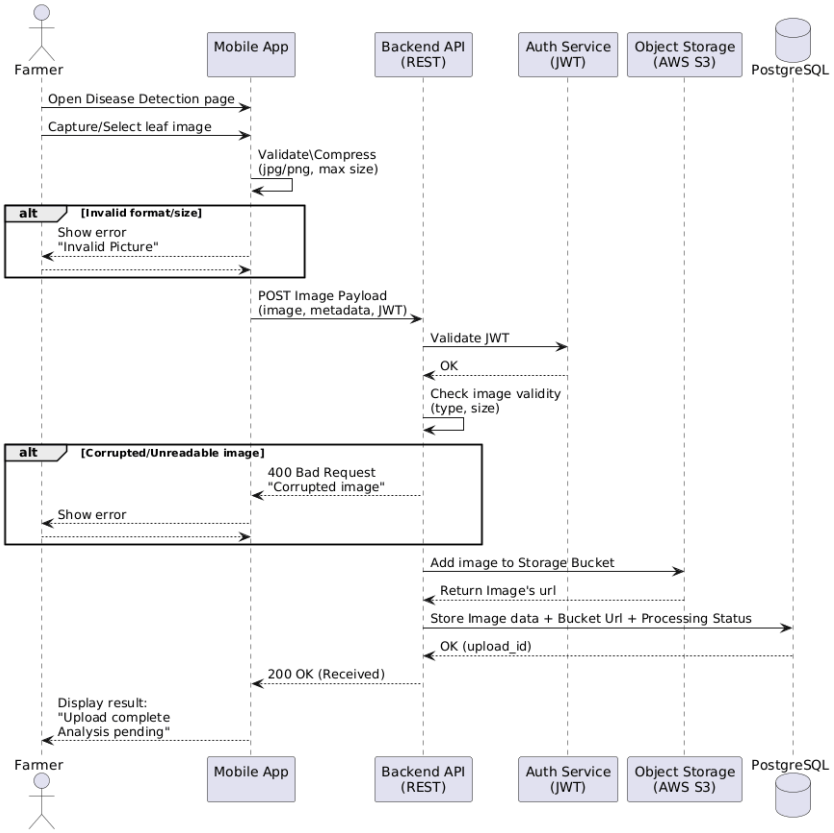

UC-04 - View Historical Data / Trends
Farmer
Mobile App
Backend API
PostgreSQL
Open History / Trends Screen
Show time-range selector
Select field + time range
Tap Confirm
GET /fields/{id}/history?from=...&to=...
Query sensor & irrigation data
for selected field and period
Historical dataset
alt
[No data available]
Info response (no data)
Show "No data for this period"
Suggest another range
[Partial data gaps]
Data + missing intervals info
Display charts
Highlight missing intervals
[Data available]
Historical data (time-series)
Display graphs
(moisture, temperature, irrigation)
Farmer
Mobile App
Backend API
PostgreSQL


UC-05 - Receive Irrigation Suggestion
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL
Open Field Dashboard
GET Irrigation Recommendation
(field_id, JWT)
Validate JWT
OK
Query latest irrigation analysis
by field_id
Recommendation data + status
alt
[recommendation_available = true]
Send irrigation recommendation
Display recommendation
[recommendation_available = false]
Return recommendation unavailable
Show info message
Farmer
Mobile App
Backend API
Auth Service
(JWT)
PostgreSQL

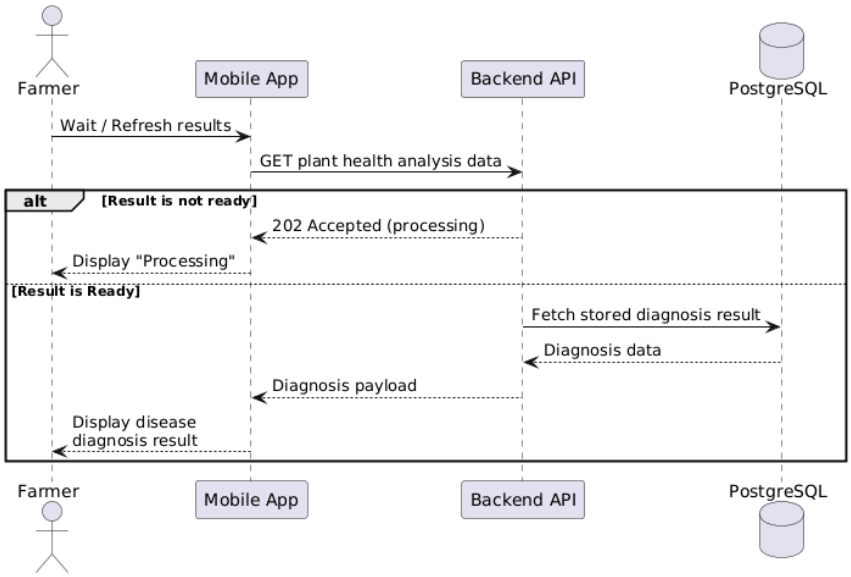

UC-06 - AI Decision-Support Chatbot
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
LLM Service
Open Chat
Send message
POST Chat message (JWT, text)
Validate JWT
OK
Fetch User Context (Farm data)
Context package
Generate answer (question + context)
alt
[LLM timeout/unreachable]
error/timeout
Show "Service unavailable"
Display error
[Success]
Answer text (+ instuctions)
Save chat_sessions/messages
saved
Answer
Display response
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
LLM Service


UC-07 - Calculate Carbon Footprint
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
Carbon Calculator Service
Open Carbon Footprint Interface
Prompt inputs (activity type, amount, unit, date, notes)
Submit inputs
alt
[Missing required inputs]
Reprompt and highlight missing fields
[Valid inputs]
POST carbon footprint payload (JWT, farm_id, inputs)
Validate JWT
OK
alt
[Invalid payload]
Error 400: Missing data
Display error
[Payload valid]
Fetch farm and activity context
Farm data + emission factor
Calculate carbon footprint with inputs and emission factor
alt
[Calculation fails]
Calculation error
Internal error 500
Display error
[Success]
Carbon footprint result + breakdown
Store carbon log
Save confirmation
Return result
Display carbon footprint result + summary
Farmer
Mobile App
Backend API
Auth Service (JWT)
PostgreSQL
Carbon Calculator Service

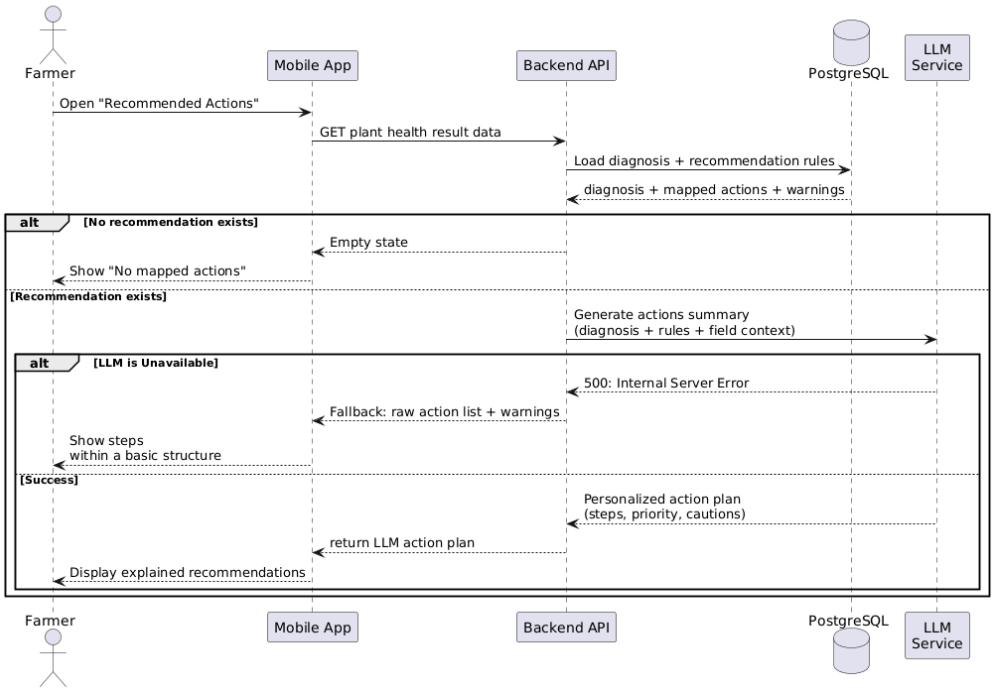
UC-08 - Upload Plant Image



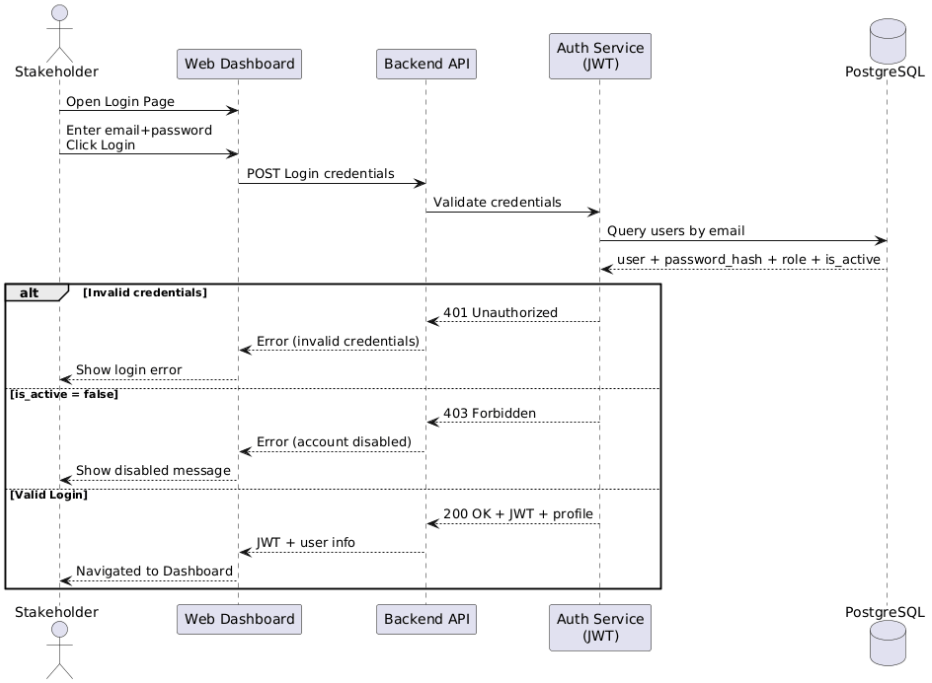
UC-09 - Receive Disease Diagnosis

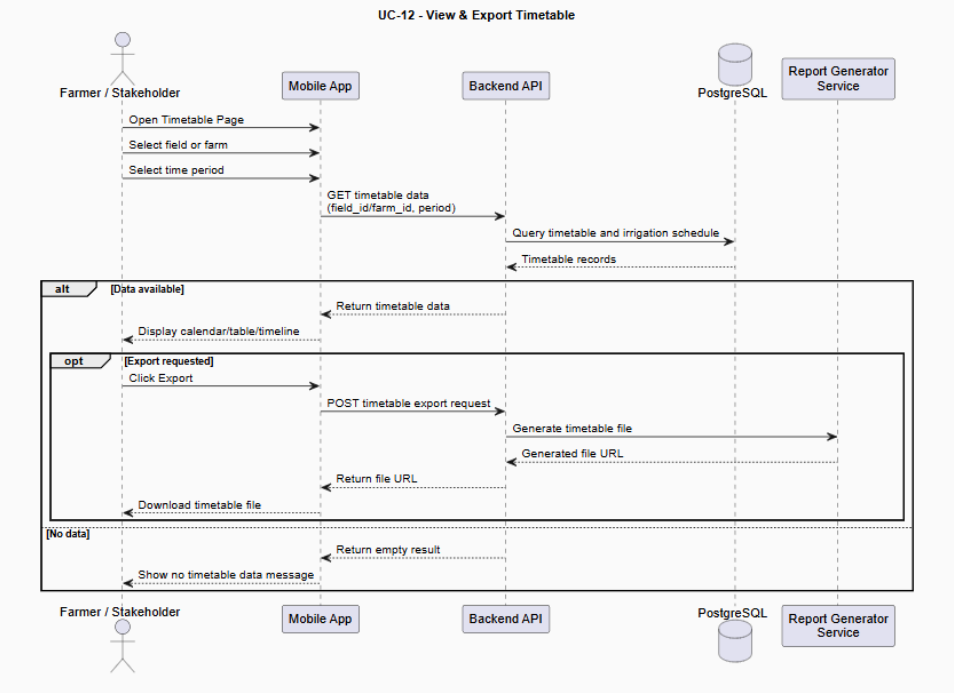


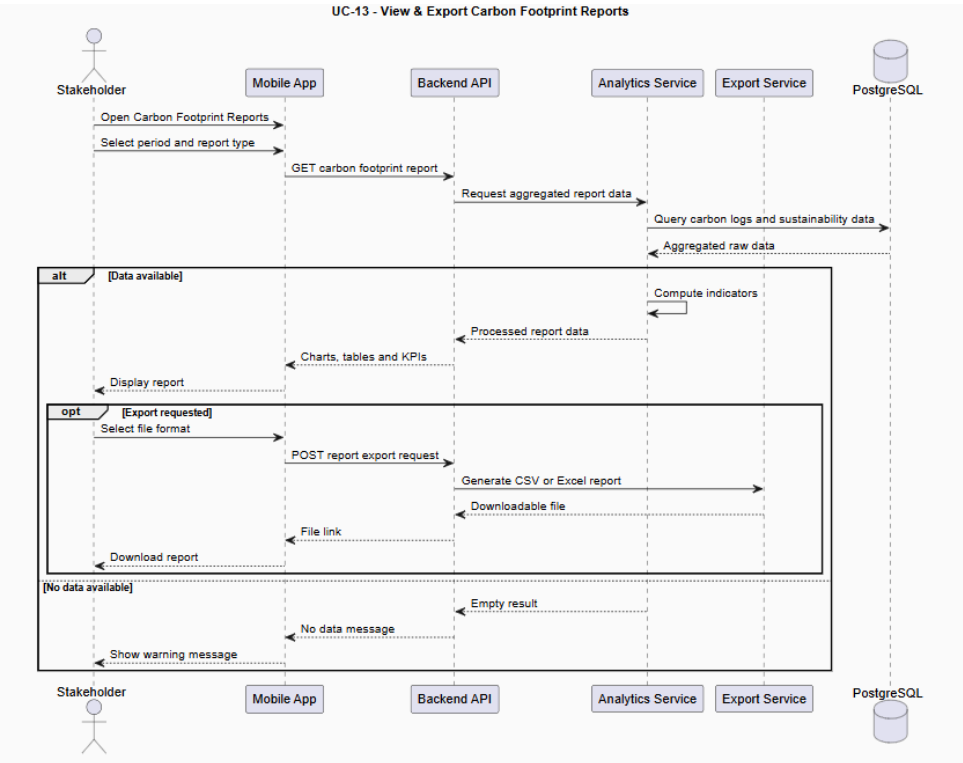
UC-10 - View Recommended Actions

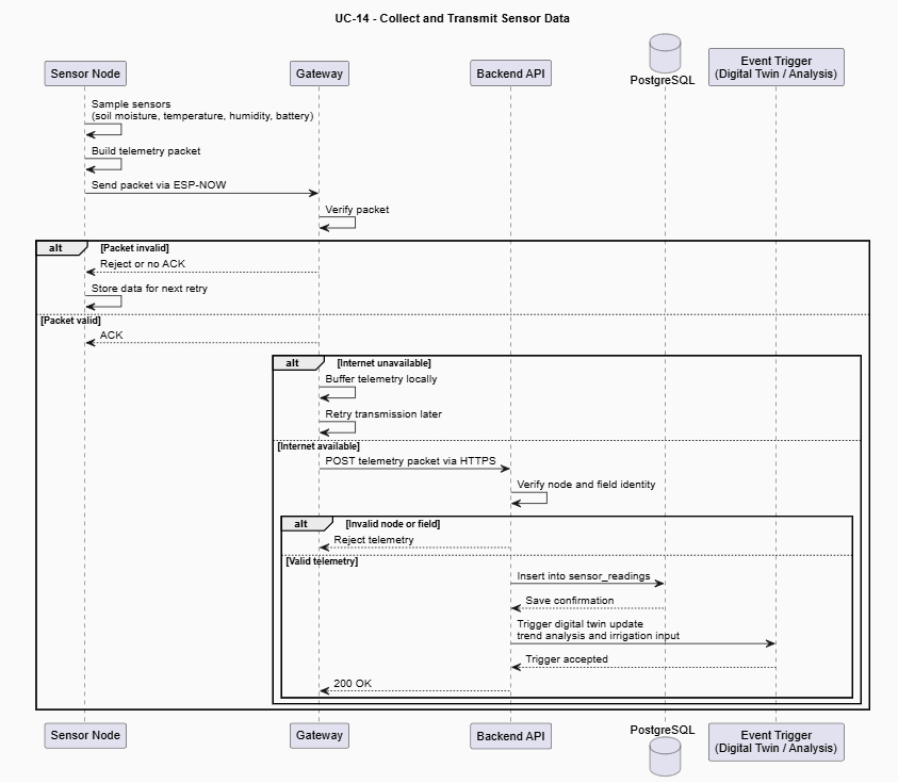


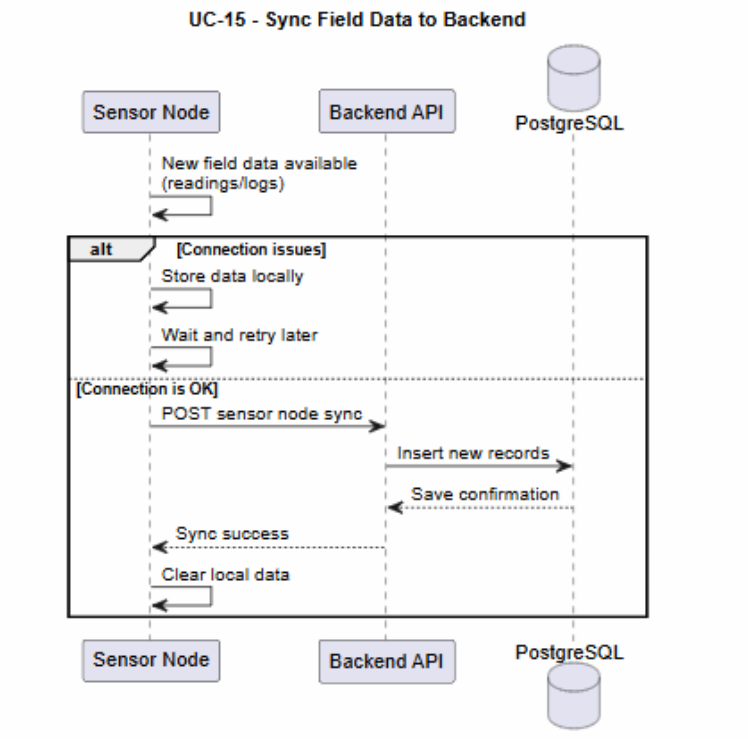
UC-11 - Stakeholder Login

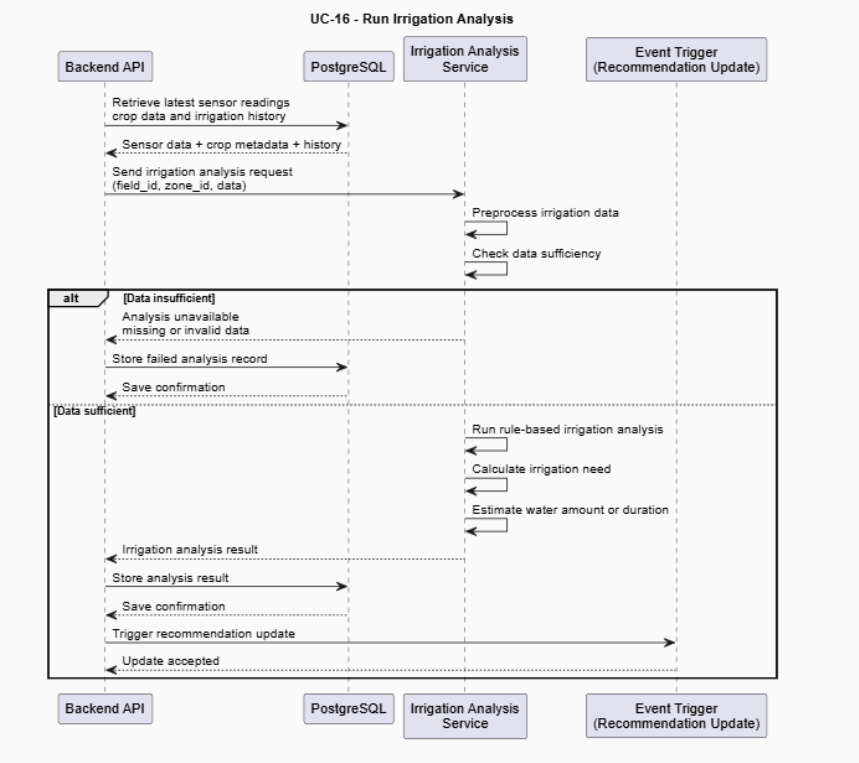


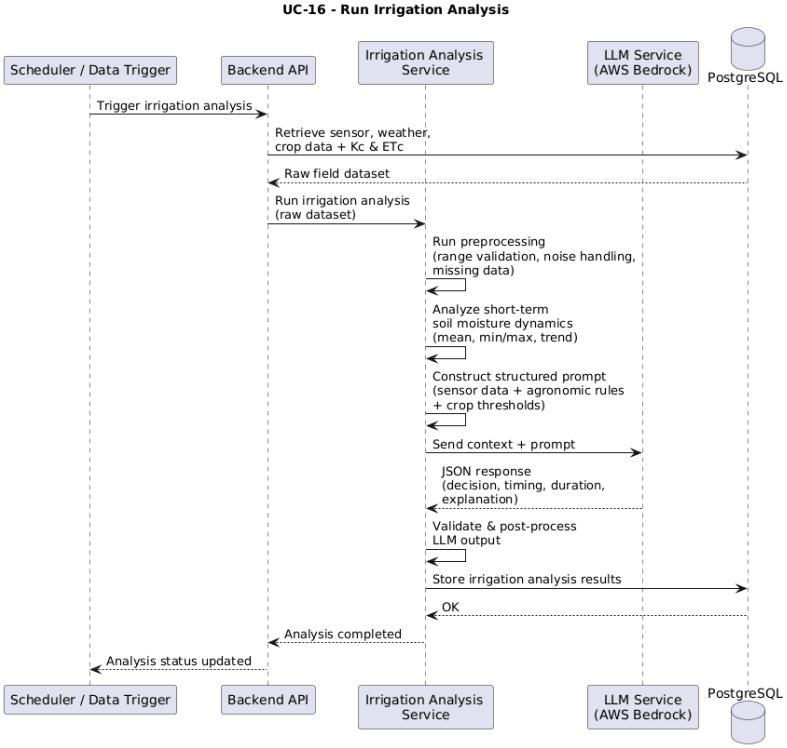

UC-12 - View & Export Timetable
Farmer / Stakeholder
Mobile App
Backend API
PostgreSQL
Report Generator Service
Open Timetable Page
Select field or farm
Select time period
GET timetable data (field_id/farm_id, period)
Query timetable and irrigation schedule
Timetable records
alt
[Data available]
Return timetable data
Display calendar/table/timeline
opt
[Export requested]
Click Export
POST timetable export request
Generate timetable file
Generated file URL
Return file URL
Download timetable file
[No data]
Return empty result
Show no timetable data message

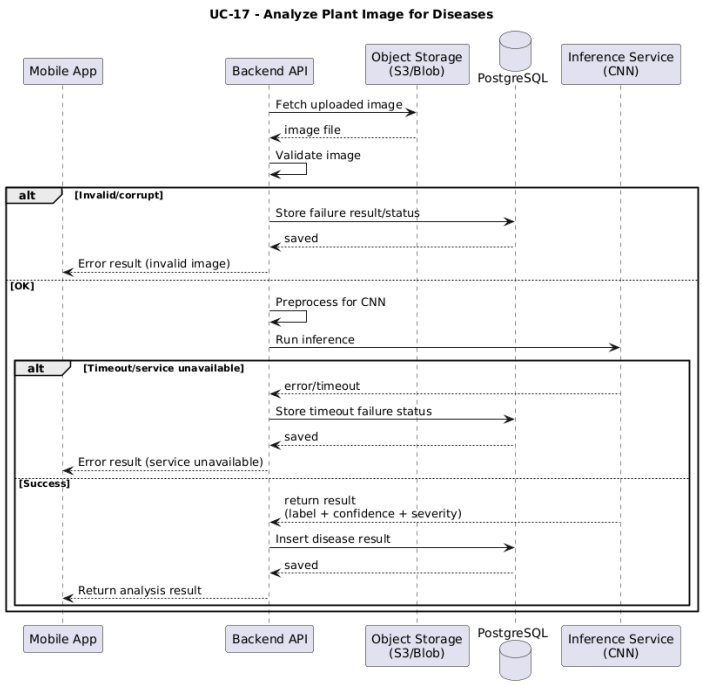

UC-13 - View & Export Carbon Footprint Reports
Stakeholder
Mobile App
Backend API
Analytics Service
Export Service
PostgreSQL
Open Carbon Footprint Reports
Select period and report type
GET carbon footprint report
Request aggregated report data
Query carbon logs and sustainability data
Aggregated raw data
alt
[Data available]
Compute indicators
Processed report data
Charts, tables and KPIs
Display report
opt
[Export requested]
Select file format
POST report export request
Generate CSV or Excel report
Downloadable file
File link
Download report
[No data available]
Empty result
No data message
Show warning message


UC-14 - Collect and Transmit Sensor Data
Sensor Node
Gateway
Backend API
PostgreSQL
Event Trigger (Digital Twin / Analysis)
Sample sensors (soil moisture, temperature, humidity, battery)
Build telemetry packet
Send packet via ESP-NOW
Verify packet
alt
[Packet invalid]
Reject or no ACK
Store data for next retry
[Packet valid]
ACK
alt
[Internet unavailable]
Buffer telemetry locally
Retry transmission later
[Internet available]
POST telemetry packet via HTTPS
Verify node and field identity
alt
[Invalid node or field]
Reject telemetry
[Valid telemetry]
Insert into sensor_readings
Save confirmation
Trigger digital twin update trend analysis and irrigation input
Trigger accepted
200 OK
Sensor Node
Gateway
Backend API
PostgreSQL
Event Trigger (Digital Twin / Analysis)


UC-15 - Sync Field Data to Backend
Sensor Node
Backend API
PostgreSQL
New field data available (readings/logs)
alt
[Connection issues]
Store data locally
Wait and retry later
[Connection is OK]
POST sensor node sync
Insert new records
Save confirmation
Sync success
Clear local data
Sensor Node
Backend API
PostgreSQL


UC-16 - Run Irrigation Analysis
Backend API
PostgreSQL
Irrigation Analysis Service
Event Trigger (Recommendation Update)
Retrieve latest sensor readings crop data and irrigation history
Sensor data + crop metadata + history
Send irrigation analysis request (field_id, zone_id, data)
Preprocess irrigation data
Check data sufficiency
alt
[Data insufficient]
Analysis unavailable missing or invalid data
Store failed analysis record
Save confirmation
[Data sufficient]
Run rule-based irrigation analysis
Calculate irrigation need
Estimate water amount or duration
Irrigation analysis result
Store analysis result
Save confirmation
Trigger recommendation update
Update accepted
Backend API
PostgreSQL
Irrigation Analysis Service
Event Trigger (Recommendation Update)


UC-16 - Run Irrigation Analysis
Scheduler / Data Trigger
Backend API
Irrigation Analysis Service
LLM Service (AWS Bedrock)
PostgreSQL
Trigger irrigation analysis
Retrieve sensor, weather, crop data + Kc & ETc
Raw field dataset
Run irrigation analysis (raw dataset)
Run preprocessing (range validation, noise handling, missing data)
Analyze short-term soil moisture dynamics (mean, min/max, trend)
Construct structured prompt (sensor data + agronomic rules + crop thresholds)
Send context + prompt
JSON response (decision, timing, duration, explanation)
Validate & post-process LLM output
Store irrigation analysis results
OK
Analysis completed
Analysis status updated
Scheduler / Data Trigger
Backend API
Irrigation Analysis Service
LLM Service (AWS Bedrock)
PostgreSQL



3.4. Methodology

3.4.1. Irrigation Recommendation

TARAS does not use a fully data-driven ML model for irrigation recommendation. Instead, it uses an agronomy-based rule engine supported by real-time sensor data and zone-specific calibration values.

The backend evaluates soil moisture, temperature, humidity, crop type, environment type, irrigation method, and growth stage. For tomato, growth-stage-based moisture thresholds determine whether irrigation is needed. In greenhouse zones, TARAS estimates irrigation duration using Kc values, simplified ET0, soil moisture deficit, and irrigation calibration. In pot-based manual zones, it calculates the required water amount in milliliters.

The output includes irrigation need, timing, duration or water volume, urgency level, predicted soil moisture after irrigation, follow-up check time, and reasoning. The LLM is used as an advisory layer to explain recommendations, while the actual decision logic remains rule-based and transparent.

3.4.1.1. Problem Formulation

The irrigation recommendation problem in TARAS is formulated as a rule-guided decision-making task. It determines whether irrigation is needed by evaluating real-time sensor data, crop characteristics, environment type, irrigation method, and plant growth stage.

3.4.1.2. Data Acquisition and Preprocessing

In TARAS, irrigation decisions use structured data collected from sensors, zone metadata, and system calibration parameters.

The main input is real-time field sensor data, especially soil moisture percentage. Air temperature and humidity are also used to represent current environmental conditions. For each zone, TARAS uses the latest valid sensor readings and averages data from available sensor nodes when needed.

Crop and environment metadata include crop type, environment type, irrigation method, planting date, and growth stage. If the growth stage is not manually provided, it is estimated from the planting date. System parameters include growth-stage-based soil moisture thresholds, Kc values, irrigation gain values, pot calibration parameters, safety limits, and default follow-up check time.

Historical irrigation data are also used for calibration. Previous irrigation actions and follow-up soil moisture measurements help estimate how much soil moisture increases after a given water amount or irrigation duration.

Before generating a recommendation, the backend performs lightweight validation. It checks whether required values such as soil moisture, temperature, humidity, crop name, environment type, irrigation mode, and growth stage are available. It also applies safety limits to prevent excessive irrigation. These steps ensure that the rule-based irrigation engine receives consistent and reliable inputs while keeping the process transparent and explainable.

3.4.1.3. Algorithmic Approach

The irrigation recommendation process in TARAS follows a rule-guided backend workflow.

First, the system aggregates real-time sensor inputs such as soil moisture, temperature, and humidity for each zone. It also retrieves crop information, environment type, irrigation method, planting data, growth stage, and calibration parameters.

Next, the backend determines the plant growth stage and compares the current soil moisture value with growth-stage-based thresholds. If the zone is a greenhouse, TARAS estimates irrigation duration using Kc values, simplified ET0, soil moisture deficit, and irrigation gain calibration[44]. If the zone uses pot-based manual

irrigation, it calculates the required water amount in milliliters using soil moisture deficit and pot calibration values.

The system then applies safety limits, estimates the predicted soil moisture after irrigation, sets urgency level, and defines a follow-up check time. The final output is stored as a structured irrigation recommendation including decision, timing, duration or water amount, predicted result, urgency, and reasoning.

3.4.1.4. System Evaluation Metrics

TARAS evaluates irrigation performance using operational and decision-oriented metrics instead of traditional ML metrics.

The main evaluation criterion is whether irrigation recommendations help soil moisture move toward the target range after irrigation. Threshold compliance is also monitored by checking whether soil moisture stays within growth-stage-based optimal and critical limits.

The system evaluates over-irrigation and under-irrigation by tracking cases where soil moisture rises above the recommended range or drops below critical thresholds. Recommendation stability is assessed by observing whether similar zone conditions produce consistent decisions over time.

Historical irrigation records and follow-up measurements are used for validation and calibration. By comparing recommended water amount or duration with the actual soil moisture change after irrigation, TARAS can improve zone-specific calibration values and make future recommendations more consistent.

3.4.2. Disease Detection

3.4.2.1. Problem Formulation

This module is designed as an image processing and ML component of the TARAS system, focusing on the analysis of plant leaf images for disease identification. The primary objective is to enable users to capture or upload a leaf image, after which the system automatically detects the presence of plant disease and classifies its type.

Formally, the task is modeled as a multi-class image classification problem. Each input image **X** is mapped to a disease label **Y**, selected from a predefined set of categories representing both healthy and diseased plant conditions. The model aims to achieve high classification accuracy while ensuring robust generalization performance on previously unseen data.

3.4.2.2. Data Acquisition and Preprocessing

3.4.2.2.1. Data Acquisition

Plant leaf images used in this study are primarily obtained from the publicly available PlantVillage dataset, which provides a large collection of labeled images under controlled conditions[7]. To improve real-world applicability and model generalization, an additional dataset was constructed to better represent field conditions. This supplementary dataset was collected using a combination of automated image extraction tools (implemented in Python) and manual collection processes.

By combining the structured and balanced nature of the PlantVillage dataset with the variability of real-world images, the overall dataset aims to better capture practical conditions encountered in real deployments.

3.4.2.2.2. Preprocessing

Images are resized so the shorter side is 256 pixels and then centre-cropped to the model input of 224 × 224. Pixel values are normalized with the standard ImageNet mean and standard deviation to match the ImageNet-pretrained backbones.

To improve generalization, especially due to the domain gap between the controlled PlantVillage dataset and real-world images, data augmentation techniques are applied. These include random rotations, horizontal flips, slight zooming, and brightness variations, simulating diverse field conditions such as different lighting, orientations, and capture angles. This preprocessing pipeline ensures both computational efficiency and robustness under real-world deployment scenarios.

3.4.2.3. Algorithmic Approach

The disease detection module uses a deep-learning image-classification approach. On the server, classification is performed by an ensemble of modern convolutional and transformer backbones (ConvNeXtV2-Base [45], [21] and two Swin-Transformer models [22]); a compact EfficientNet-B0 model [24], distilled from this ensemble [4], runs on-device for low-latency inference.

Preprocessed images are passed through the CNN backbone, which extracts high-level visual features such as texture, color, and structural patterns. These features enable the model to distinguish between healthy and diseased leaves, as well as different disease categories.

A pretrained model is used to leverage general visual knowledge from large-scale datasets, improving robustness under varying real-world conditions such as lighting changes, background variability, and leaf orientation.

The final prediction is generated through a classification layer that maps extracted features to predefined disease classes, selecting the label with the highest probability.

3.4.2.4. Model Selection and Comparative Evaluation

Several backbone families were considered, spanning lightweight CNNs (MobileNetV3, EfficientNet-B0), deeper CNNs (ResNet, EfficientNet), and modern convolutional/transformer architectures (ConvNeXt, Swin).

The deployed solution combines a high-accuracy server-side ensemble (ConvNeXtV2-Base + Swin-Small + Swin-Base, equal-weight softmax average) with a distilled EfficientNet-B0 student for on-device use, balancing accuracy against latency and model size.

3.4.2.5. System Evaluation Metrics

Model performance is to be evaluated using multiple complementary metrics to capture different aspects of classification quality:

- **Accuracy**: Overall proportion of correctly classified images
- **Precision**: Ability of the model to avoid false disease predictions
- **Recall**: Ability of the model to correctly identify diseased samples
- **F1-score**: Harmonic mean of precision and recall

3.4.2.6. Implementation Details

The disease detection module is implemented in PyTorch (using the timm model library) and trained offline. For deployment it is exported in two forms: the server ensemble is served on AWS Lambda (PyTorch, with an ONNX-Runtime variant).

The distilled EfficientNet-B0 student is exported to TensorFlow Lite (fp16) for on-device inference in the mobile app. The module classifies a preprocessed leaf image and outputs the most probable disease label among the 14 classes; its modular structure allows independent model updates without affecting the rest of the system.

3.4.3. LLM-Based Advisory & Decision Support

3.4.3.1. Problem Formulation

The LLM-based advisory component aims to translate complex agricultural data into clear, context-aware decision support. While the system's predictive modules output raw numbers and categorical labels (such as disease types or moisture levels), farmers often need these results explained in plain language. Therefore, we formulate this as a natural language reasoning task: transforming structured system data into actionable, easy-to-understand agronomic recommendations.

To act as an intelligent bridge between the raw data and the user, the language model processes two main inputs: the real-time system context (soil moisture, irrigation thresholds, disease status) and the user's direct query. By combining these, it turns raw metrics into practical advice.

3.4.3.2. Input Representation and Context Construction

The LLM needs a well-structured input to provide accurate answers. We build this context by combining irrigation predictions, disease detections, and current environmental data.

To prevent the model from confusing background system data with the user's chat message, we isolate the information. The assistant is given a cached system prompt that defines its role, brevity rules, a condensed agronomic reference, and a set of callable tools. Rather than receiving all field data up front, the model is provided only a lightweight inventory of the user’s farms, fields, zones (with their identifiers) including the chat history, and is instructed to fetch the specific data it needs, current sensor values, zone thresholds, irrigation history, carbon summary, disease history, active alerts, by calling the corresponding tools on demand. This keeps each request focused and avoids dumping unprompted technical reports into the conversation. This targeted approach prevents the assistant from giving unprompted, robotic technical reports and keeps the conversation natural and relevant to the farmer's needs.

3.4.3.3. Response Generation and Explainability

The assistant generates actionable advice by explaining the system's findings in everyday language. It explicitly references key data points, such as current soil moisture or visual disease symptoms, to back up its statements. This approach builds user trust because it goes beyond simply telling the farmer *what* to do; it explains *why* the recommendation is being made based on their specific field conditions.

3.4.3.4. System Evaluation and Limitations

Because the LLM acts as an interpreter rather than a prediction engine, evaluating it focuses on qualitative metrics: how closely it sticks to the provided data, how logical its answers are, and how useful the advice is to the farmer. The main goal is to maintain "zero-hallucination" boundaries—ensuring the model never invents facts.

However, the system has clear limitations. The assistant is only as reliable as the data it receives. If the IoT sensors or camera modules provide incorrect readings, the LLM will base its advice on that flawed data. Additionally, while the strict prompt prevents the AI from making things up, it also means the system might struggle to provide advice on situations it hasn't been programmed to handle, such as entirely new crop types or undocumented diseases. Therefore, the assistant is designed to be a helpful decision-support tool, engineered to assist—not replace—the judgment of expert agronomists.

3.4.3.5. Implementation Details

The advisory module is implemented as a service inside the TARAS Node.js/Express backend and is reached from the mobile app over a REST endpoint. On each request the backend builds the per-request context (the farm/field/zone inventory and recent history) and runs an agentic tool-calling loop with the language model: the model may issue one or more tool calls, each of which the backend executes against the PostgreSQL database via Prisma (live reads of sensor metrics, thresholds, irrigation jobs, alerts, etc.) before returning the results to the model; the loop continues until the model produces its final natural-language answer, which is returned to the app. The current LLM provider used is Anthropic’s API with prompt caching on the system prompt.

3.4.4. Sustainability & Carbon Footprint Calculation

3.4.4.1. Problem Formulation

This module calculates the carbon footprint of agricultural activities in the farm. The goal is to estimate how much greenhouse gas is produced during operations such as irrigation, energy use, fuel consumption and fertilizer application.

The problem is defined as converting activity data into carbon emission values. Each activity produces a certain amount of emissions based on its usage and a corresponding emission factor.

The system takes activity inputs and calculates total carbon emissions using a simple rule-based formula stated in 4.4.3 Calculation Approach.

The emission factors and calculation method used in this system are obtained from standardized sources such as IPCC and FAO, ensuring consistency and reliability in the calculations. This approach is transparent and does not rely on machine learning models.

3.4.4.2. Data Acquisition & Input Representation

The carbon footprint calculation module uses data collected from multiple sources within the system.

The main inputs include:

- Energy consumption data
- Fuel usage data
- Fertilizer application

These inputs are obtained from user inputs. For example, energy or water consumption can be recorded by the user when a bill is received.

In addition to activity data, the system uses emission factors obtained from standardized sources such as IPCC and FAO. These values are stored within the system and matched with the corresponding activity type.

All inputs are converted into a consistent format before calculation. This includes unit standardization and simple validation to ensure values are within reasonable ranges.

This structure allows the system to combine data from different modules and represent them in a unified way for carbon footprint calculation.

3.4.4.3. Calculation Approach

The carbon footprint is calculated using a simple rule-based approach based on activity data and predefined emission factors. For each activity (such as energy use, fuel consumption or fertilizer application), the system calculates emissions using the following formula:

Carbon Emissions = Activity Amount × Emission Factor

Each activity is processed separately, and the total carbon footprint is obtained by summing all individual emissions. The calculation process follows these steps:

- Activity data is collected from system inputs (user input or logs)
- The corresponding emission factor is selected based on the activity type
- Emissions are calculated for each activity
- All emissions are summed to obtain the total carbon footprint

The emission factors are predefined and stored in the system based on standardized sources such as IPCC and FAO.

3.4.4.4. Output Metrics & Interpretation

The system presents carbon footprint results in a clear and simple way. The main output is the total carbon emissions (kg CO_2-equivalent), representing the overall environmental impact.

In addition, emissions are shown as a breakdown by activity type (such as, energy, fuel, fertilizer), helping users see which sources contribute the most.

Emissions can also be displayed in normalized forms, such as:

- per area
- per production amount (if available)

This allows users to better understand and compare their environmental impact.

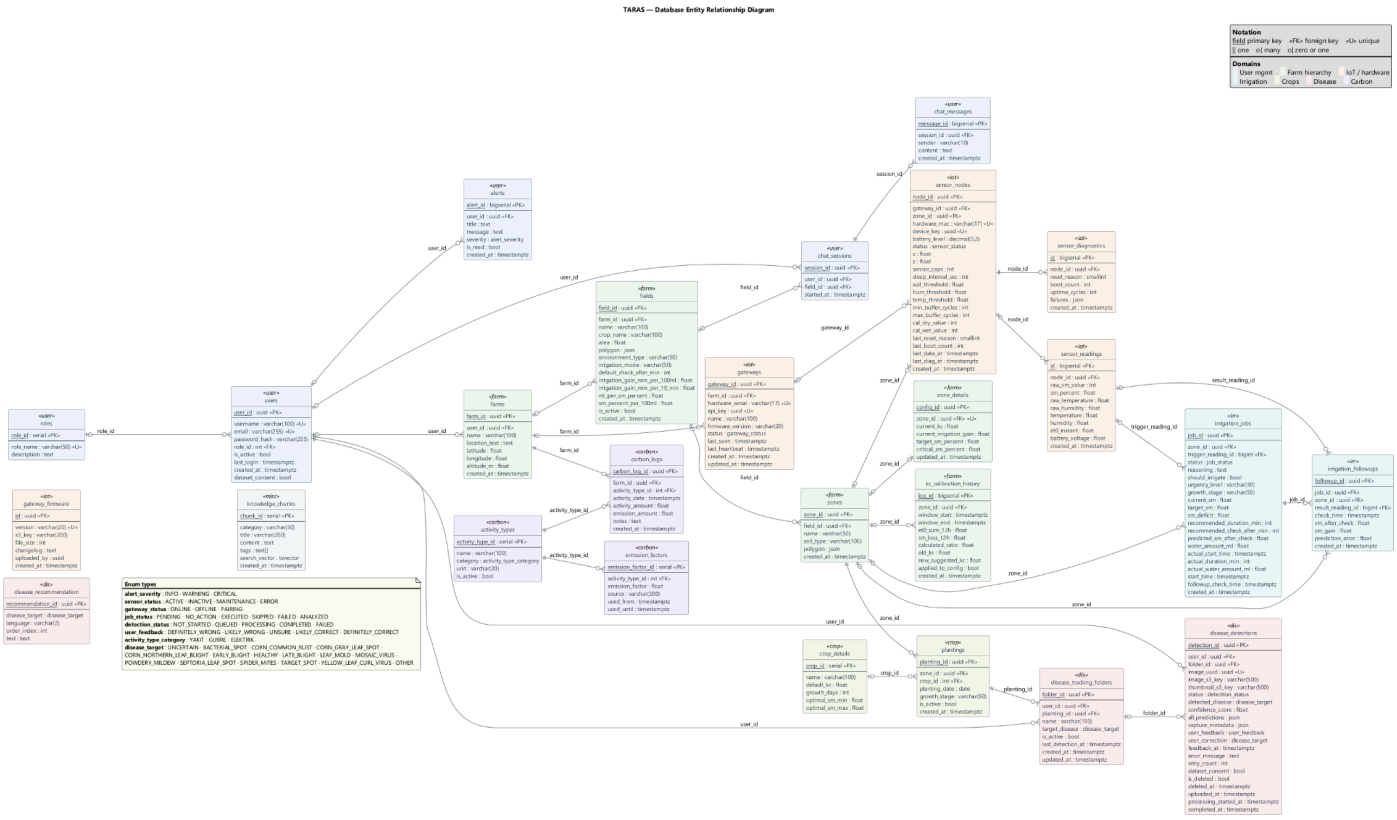
3.4.4.5. Implementation Details

The carbon footprint calculation is implemented as part of the TARAS backend within the analytics layer. The system uses activity data along with predefined emission factors obtained from sources such as IPCC and FAO. Before calculation, input values are standardized into common units and checked for basic validity

Carbon emissions are calculated using a rule-based formula for each activity, and the results are aggregated to obtain the total footprint. The calculated outputs are then stored and sent to the user interface through backend services, where they are displayed together with other system outputs.

3.5. Data Design

3.5.1. Entity-Relationship Diagram



3.5.2. Data Pipeline

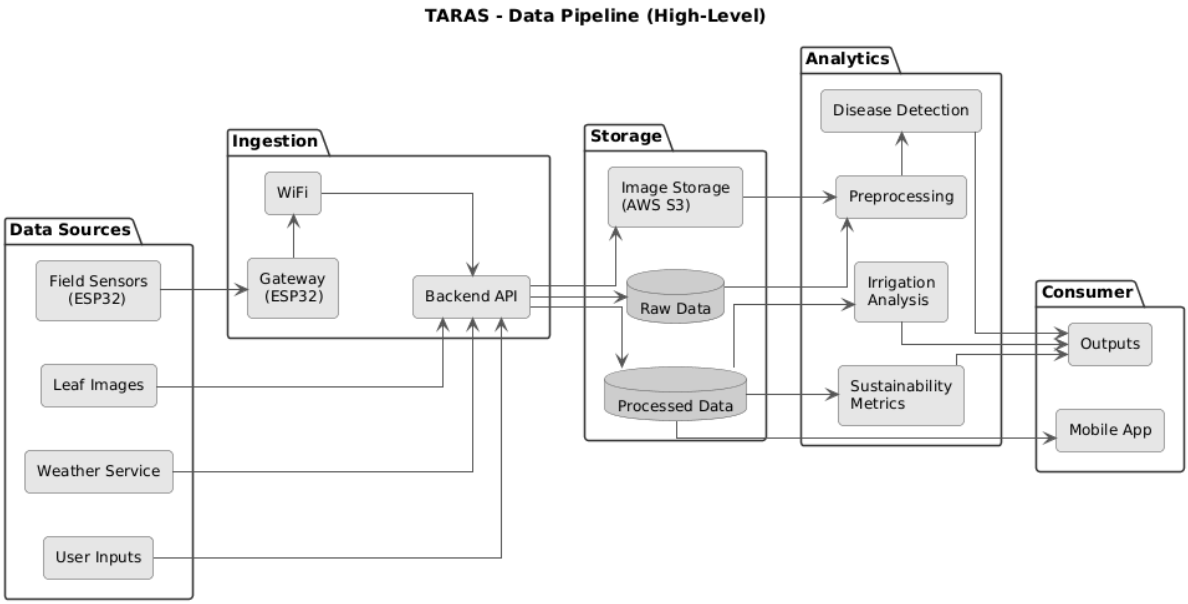
The TARAS data pipeline defines how heterogeneous data collected from field sensors, user inputs, and image uploads are transformed into reliable and analysis-ready information. The pipeline is designed to support real-time monitoring, historical analysis, and decision-support operations while maintaining data consistency and traceability.

Field sensor measurements such as soil moisture and environmental conditions are either transmitted to gateway devices first. In parallel, crop metadata, irrigation parameters and plant images are received through backend interfaces. All incoming data are timestamped and associated with field and device identifiers to ensure coherent integration.

Before storage, the data undergo preprocessing steps including timestamp alignment, unit normalization, basic quality checks, and context-aware filtering to handle missing values and abnormal sensor readings. Processed

numerical data are stored in a centralized database, while image data are stored in dedicated storage buckets with linked metadata.

The processed data are then consumed by analytical modules responsible for irrigation estimation, disease detection, and sustainability analysis. Derived outputs such as irrigation recommendations, classification results and environmental indicators are stored as processed records and delivered to the mobile app through backend APIs. This structured pipeline enables consistent data flow across sensing, analytics and visualization layers and provides a scalable foundation for the TARAS platform.



3.6. User Interface Design

3.6.1. Interface Hierarchy

The interface follows a hierarchical structure that prioritizes critical agricultural information. Core metrics such as soil moisture, temperature, and field status are placed at the highest level and presented through large, clearly separated data cards. This allows users to access essential information with minimal navigation and reduces cognitive effort during decision-making.

The hierarchy is intentionally simple to support farmers with different technical backgrounds, especially older users with limited digital experience. By minimizing nested menus and visual clutter, the interface helps users quickly identify actionable insights without feeling overwhelmed.

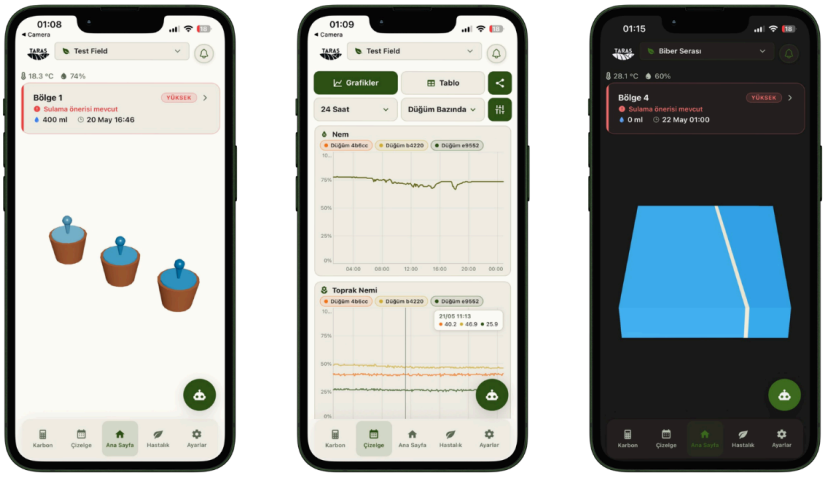
3.6.2. Visual Presentation

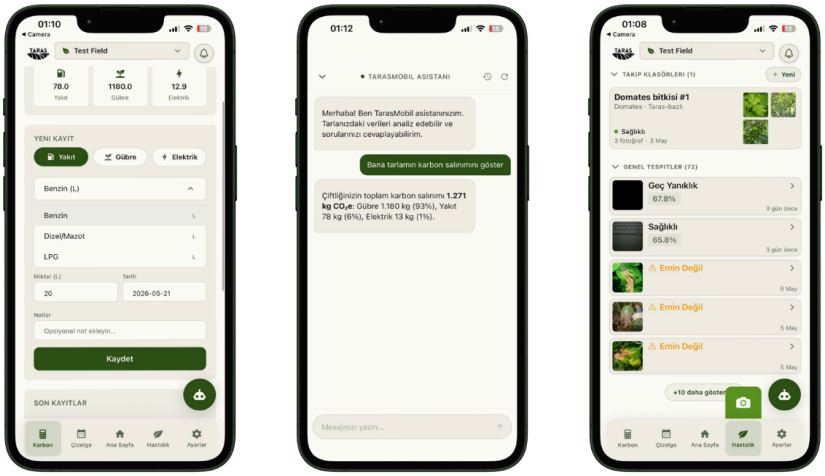
A balanced and minimalistic color palette was adopted for the visualization and interaction layer. The palette is kept simple so that digital twin visualizations remain visually distinct from the interface background. Light and dark modes assign inverse roles to background and text elements, preserving visual separation and comfort under different lighting conditions.

3.6.3. Accessibility and Responsivity

The interface design adheres to web accessibility and readability principles, including WCAG contrast recommendations, to support usability across varying lighting conditions and display settings. Text sizes, spacing, and contrast levels are specified to compensate for age-related declines in visual acuity and cognitive flexibility.

To further reduce cognitive load, essential information is structured to be presented in a concise and visually separated manner. In addition, an AI-powered conversational assistant is provided as a core interaction mechanism to support users with varying levels of technological literacy [46].





3.7. Conclusion

This document outlined the system architecture and methodology of the TARAS precision agriculture platform, focusing on its layered structure, modular subsystems and data-driven decision logic. By combining field sensing, agronomy-based irrigation analysis, AI-supported disease detection and user-centered interface design, TARAS provides a scalable and maintainable foundation for intelligent agricultural support. The proposed architecture enables future extensions while ensuring reliable and accessible system operation.

4. Implementation

4.1. Implementation

This chapter describes the engineering implementation of the TARAS platform, covering the hardware sensing infrastructure, communication protocols, cloud backend services, ML integration, and mobile application. The focus is on how each subsystem was built, connected, and deployed as a unified precision agriculture system.

4.1.1. System Overview

TARAS is an intelligent agriculture platform that integrates IoT-based sensing, digital-twin visualization, AI-driven irrigation recommendation and computer-vision-based plant disease detection into a single, cohesive system. The platform is designed to help farmers optimize water usage, detect crop diseases early and make data-driven decisions through an accessible mobile interface.

The system comprises six core subsystems:

IoT Sensor Nodes: Battery-powered ESP32-C6 microcontrollers equipped with capacitive soil-moisture sensors and SHT31 temperature/humidity sensors, deployed across the field to collect environmental data at regular intervals.

Gateway: A mains-powered ESP32 WROOM-32 device that aggregates sensor data from up to 20 nodes via ESP-NOW and forwards it to the cloud backend over Wi-Fi using HTTP POST and Socket.IO.

Cloud Backend: A Node.js/Express API server deployed on AWS EC2, backed by PostgreSQL, providing RESTful endpoints for telemetry ingestion, irrigation analysis, disease detection, LLM-based advisory, and real-time data streaming via Socket.IO.

Machine Learning & Computer Vision: A three-model deep-learning ensemble (one ConvNeXt-V2 and two Swin Transformers) for 14-class plant disease classification, served via ONNX Runtime on AWS Lambda, complemented by an on-device distilled EfficientNet-B0 (TFLite) for offline live scanning, alongside a rule-based irrigation recommendation engine running on the backend.

Irrigation Recommendation Module: A decision-support module that uses real-time soil moisture, temperature, humidity, and crop-specific thresholds to determine irrigation needs, recommend water amount or timing, and provide a reason for each suggestion. The module runs within the Node.js/Express backend, sharing the same compute and PostgreSQL database as the rest of the API.

Mobile Application: A React Native (with Expo) cross-platform application providing a digital-twin visualization, disease detection analysis, irrigation recommendations, an advisory AI chatbot, and carbon footprint estimation.

The end-to-end data flow operates as follows: sensor nodes collect soil moisture, temperature, and humidity readings at configurable intervals (default: every 5 minutes). Readings are buffered in RTC memory and transmitted in batches via ESP-NOW to the gateway. The gateway validates, formats, and POSTs the telemetry to the cloud API, where it is persisted in PostgreSQL and made available to the irrigation engine, digital twin, and mobile clients in real time via Socket.IO.

4.2. Hardware Implementation

4.2.1. Sensor Node Design

Each sensor node is built around the ESP32-C6 SuperMini module, a RISC-V-based microcontroller featuring integrated 802.11b/g/n/ax Wi-Fi and Bluetooth 5 LE. The C6 variant was selected for its superior power efficiency compared to the standard ESP32, enabling battery-powered operation measured in months rather than days. The CPU base clock is set to 40 MHz from the crystal oscillator (bypassing the PLL), which is sufficient for I²C communication at 100 kHz and ADC sampling. When ESP-NOW transmission is required, the clock is dynamically raised to 80 MHz (the minimum for WiFi radio operations) and returned to 40 MHz afterward, minimizing average power consumption.
The sensor node interfaces with two primary sensing elements:

SHT31 Temperature/Humidity Sensor: Connected via I²C (SDA: GPIO 4, SCL: GPIO 7, address 0x44). This digital sensor provides high-accuracy temperature (±0.3°C) and humidity (±2% RH) measurements. No soft reset is performed at boot; the SHT31's single-shot measurement mode operates correctly from any prior state, and omitting the reset saves approximately 40 ms per wake cycle.

Capacitive Soil Moisture Sensor: Connected to the ADC on GPIO 0. Unlike resistive sensors, capacitive probes do not corrode over time, making them suitable for long-term field deployment. Raw ADC readings are scaled and converted to soil moisture percentage using per-node calibration parameters (minimum and maximum raw soil moisture readings) stored in the backend.

The pin configuration of the ESP32-C6 SuperMini sensor node is summarized below:

GPIO	Function	Notes
4	SDA (I²C)	SHT31 temperature/humidity
7	SCL (I²C)	SHT31 temperature/humidity
0	ADC	Capacitive soil moisture

4.2.2. Sensor Data Acquisition

Sensor readings are acquired at a configurable sampling interval, defaulting to 300 seconds (5 minutes) in production. At each wake cycle, the node initializes the I²C bus, reads the SHT31 sensor for temperature and humidity, samples the ADC for soil moisture, and stores the result as a compact 4-byte *stored_reading_t* structure in RTC memory. Temperature is stored as an int16 multiplied by 100 (preserving two decimal places), humidity as a uint8 (the raw value divided by 50 for storage, then multiplied by 50 on decode, yielding ±0.5% RH precision), and soil moisture as a uint8 multiplied by 16. Sensor failure is indicated by sentinel values (0xFF for humidity and soil, -30000 for temperature).

The RTC buffer can hold up to 2,000 readings, equivalent to approximately 7 days of data at a 5-minute interval. When the buffer reaches the configured send threshold (based on minimum buffer cycles, maximum buffer cycles, or significant threshold changes in temperature, humidity, or soil moisture), readings are expanded to 7-byte *reading_entry_t* structures and transmitted to the gateway.

Calibration of soil moisture values is performed at the backend level. Each sensor node has its minimum and maximum raw ADC readings stored in the database. When a reading is received, the backend maps the raw value to a 0–100% moisture scale using linear interpolation between these calibration bounds.

4.2.3. Communication Layer

The communication architecture uses a two-hop design: sensor nodes communicate with the gateway via ESP-NOW (a connectionless, low-overhead protocol operating on the 2.4 GHz Wi-Fi band), and the gateway forwards data to the cloud over standard Wi-Fi.

ESP-NOW Protocol: All ESP-NOW packets are authenticated using HMAC-SHA256 signatures with per-node encryption keys established during the pairing process. The protocol supports multiple packet types including PAIR_BEACON (gateway discovery), PAIR_ACCEPT/PAIR_COMPLETE (credential exchange), SENSOR_DATA (batched readings), DATA_ACK (acknowledgments), and CONFIG_UPDATE (remote configuration changes). Each SENSOR_DATA packet can carry up to 33 readings (limited by the 250-byte ESP-NOW maximum payload). Both devices are forced to 802.11b/g/n mode to ensure cross-compatibility between the ESP32-C6 (which supports WiFi 6/802.11ax) and the ESP32 WROOM-32 gateway. To prevent replay attacks, the gateway rejects duplicate HMAC counters from each node (duplicate-only rejection rather than strict monotonic ordering, to tolerate counter resets after deep-sleep anomalies). Each sensor node persists its HMAC counter to NVS flash every 10 increments, ensuring counter continuity across reboots without excessive flash wear. For development and factory provisioning, the gateway also supports serial-based credential input as a fallback to BLE (activated by sending 'SERIAL' during the BLE advertising window).

Gateway Forwarding: The gateway is a mains-powered ESP32 WROOM-32 running at 80 MHz (sufficient since ESP-NOW is interrupt-driven and HTTP POST is WiFi-bound). Upon receiving and HMAC-verifying a sensor data packet, it sends a signed ACK back to the sensor node and constructs a JSON payload containing the readings. This payload is transmitted to the backend via HTTP POST to the */api/sensors/device/data* endpoint, authenticated with a device-key header (X-Device-Key). If the HTTP transmission fails (e.g., due to connectivity loss), the gateway buffers the data in a compact format on SPIFFS (up to 64 KB, approximately 8,877 readings or 6.2 days of offline storage for 5 nodes). Buffered records are flushed every 30 seconds when connectivity is restored. To minimize sensor node radio-on time, the gateway employs a deferred ACK pipeline: it sends a signed acknowledgment to the sensor node immediately upon frame validation, then queues the data internally and batches the HTTP POST to the backend after all pending packets from that wake cycle have been processed.

Real-Time Streaming: In addition to HTTP telemetry, the gateway maintains a persistent Socket.IO connection to the backend for real-time events including gateway authentication, heartbeat signals, node discovery notifications, and pairing completion events.

Diagnostics: The protocol defines a PKT_DIAGNOSTIC (0x12) packet type that sensor nodes transmit periodically alongside data packets. Diagnostic payloads

use an enum-based, variable-length format covering 10 failure categories: send failures, ACK misses, sensor read failures, soil probe failures, NVS errors, WiFi failures, I²C bus errors, channel recovery events, buffer overflows, and boot/reset reason codes. The gateway forwards these to the backend, where they are stored in the sensor_diagnostics table for fleet health monitoring.

4.2.4. Power Management

Battery life is a critical design parameter for field-deployed sensor nodes. The system employs several power optimization strategies:

Deep Sleep: In production mode, the sensor node enters deep sleep between sampling cycles, drawing microamp-level current. The wake cycle (sensor read, optional data transmission) takes approximately 34 ms for non-sending cycles and 195 ms for sending cycles, resulting in a duty cycle of approximately 0.01%.

TX Power: Transmission power is set to 20 dBm (API value 78), the maximum for the ESP32-C6, to ensure reliable through-wall range across the field. While higher TX power increases instantaneous current draw during the brief transmission window, the deep-sleep duty cycle ensures that radio-on time represents a negligible fraction of total energy consumption.

Dynamic CPU Frequency: The base clock is set to 40 MHz from the crystal oscillator (no PLL), approximately 60% below the default 160 MHz. The CPU dynamically switches to 80 MHz only during WiFi/ESP-NOW radio operations and returns to 40 MHz immediately afterward, keeping average power consumption well below what a fixed 80 MHz clock would draw.

With these optimizations, a sensor node powered by a 1000 mAh Li-Ion battery can operate for months in normal production mode at a 5-minute sampling interval.

4.3. Backend & Data Pipeline

The TARAS backend is implemented as a TypeScript/Express server running on AWS EC2, with PostgreSQL hosted on AWS RDS as the primary data store. The server exposes RESTful API endpoints and maintains WebSocket connections via Socket.IO for real-time data delivery to mobile clients.

4.3.1. Database Design

The data model is managed through Prisma ORM with a hierarchical structure: **User → Farm → Field → Zone → SensorNode → SensorReading**. Each Zone contains a ZoneDetail record holding irrigation parameters such as crop coefficient (Kc), irrigation gain (mm/min), and target soil moisture values. The IrrigationJob model tracks recommendation history with statuses including PENDING, EXECUTED, SKIPPED, and ANALYZED.

Key entities in the data model include: sensor_nodes (device registration, calibration bounds, MAC address, status), sensor_readings (timestamped

telemetry with soil moisture, temperature, humidity, and error bitmask), irrigation_jobs (recommendation output, predicted vs. actual soil moisture for calibration), disease_results (image URL, classification label, confidence), and chat_sessions/chat_messages (LLM advisory conversation history).

4.3.2. Data Processing Pipeline

Incoming telemetry follows a structured processing pipeline. When the backend receives a POST from the gateway, it validates the device key against registered sensor nodes, checks the data schema (rejecting records with missing or out-of-range values), maps raw ADC soil moisture readings to calibrated percentage values using the per-node min/max bounds, and persists the validated readings as SensorReading records via Prisma's createMany batch insert. The system also inspects the sensor error bitmask in each reading and creates rate-limited Alert records (with severity levels INFO, WARNING, or CRITICAL) when sensor faults are detected.

After successful persistence, the backend emits a sensor-update event via Socket.IO to all clients subscribed to the relevant field room, enabling real-time dashboard updates in the mobile application.

4.3.3. Irrigation Recommendation Engine

The irrigation recommendation system is designed as a rule-based decision model combined with a physically inspired water-demand estimation and an adaptive calibration layer. This architecture was chosen over a ML approach to ensure interpretability, controllability, and stable behavior under real-world constraints where training data is limited.

The engine operates with a clear separation of responsibilities: fixed thresholds determine when to irrigate, ET_0-based estimation determines how much water is needed, and a calibration layer corrects how much water is actually applied.

Decision Logic (When to Irrigate)

The decision of whether irrigation is required is determined using fixed soil-moisture thresholds defined per crop growth stage (seedling, vegetative, flowering/fruiting, ripening). Each stage specifies a recommended minimum/maximum moisture range, critical minimum/maximum bounds, and a target soil moisture value. For example, the flowering and fruiting stages require higher minimum moisture (65%) reflecting the crop's increased water sensitivity, while the vegetative stage tolerates a wider range (50–65%). These thresholds are intentionally kept fixed as design parameters to maintain system stability and interpretability.

Water Demand Estimation (How Much to Irrigate)

Once irrigation is triggered, the required water amount is estimated using a simplified FAO-inspired approach. Crop evapotranspiration is computed as ETc =

ET_0 × Kc, where ET_0 is the reference evapotranspiration and Kc is a stage-dependent crop coefficient (e.g., 0.60 for seedling, 1.15 for flowering/fruiting). The total required irrigation combines the evapotranspiration-based demand with a soil-moisture deficit adjustment: the deficit (target_sm − current_sm) is multiplied by an initial heuristic coefficient of 0.5 to convert percentage-point deficit to mm of estimated water demand. If the irrigation gain (mm/min) is available for the field, duration is computed directly; otherwise, the system falls back to base duration estimates per irrigation mode.

Calibration Layer

The calibration layer is applied only to irrigation amount and duration, not to thresholds. It compensates for sensor inaccuracies, soil variability, irrigation system efficiency, and environmental differences. The calibration factor is computed from the running average of prediction errors: after each recommendation, the system compares the predicted soil moisture one hour post-irrigation with the actual observed value. Once at least 10 recommendations have been executed, the calibration factor is computed as 1 + (average_prediction_error / 100), increasing water application when the system consistently underpredicts consumption and decreasing it otherwise. Below 10 recommendations, all calibration factors default to 1.0.

Urgency and Timing

The engine assigns urgency levels based on proximity to critical thresholds: HIGH when soil moisture is within 5 points of the critical minimum, MEDIUM when within 5 points of the recommended minimum, and LOW otherwise. High-urgency recommendations trigger immediate irrigation; medium-urgency recommendations schedule irrigation within preferred agricultural time windows (05:00–09:00 or 18:00–23:00); low-urgency conditions produce no recommendation. ETc-based atmospheric demand can elevate urgency from medium to high when current evapotranspiration exceeds 1.2 times the rolling 7-day mean.

4.4. Computer Vision Integration

The disease detection module is deployed as a containerized AWS Lambda function, separate from the main backend to enable independent scaling and avoid impacting API latency. The inference pipeline operates as follows:

When a user uploads a leaf image through the mobile application, the image is sent to the backend API. The backend stores the original image in AWS S3, then invokes the Lambda function with an S3 reference payload ({s3_bucket, s3_key}), avoiding the 6 MB Lambda payload size limit. The Lambda function loads the three-model ONNX ensemble (one ConvNeXt-V2 and two Swin Transformers) via ONNX Runtime, preprocesses the image (resize to 224×224 or 384×384 pixels), runs inference, and returns a classification result containing the predicted disease label, confidence score, and probability distribution across all 14 classes.

The 14 target classes span tomato, corn, and other crops: Bacterial Spot, Corn Common Rust, Corn Gray Leaf Spot, Corn Northern Leaf Blight, Early Blight, Late Blight, Leaf Mold, Mosaic Virus, Powdery Mildew, Septoria Leaf Spot, Spider Mites, Target Spot, Yellow Leaf Curl Virus, and Healthy.

The Lambda function runs on arm64 (AWS Graviton2) with 1,769 MB of memory and a 180-second timeout. Cold starts take approximately 14 seconds (loading the three ensemble models), while warm invocations complete in approximately 6 seconds. The backend stores the classification result, confidence, and image URL in the disease_results table and returns the result to the mobile client. The Lambda function is invoked synchronously (InvocationType: RequestResponse) but called in a non-blocking pattern from the API handler, so the client polls for completion rather than waiting on the HTTP request.

4.5. LLM-Based Advisory System

The TARAS advisory module integrates Large Language Models through standard REST APIs, avoiding dependency on vendor-specific SDKs. The system adopts a hybrid architecture that combines agentic AI with a Retrieval-Augmented Generation (RAG) pipeline to ensure accurate and context-aware recommendations.

When a farmer submits a query, the AI agent orchestrates multiple tools to gather relevant information. Real-time zone telemetry, such as soil moisture levels and critical thresholds, is retrieved from the PostgreSQL database via dedicated data access tools. In parallel, a RAG pipeline retrieves agronomic knowledge (e.g., irrigation strategies, crop-specific guidelines) from a curated knowledge base using PostgreSQL full-text search.

The retrieved documents are then used by the model during response generation, ensuring that outputs are grounded both in live field data and validated domain knowledge. This significantly reduces hallucinations while improving the reliability of recommendations.

To handle provider-specific API requirements, a dedicated adapter service formats requests into the exact JSON schema expected by the selected AI endpoint. This decoupled design allows the system to switch or upgrade the underlying model without modifying the core backend, agent logic, or retrieval pipeline.

4.6. Carbon Footprint Calculation

The carbon footprint module is implemented as a backend-driven calculation service that estimates farm-level operational emissions based on user-provided activity data. The system considers three primary activity categories: electricity consumption, fuel usage (diesel, LPG, gasoline), and nitrogen fertilizer application.

The calculation follows a factor-based approach defined as:

Carbon Emissions = Activity Amount × Emission Factor

Emission factors are predefined in the system based on standardized sources such as IPCC and FAO. Each activity type is mapped to a fixed coefficient, ensuring consistency and transparency in the calculation process.

The module is built on a log-based data structure where each user input is stored as an activity record. Each carbon_log entry includes the associated field, activity type, activity date, activity amount, and the computed emission value. Activity types are defined separately to standardize categories such as electricity, fuel, and fertilizer, while emission factors are stored independently and linked to activity types with validity ranges, allowing future updates without affecting historical calculations.

During computation, the emission value is calculated at the moment each activity is recorded. When a new activity log is created, the system applies the corresponding emission factor to the activity amount and stores the result as emission_amount within the carbon_log record. In addition, the system updates the cumulative carbon value stored at the farm level by incrementing the total_CO_2e field in the farm table. This value represents the total emissions generated by the farm over time and is maintained independently from filtered or date-based queries used in the user interface.

When filters such as date range or activity type are applied, the system does not modify stored values. Instead, it performs a read-time aggregation by querying the relevant carbon_log entries that match the selected filters. The filtered logs are retrieved from the database, and their stored emission_amount values are summed to generate the filtered result. This allows the user to view emissions for a specific period, activity type, or tag without affecting the cumulative total_CO2 value maintained at the farm level.

4.7. Mobile Application

The TARAS mobile application is built with React Native using the Expo framework, supporting both Android and iOS from a single codebase. The application communicates with the backend via RESTful APIs (authenticated with JWT tokens stored in AsyncStorage) and receives real-time sensor updates through Socket.IO WebSocket connections.

The primary screens and features include:

Digital Twin Dashboard: The central screen renders an interactive 3D visualization of the agricultural field using React-Three/Fiber and custom GLSL shaders. Real-time sensor data (soil moisture and temperature) is interpolated across field zones and displayed as a color-coded heatmap. Users can rotate the perspective by dragging and tap specific zones to view detailed readings. Zones where values fall outside optimal ranges are visually highlighted to alert the farmer.

Disease Detection: Users can capture a leaf image using the device camera or upload from the gallery. The image is sent to the backend, which invokes the Lambda-based ensemble for classification. The result (disease type, confidence) is displayed on screen along with treatment recommendations generated by the system.

Irrigation Recommendations: The timetable screen presents a schedule of irrigation tasks based on the rule-based engine's output, showing recommended timing, duration, urgency level and reasoning.

AI Advisory Chatbot: A floating button accessible from all primary screens opens the conversational AI interface, where farmers can ask questions about their field conditions and receive context-aware guidance from the LLM.

Carbon Footprint Calculator: Farmers input energy, fuel, and fertilizer usage data through a structured form. The backend computes estimated CO_2e emissions using predefined emission factors and returns a breakdown for visualization.

The application supports light/dark/system theme modes, Turkish/English localization (all UI strings are managed through a centralized strings.ts file), and multi-user role-based access (farmer and stakeholder roles with different visibility permissions).

4.8. System Integration

All subsystems are integrated through the cloud backend, which serves as the central hub for data flow, service orchestration, and client communication. The end-to-end pipeline operates as follows:

Sensor data is collected by ESP32-C6 nodes, transmitted via ESP-NOW to the gateway, forwarded via HTTP POST to the backend API, validated and persisted in PostgreSQL, and streamed to mobile clients via Socket.IO. The irrigation engine periodically retrieves the latest sensor time series, evaluates rule-based thresholds, computes ETc-based water demand, applies calibration corrections, and stores the recommendation as an IrrigationJob. Disease detection operates as an event-driven pipeline: image upload triggers Lambda invocation, inference results are stored and returned to the client. The LLM advisory system pulls the latest sensor data, irrigation recommendations, and disease results to construct contextual prompts. The digital twin visualization on the mobile app consumes real-time sensor streams and renders interpolated heatmaps over the field geometry.

5. Testing & Validation

5.1. Purpose of Testing and Validation

This document explains the tests we ran on the TARAS hardware and what the results showed. Read alongside the SRS, it shows which hardware requirements were tested, how, and whether they passed.

We organised the testing around three goals. First, every hardware requirement in the SRS should connect to at least one test with a recorded result, so nothing slips through uncovered. Sections 5 and 10 are where we make that mapping explicit. Second, we wanted to catch bugs at the level where they're cheapest to fix, which is why the tests are split across component-level, communication, integration, reliability, and field categories rather than lumped together. Third, we wanted at least some of our

evidence to come from conditions close to real use, because a bench is not a field and hardware that looks fine indoors can fail the moment it's outside.

Not every requirement can be fully verified within a thesis timeline. For example claims about long-term battery life and multi-week reliability are extrapolated from shorter measurements rather than being observed directly due to time constraints. Section 16 sets out these boundaries and section 17 describes the follow-up testing that would close them.

5.2. Scope of Testing

- **Sensor node:** ESP32-C6 SuperMini, SHT31 temperature/humidity on I²C, capacitive soil probe on ADC, power path, and the firmware running on it.
- **Gateway:** ESP32 WROOM-32, mains-powered. Collects packets from sensors, verifies them, buffers when offline, forwards to the backend over HTTPS and Socket.IO. BLE provisioning is also tested here.
- **Power:** 3.7 V Li-Po supply, regulation to 3.3 V, behaviour under low voltage, measured draw per mode.
- **Communication:** ESP-NOW between sensor and gateway (HMAC-signed, replay-protected), HTTPS and Socket.IO between gateway and backend.
- **Enclosure:** build quality, moisture sealing, cable strain relief.
- **End-to-end integration:** sensor reading through to the mobile app.

5.3. Test Strategy

Tests are tied to requirements. Every hardware requirement maps to at least one test, and every test carries a status of PASS, FAIL, PARTIAL, PENDING, or N/A. Requirements without a test are tracked as gaps, not ignored.

Tests are grouped by level because different bugs surface at different levels:

- Unit tests (Section 7) exercise one component at a time.
- Communication tests (Section 8) focus on the radio and network links.
- Integration tests (Section 9) combine components.
- Reliability tests (Section 11) simulate disruptions such as brief outages, sensor faults, and power cuts.
- Field tests (Section 12) step outside the lab.

Where a test produces a number such as current draw, RSSI, or timing, we record the number. A PASS with no supporting measurement is weak evidence; a measured value with a stated method is stronger. Binary checks are recorded as PASS/FAIL without further detail.

5.4. Test Environment and Setup

Testing happened in two places: a lab bench for controlled work and a small pot setup that stood in for a field.

5.4.1. Lab Bench

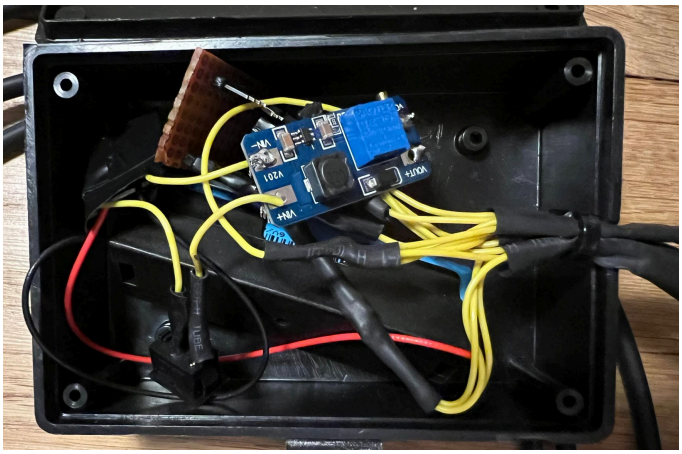
Sensor MCU	ESP32-C6 SuperMini, the same variant we plan to deploy.
Gateway MCU	ESP32 WROOM-32 on a dev board, USB-powered during tests.
Sensors	SHT31 on I²C; capacitive soil probe on ADC. A three-probe setup for multi-probe calibration.
Power	USB bus for dev; 3.7 V Li-Po for battery-mode tests. LDO regulator to the 3.3 V rail.
Instruments	Digital multimeter, USB current meter for sustained draw, oscilloscope for transients and I²C integrity.
Serial	Arduino IDE serial monitor at 115200 baud
Backend	Production backend over HTTPS, so request shapes, auth, and database writes in tests are the same as in deployment.

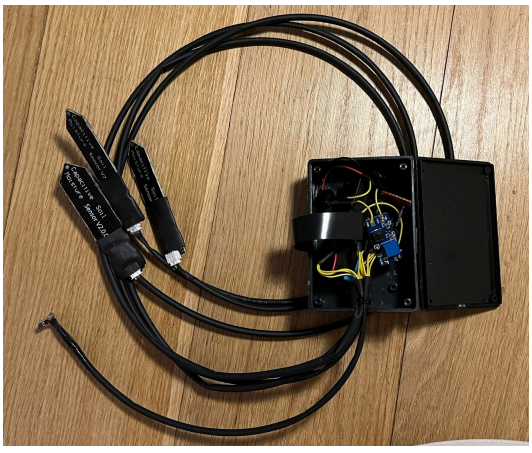
5.4.2. Pot Setup

For anything that depends on real soil or humidity, we used a three-pot setup. Each pot has its own soil probe, independently wetted, which lets us exercise per-sensor calibration with three different dry/wet baselines at once. The gateway sits on the bench at a distance roughly matching field use, connected via Wi-Fi to the same production backend used in deployment.

5.4.3. Firmware State

Tests reported here ran against the current production firmware for both sensor and gateway, which is the HTTPS-capable build with TLS validation against a pinned Let's Encrypt root CA. The three-probe calibration variant of the sensor firmware is what we used for the multi-probe work.





5.5. Requirement-to-Test Traceability

The table below maps every hardware requirement to its test IDs and current status. PASS means the test ran and passed. PARTIAL means the main path works but at least one sub-case is still outstanding. PENDING means the test is defined but not yet run, usually because it is long-running or needs extra setup.

Req ID	Requirement	Component	Test IDs	Expected	Status
HW-01	Microcontroller–sensor interface: 3.0-3.6 V rail, I²C and ADC sampled periodically.	Sensor node	T-7.1, T-7.2	Readings valid, no bus errors.	PASS
HW-02	Sensor telemetry reaches the cloud reliably.	Sensor - Gateway - Cloud	T-8.1, T-8.2, T-9.1	DB row appears within one send cycle.	PASS
HW-03	Power supply stable, deep sleep works, brownout survivable.	Power / battery	T-7.3a–f	Rail stable; sleep current within design envelope.	PARTIAL
HW-04	Enclosure seals against moisture and dust outdoors.	Enclosure	T-12.2	No ingress after exposure.	PENDING
HW-05	Gateway Wi-Fi uplink to backend over HTTPS.	Gateway	T-8.2	HTTP 200 within retry window; TLS chain validated.	PASS
HW-06	Gateway buffers data when connectivity is lost.	Gateway	T-8.2f, T-9.2	Buffered readings replay on reconnect.	PASS
HW-07	HMAC signing and replay protection on ESP-NOW link.	Sensor / Gateway	T-8.1a/b T-8.1g/h	Valid packets accepted; forged/replayed rejected.	PARTIAL
HW-08	Gateway heartbeat exposes health and firmware version.	Gateway	T-8.2e	Heartbeat every 60 s, correct payload.	PASS
HW-09	BLE provisioning onboards a new gateway.	Gateway	T-9	Config saved; device boots into normal mode.	PASS
HW-10	OTA firmware update works and fails safely.	Gateway	T-9.7	Update succeeds or fails cleanly.	PARTIAL
HW-11	Sensor faults detected; pipeline continues.	Sensor node	T-7.1c/d, T-7.1g	Sentinel reported; non-faulty path continues.	PASS

Req ID	Requirement	Component	Test IDs	Expected	Status
HW-12	Batch telemetry - multiple readings per packet.	Sensor node	T-8.1c/d	Up to 33 readings per packet; multi-packet flush works.	PASS
HW-13	Pairing flow - approve and decline through mobile app.	Gateway / Mobile	T-9.3, T-9.4	Both paths terminate correctly.	PASS
HW-14	Send backoff after sustained failure; channel recovery.	Sensor node	T-8.1e/f	Frequency reduced after 6 failures; channel scan every 6th.	PASS
HW-15	Multi-day stability.	System	T-11.2	72 h clean run, no leaks, no missed data.	PENDING
HW-16	Multi-sensor on a single gateway.	System	T-9.5	Interleaved packets all received.	PENDING
HW-17	Diagnostic telemetry reaches the backend.	Sensor / Gateway / Backend	T-9.8	Diagnostic row with failure counters.	PASS

5.6. Test Case Design Method

Every test case in sections 7 to 12 is written with the same five fields:

- A unique ID (T-"*section*"."*number*")
- A short description of what's tested
- The method used
- The observed result
- And the status (PASS, FAIL, PARTIAL, PENDING, or N/A).

5.7. Unit-Level Hardware Tests

Unit tests exercise one component at a time under controlled conditions. Catching bugs here is cheaper than catching them at the integration level.

5.7.1. Sensor Tests

We checked that raw measurements arrive at the microcontroller consistently. For the SHT31 we focused on I²C integrity and the failure-sentinel path (what happens when the sensor is unplugged). For the soil probe we focused on calibration stability and sentinel handling.

Test ID	What is tested	Method	Observed result	Status
T-7.1a	SHT31 I²C presence	Run firmware; check I²C 0x44 ACK on init.	ACK on both warm wakes and cold boot.	PASS
T-7.1b	SHT31 reading validity	Take 20 readings at ~25 °C / ~40% RH, compare against reference.	All within ±0.5 °C and ±3% RH of reference.	PASS
T-7.1c	SHT31 failure sentinel	Disconnect SHT31 mid-run.	Firmware reports sentinel value; threshold send is not triggered.	PASS
T-7.1d	SHT31 recovery	Reconnect after fault.	Normal readings resume; no stale sentinel.	PASS
T-7.1e	Soil probe: dry	Record raw ADC in fully dry soil for 60 s, per probe.	Three probes' dry values all in the expected 2700–2800 range; stable to within tens of counts.	PASS
T-7.1f	Soil probe: wet	Record raw ADC in fully wet soil for 60 s, per probe.	Three probes' wet values all in the expected 900–980 range; stable.	PASS
T-7.1g	Soil probe disconnect	Pull soil probe lead mid-run.	Firmware reports 0xFFFF sentinel; threshold path skipped.	PASS

Consistent dry and wet baselines across probes is what lets each probe map its own raw range to a common 0–100% output, so calibration numbers are comparable between sensors.

5.7.2. Microcontroller Tests

The ESP32-C6 runs the sensor firmware. Boot behaviour matters a lot on a deep-sleep device. If a warm wake fails to complete, it drains the battery instead of crashing visibly.

Test ID	What is tested	Method	Observed result	Status
T-7.2a	Cold boot	Power-cycle; watch serial log.	Reaches normal or pairing mode within expected boot time.	PASS
T-7.2b	Warm wake	Let device deep-sleep one cycle.	RTC state preserved; cycle count increments; no re-init of stable state.	PASS
T-7.2c	Serial log sanity	Run with debug enabled.	Log lines match expected [PREFIX] action: data format.	PASS

Test ID	What is tested	Method	Observed result	Status
T-7.2d	Batch send loop	Fill buffer; trigger send.	All readings sent across multiple packets; buffer drains cleanly.	PASS

Test T-7.2c confirms the serial log format is stable across builds, which the diagnostic subsystem in section 14 later depends on.

5.7.3. Power and Battery Tests

Test ID	What is tested	Method	Observed result	Status
T-7.3a	3.3 V rail stability	Measure under no-load, active-read, TX load.	Within 3.0–3.6 V in all modes.	PASS
T-7.3b	Active-mode current (read)	Measure during I²C + ADC sample.	~30 mA for ~34 ms, matching design.	PARTIAL
T-7.3c	Active-mode current (TX)	Measure during ESP-NOW send.	~150 mA peak at 20 dBm, matching design.	PARTIAL
T-7.3d	Light-sleep current	Measure during 4.5 s light-sleep windows in pairing mode.	~1 mA sustained.	PASS
T-7.3e	Deep-sleep current	Measure across 5-min deep-sleep windows.	~10 µA sustained.	PASS
T-7.3f	Brownout handling	Gradually reduce supply.	Safe low-power entry; orderly restart on recovery.	PENDING

Power tests confirm the rail is stable across operating modes and that the current draw in each mode matches what the design assumes. We used a digital multimeter for bench currents, and for deep-sleep characterisation the device goes through a shunt resistor with the voltage across it captured on an oscilloscope.Communication and Connectivity Tests

Two links have to work for telemetry to reach the cloud: sensor to gateway over ESP-NOW, and gateway to backend over HTTPS and Socket.IO. They fail in different ways, so we tested them separately before combining them in section 9.

5.7.4. Sensor to Gateway

Test ID	What is tested	Method	Observed result	Status
T-8.1a	HMAC happy path	Pair, send a normal packet.	Gateway verifies HMAC and forwards.	PASS

Test ID	What is tested	Method	Observed result	Status
T-8.1b	Replay counter	Watch counter across cycles.	Increments monotonically; duplicates rejected.	PASS
T-8.1c	33-reading batch	Fill buffer, trigger send.	One packet carries 33 readings; gateway accepts all.	PASS
T-8.1d	Multi-packet flush	Buffer beyond one-packet capacity.	Multiple packets sent until buffer drains.	PASS
T-8.1e	Send backoff	Block the gateway; observe sensor after repeated failures.	After 6 consecutive failures, send rate drops.	PASS
T-8.1f	Channel recovery	Change gateway channel mid-run.	Sensor scans channels every 6th failure and recovers.	PASS
T-8.1g	Forged packet rejected	Inject packet with invalid HMAC.	Gateway logs failure; packet discarded.	PENDING
T-8.1h	Replay attack rejected	Replay a captured valid packet.	Gateway detects replay; packet discarded.	PENDING

The sensor-to-gateway link uses ESP-NOW with HMAC-SHA256 signed packets. DATA_ACK from the gateway is the success signal. A packet that is sent but not ACKed stays in the sensor's buffer for the next cycle.

5.7.5. Gateway to Backend

Test ID	What is tested	Method	Observed result	Status
T-8.2a	HTTPS POST	Forward a telemetry batch.	HTTP 200; readings in DB.	PASS
T-8.2b	TLS chain validation	Run HTTPS handshake against backend.	Cert validates against pinned root; no insecure fallback.	PASS
T-8.2c	Diagnostic POST	Trigger a diagnostic packet; gateway forwards it.	Diagnostic row in DB with correct reset reason and counters.	PASS
T-8.2d	Socket.IO auth	Observe gateway:auth on connect.	Server responds with auth_result; gateway joins its room.	PASS

Test ID	What is tested	Method	Observed result	Status
T-8.2e	Heartbeat payload	Watch for a minute.	Heartbeat carries gateway_id, uptime, free heap, RSSI, paired node count, buffered unsent, firmware version.	PASS
T-8.2f	Retry under transient failure	Briefly interrupt backend.	Gateway buffers to local storage and replays after recovery.	PASS
T-8.2g	Backend restart tolerance	Restart backend while gateway is running.	Socket.IO reconnects; no data loss.	PENDING

The gateway talks to the backend two ways: HTTPS POST for telemetry and diagnostic uploads, and Socket.IO for real-time control plane (heartbeat, pairing events, OTA triggers). Both terminate at the same backend host behind nginx with a Let's Encrypt cert, and the gateway validates the server certificate against a pinned root CA baked into the firmware.T-8.2b (TLS chain validation) matters a lot. An earlier firmware build bypassed certificate validation during development, and production firmware validates against a pinned root. A failure here would be a release-blocker, not a warning.

5.8. Integration Tests

Integration tests combine components that have already passed their unit tests and exercise workflows that only make sense across component boundaries. Two main flows are covered: a sensor reading making it all the way to the mobile dashboard, and a pairing cycle bringing a new sensor into a running gateway.

T-9.7 is one of the strongest PASS results in the document. We tested this live on our deployed gateway during a firmware upgrade: the gateway downloaded the new build, rebooted, reconnected, and the updated firmware version showed up in the database within about a minute. That is production evidence, not a bench demo.

Test ID	What is tested	Method	Observed result	Status
T-9.1	End-to-end telemetry	Trigger a sensor cycle; trace to the dashboard.	Reading in DB; mobile client receives via Socket.IO; dashboard updates.	PASS
T-9.2	Gateway power cycle	Cut gateway power mid-cycle; restore after one sensor cycle.	Sensor buffers during outage; buffered readings replay on return.	PASS
T-9.3	Pairing: approve path	Start pairing from mobile app; power unpaired sensor.	Beacon - accept - complete; node visible in app.	PASS

Test ID	What is tested	Method	Observed result	Status
T-9.4	Pairing: decline path	Start pairing; decline from app.	Rejection delivered; sensor restarts scan; gateway returns to normal mode.	PASS
T-9.5	Multiple sensors	Pair two sensors to one gateway; run for several cycles.	Both sensors' readings reach backend; interleaved packets handled correctly.	PENDING
T-9.6	Calibrated round-trip	Send raw ADC for a probe with dry/wet values configured.	Backend stores calibrated %; mobile shows %, not raw ADC.	PASS
T-9.7	OTA end-to-end	Upload new gateway firmware; trigger OTA.	Gateway downloads, flashes, reboots, reconnects, reports new version.	PASS
T-9.8	Diagnostic end-to-end	Force a sensor-side failure.	Diagnostic row reaches backend with correct failure type and count.	PASS

T-9.5 is PENDING because a single gateway was enough for development, and a two-sensor test has not yet been scheduled. It is listed in the future work in section 17.

5.9. Functional Validation Against SRS Requirements

The following table summarizes each hardware requirement in terms of expected behaviour, lists the tests, shows the outcomes and records the status.

Req ID	Requirement behaviour	Test IDs	Observed outcome	Status
HW-01	MCU reads SHT31 over I²C and soil probe over ADC within the 3.0–3.6 V envelope.	T-7.1, T-7.2, T-7.3a	Rails in range; I²C ACKs; soil ADC in expected bands.	PASS
HW-02	Sensor telemetry reliably reaches the cloud, with buffering during transient failure.	T-8.1, T-8.2, T-9.1, T-9.2	End-to-end path works; buffered readings replay after gateway reboot.	PASS
HW-03	Power: stable regulation, deep sleep works, brownout survivable.	T-7.3a–f	Rail stable; active currents match design; deep-sleep and brownout measurements pending.	PARTIAL
HW-04	Enclosure protects against outdoor dust/moisture.	T-12.2	Visual seal looks good; formal IP test not yet done.	PENDING

Req ID	Requirement behaviour	Test IDs	Observed outcome	Status
HW-05	Gateway uplink delivers telemetry and control traffic over HTTPS.	T-8.2	HTTP 200 and Socket.IO auth both verified.	PASS
HW-06	Gateway store-and-forward handles outages.	T-8.2f, T-9.2	Local buffering works; readings replay on reconnect.	PASS
HW-07	HMAC + replay protection on sensor–gateway link.	T-8.1a/b, T-8.1g/h	Happy path and counter checks pass; adversarial tests planned.	PARTIAL
HW-08	Gateway heartbeat surfaces health and diagnostics.	T-8.2e	Cadence and payload match expectation.	PASS
HW-09	BLE provisioning onboards a gateway.	T-9	Happy path verified.	PASS
HW-10	OTA is safe and complete.	T-9.7	Live upgrade succeeded on deployed hardware; fault-injection variants planned.	PARTIAL
HW-11	Sensor faults detected; pipeline continues.	T-7.1c/d, T-7.1g	Sentinels reported; pipeline unaffected; recovery clean.	PASS
HW-12	Batch telemetry and multi-packet flush.	T-8.1c/d	33-reading batches and multi-packet flush both confirmed.	PASS
HW-13	Pairing, approve and decline.	T-9.3, T-9.4	Both paths confirmed end-to-end with mobile client.	PASS
HW-14	Backoff and channel recovery.	T-8.1e/f	Rate drops after 6 failures; channel scan every 6th failure.	PASS
HW-15	Multi-day stability.	T-11.2	Scheduled but not executed yet.	PENDING
HW-16	Multi-sensor scalability.	T-9.5	Scheduled; single-sensor was enough for development.	PENDING
HW-17	Diagnostic telemetry reaches backend.	T-9.8	Diagnostic rows observed with correct reset reason and counters.	PASS

5.10. Reliability and Robustness Tests

Reliability tests check whether the system keeps working, or fails properly , when conditions change. The following cases cover the kinds of disruption a real deployment will see: brief power cuts, network drops, component faults, and long unattended runs.

Test ID	What is tested	Method	Observed result	Status
T-11.1	Sustained operation (several hours)	Leave the setup running unattended; monitor heap and data completeness.	No missed readings; heap stable.	PASS
T-11.2	Multi-day stability (72 h)	Run for 72 consecutive hours.	Scheduled, not yet executed.	PENDING
T-11.3	Brief Wi-Fi outage (<60 s)	Unplug router for under a minute.	Gateway drops and reconnects; no data loss; local buffer used if a sensor cycle lands during outage.	PASS
T-11.4	Extended Wi-Fi outage (>10 min)	Keep router off past one send cycle.	Sensor data buffered locally; replays on reconnect; buffer doesn't overflow within test window.	PASS
T-11.5	Gateway power cycle	Cut gateway power mid-sensor-cycle.	Sensor retains readings; gateway boots cleanly; buffered data reaches backend.	PASS
T-11.6	Intermittent sensor fault	Periodically disconnect and reconnect SHT31.	Sentinels raised during outages; normal telemetry resumes cleanly.	PASS
T-11.7	Brownout under low battery	Gradually reduce battery voltage.	Safe mode entered; orderly restart on recovery.	PENDING

The gateway power-cycle recovery path was hardened after earlier builds lost data during brief outages. Testing fed directly back into firmware changes, not just into a report.

5.11. Field Testing / Real-World Validation

Field testing here means running the hardware under conditions closer to real deployment than a bench allows. Section 16 describes the parts not yet covered.

Test ID	What is tested	Method	Observed result	Status
T-12.1	Three-pot indoor deployment	Run the three-probe setup indoors for a full day.	All three probes report consistent calibrated values; dashboard reflects gradients between pots.	PASS

Test ID	What is tested	Method	Observed result	Status
T-12.2	Enclosure ingress under damp conditions	Expose enclosure to a damp outdoor environment; open and inspect.	No water or dust ingress; connectors and PCB clean.	PENDING
T-12.3	Radio range check	Position sensor and gateway at realistic separation with obstructing structures.	Packets delivered and ACKed; no persistent channel-recovery trigger.	PARTIAL
T-12.4	Live firmware upgrade	Trigger OTA on deployed hardware; watch reconnect and resumed data flow.	Upgrade succeeded in a single pass; firmware version updated within about a minute; no data loss.	PASS
T-12.5	Outdoor multi-hour operation	Deploy in an outdoor or semi-outdoor setting for several hours.	Continuous telemetry; no unexpected restarts.	PENDING

5.12. Test Results and Observations

Across sections 7 through 12, we defined 52 test cases. Of these, 40 are PASS, 3 are PARTIAL, and 9 are PENDING. Nothing is in FAIL.

5.12.1. Quantitative Summary

Category	PASS	PARTIAL	PENDING
Unit tests (section 7)	14	2	1
Communication tests (section 8)	12	0	3
Integration tests (section 9)	7	0	1

Reliability tests (section 11)	5	0	2
Field tests (section 12)	2	1	2

5.12.2. Observations

Every pairing test passed on the first attempt once the firmware bugs in section 14 were resolved. Pairing has been steady since.

The OTA path was not left as a paper exercise. We ran it live on the deployed gateway during a firmware upgrade and watched the new version show up in the database about a minute later. That is stronger evidence than a bench-only result would be.

The diagnostic subsystem turned out to be more useful than expected. Its job is to surface sensor-side failure counters to the backend, but in practice it is how we know a fielded sensor is healthy. Several firmware improvements came directly from watching diagnostic traces on the three-pot setup.

Pending tests cluster in three places: formal power profiling (the PARTIAL and PENDING rows in 7.3), adversarial security cases (T-8.1g, T-8.1h), and the remaining long-duration or outdoor runs (T-11.7, T-12.2, T-12.5). The first two need a small amount of extra tooling. The third mostly needs time.

5.13. Issues Encountered During Testing and Corrective Actions

The table below lists the most consequential bugs we hit during hardware development, the root cause, what we did about it, and whether we re-verified afterwards.

#	Issue	Root cause	Fix	Verified
1	Gateway ACK timeouts for subsequent sensor packets during burst arrival.	HTTP POST in the sensor-data handler blocked the main loop, delaying ACK emission.	Deferred-ACK pipeline: ACK immediately, enqueue POST, flush afterwards.	PASS
2	ESP32-C6 sensor packets not received by ESP32 WROOM gateway.	ESP32-C6 defaulted to 802.11ax; ESP32 WROOM tops out at 802.11n.	Force protocol to 802.11b/g/n on the sensor before ESP-NOW init.	PASS
3	Second and subsequent sensor sends failed silently after pairing.	On ESP32-C6, setting Wi-Fi mode does not restart the radio if the mode is unchanged, so the radio	Explicitly restart the Wi-Fi stack after mode change in the	PASS

#	Issue	Root cause	Fix	Verified
		stayed off after a previous stop.	normal-mode send path.	
4	First packet received after pairing, all subsequent packets silently lost.	Double call to exit-registration-mode caused two ESP-NOW deinit/reinit cycles and corrupted the radio state.	Call exit-registration once, from the accept handler only.	PASS
5	Strict monotonic replay counter caused data gaps after reboots.	Counter could drift across reboots while HMAC was still fine.	Relax to duplicate-only rejection (same counter - drop, lower - accept with log). HMAC stays the real security gate.	PASS
6	Sensor sent data immediately when SHT31 was disconnected.	Threshold check triggered on the sentinel value (huge delta).	Skip threshold check when the reading is a sentinel.	PASS
7	Gateway emitted Socket.IO events before authentication; server silently dropped them.	Gateway waited for the WebSocket-connected flag rather than the authenticated flag.	Gate emission on authenticated, not connected.	PASS
8	TLS connections bypassed certificate validation during development.	Temporary insecure flag used before root CA was bundled.	Bundle Let's Encrypt root CA in firmware and validate on every HTTPS client.	PASS
9	Socket.IO over TLS on ESP32: the library's public SSL entry points used insecure mode.	The CA-validating SSL init is gated behind a flag ESP32 doesn't define.	Subclass the Socket.IO client to expose CA-validating SSL init on ESP32.	PASS

Most of these bugs were not in the obvious place. Several were second-order effects of earlier state (double radio deinit, stale auth flag), and the ESP32-C6 protocol mismatch showed up as pure silent failure with no error path at all. We put real effort into negative-path tests: tests designed to force failure modes and see whether the system surfaces, recovers, or swallows them. Most of the fixes above came out of tests like those.

5.14. Validation Summary

Across 17 hardware requirements: 11 PASS, 3 PARTIAL, and 3 PENDING. Nothing is in FAIL. The PARTIAL requirements are power/battery measurements, security negative-path testing, and OTA fault-injection coverage. The PENDING requirements are enclosure ingress testing, multi-day stability, and multi-sensor scalability.

Across the full test plan, 40 tests are executed and verified, and 9 remain defined but not yet run.

No requirement is failing. That does not mean nothing went wrong, as Section 14 shows, but every issue has either been resolved and re-verified or recorded as PARTIAL/PENDING with a planned next step. The remaining gap is additional validation work, not an unresolved defect. The deployed gateway has been running on the current firmware since the upgrade with no observed regressions, which provides additional evidence alongside the targeted tests.

5.15. Limitations of Current Testing

Not every dimension of the system has been tested to the depth it deserves. The main gaps are summarised below.

- **Duration:** The reliability evidence covers hours to days rather than weeks to months. A thesis timeline does not fit a season-long deployment, so claims about long-term battery life and multi-week reliability are extrapolated from shorter runs rather than measured directly. The extrapolation is defensible, but it is still an extrapolation.
- **Scale:** Tests ran on at most one gateway and a handful of sensors. A real farm might have multiple gateways with tens of sensors each. ESP-NOW should scale in principle, but interleaving packets from many nodes on one gateway has not been stress-tested, and the backend has not seen synthetic high-cardinality load. Both are needed before claims about "a farm" rather than "a test field".
- **Environment:** Outdoor testing has been limited to controlled damp exposure and bench work that approximates field use. No direct sun, no rain-and-temperature extremes across a full diurnal cycle, no wind-driven dust. The enclosure ingress test is still pending.
- **Instrument precision:** Power measurements come from a digital multimeter rather than a precision current analyser. The measured values are not close to the decision boundaries, so this does not change any PASS or FAIL call, but a reader looking for a single authoritative deep-sleep figure should know we used the instruments available, not lab-grade ones.
- **Sample size:** Tests ran on a small set of sensor nodes and one gateway. Unit-to-unit variability (probe calibration, antenna matching, ADC offsets) has not been characterised across a statistically meaningful sample. The per-unit calibration step partly compensates.

5.16. Future Testing and Improvement Plan

Next steps, ordered by how much confidence they would add relative to the effort required.

- **Close PARTIAL and PENDING rows in section 10:** Record measured deep-sleep and active currents, run the formal enclosure ingress test, execute the adversarial security tests, and complete OTA fault-injection variants. None of this needs new infrastructure, just bench time.
- **Multi-sensor scalability:** Pair two then three sensors to a single gateway, run for a while, check packet interleaving and backend ingestion. The most likely place to surface a concurrency issue.
- **Cross-unit calibration data:** Collect dry/wet baselines and RSSI at a fixed distance across 5 to 10 units if available. Use the spread to tighten the calibration procedure and produce per-unit tolerance numbers.
- **Field deployment fragment:** Deploy one gateway and two sensors outdoors for as long as possible, ideally two weeks across changing weather. Capture the diagnostic trace and compare observed failure rates against what short runs predicted.
- **Synthetic backend load:** Replay synthetic sensor traffic at multiples of the normal rate and measure backend latency and database throughput. Needed before any scalability claim.
- **Precision power profiling:** Borrow or buy a precision current analyser, re-measure deep-sleep and brownout currents, produce a single authoritative battery-life figure per supported battery capacity.
- **Automated regression harness:** Wrap the main integration tests (HMAC, pairing, end-to-end, OTA) in scripts so they can be re-run on every firmware change. Turns releases from one-off events into a routine.

6. Experiment & Results

6.1. Introduction

6.1.1. Purpose of This Document

This document presents the experimental evaluation of the machine learning components in the TARAS platform. It reports the methodology, evidence, and observed performance of the models under both controlled and deployment-oriented conditions.

The evaluation covers two subsystems: the vision-based disease detection module and the LLM-based advisory module. Since these address different tasks, they are evaluated differently: the disease module with quantitative classification metrics, and the advisory module with qualitative, scenario-based testing.

6.2. Disease Detection Module

6.2.1. Evaluation Philosophy

The disease classifier is judged on real-world (field) performance, not

laboratory accuracy. Models trained on controlled datasets like PlantVillage reach 99%+ on similar lab images, but that does not transfer to field photos with uneven lighting, cluttered backgrounds, and partial occlusion.

The test set is therefore sliced by source. The field slice (PlantDoc photographs, 332 images) is the primary deployment benchmark. The lab slice (PlantVillage, 1,594 images) is a controlled reference that is already near-saturated (>0.99 macro F1), so it is not a meaningful discriminator between models. All model-selection decisions are driven by the field slice.

6.2.2. Experimental Design Principles

6.2.2.1. Real-World-Oriented Evaluation Strategy

Field images are the hard, deployment-relevant distribution. The field slice (PlantDoc, 332 images) is the primary benchmark; the lab slice (PlantVillage, 1,594 images) is a secondary, near-saturated reference. Reporting both makes the lab-to-field gap explicit instead of hiding real-world difficulty behind a high lab score.

6.2.2.2. Fair Model Comparison Protocol

Every candidate backbone uses the same split, preprocessing, augmentation, optimizer, and metrics. Performance differences therefore reflect architecture and input resolution, not training conditions.

6.2.2.3. Separation of Benchmark vs Deployment Experiments

The project ships two models from the same pipeline:

- A cloud (server) model optimized for maximum accuracy, used for server-side inference.
- A mobile model optimized for size and latency, run on-device.

Single backbones are benchmarked first. The strongest are combined into an ensemble for the server, and a compact student is distilled from a strong teacher for mobile.

6.2.3. Dataset and Data Pipeline

6.2.3.1. Dataset Sources

The dataset spans 14 disease and healthy classes across 7 crops (tomato, potato, corn, pepper, cherry, peach, squash), drawn from

three sources:

- **PlantVillage (lab):** 33-35k controlled-lighting, single-leaf images. Easy distribution.
- **PlantDoc (field):** 1.5k uncontrolled field photos with complex backgrounds. The production-relevant, hard distribution.
- **Real-world blend:** 40-74 extra field images, mostly covering spider_mites, which PlantDoc does not include.

Combining lab and field sources is what lets the model learn clear symptoms from clean images while staying robust on real photos. Domain shift between these sources is the central challenge of the task.

6.2.3.2. Dataset Construction Strategy

The 28,600 training images are heavily lab-dominated: 27,879 PlantVillage (97.5%), 681 PlantDoc (2.4%), and 40 real-world (0.1%). This field scarcity is the key constraint on real-world performance. It is countered with loss-side source weighting (Section 6.2.5.2) rather than by collecting more field data, which was not available.

6.2.3.3. Data Cleaning and Validation

The combined sources are deduplicated and leakage-audited before splitting. Exact duplicates (pHash and dHash) and near-duplicates (perceptual-hash Hamming distance ≤ 4 within a source) are clustered with a union-find pass, and whole clusters are kept on one side of the split. This guarantees no near-duplicate leaks from training into tests, which would otherwise inflate scores. A label-noise audit also removes images with uncertain or inconsistent labels.

6.2.3.4. Data Preprocessing Pipeline

All images are resized so the shortest side is 256, then center-cropped to 224x224, converted to tensors, and normalized with ImageNet mean and standard deviation in RGB order. The same preprocessing is used at training and inference so input statistics match.

6.2.3.5. Data Augmentation Strategy

Augmentation is applied to the training set only. It includes random-resized cropping (scale 0.7-1.0), horizontal flips, color jitter, motion and Gaussian blur, Gaussian noise, JPEG compression, and random shadows. This "field domain

randomization" makes the model less dependent on clean lab conditions. Validation and test sets are left unchanged for comparable evaluation.

6.2.3.6. Dataset Splitting Strategy

The 35,924 images are split 28,600 train / 5,383 validation / 1,941 tests, stratified per class and per source, with the test set balanced to 30-150 images per class. Each test image is tagged by source: 332 fields (PlantDoc), 1,594 lab (PlantVillage), and 15 real-world.

Three classes (mosaic_virus, spider_mites, target_spot) have zero field-test images, because PlantDoc does not cover them, so field macro F1 is computed over the 11 classes that have field coverage. Best-checkpoint selection during training maximizes a composite score.

0.5 × validation macro F1 + 0.5 × field macro F1

This balances clean-data stability against real-world performance and keeps selection aligned with deployment.

6.2.4. Model Architectures

6.2.4.1. CNN-Based Models

ConvNeXt is the convolutional backbone family evaluated in two variants: ConvNeXt-V1-Base and ConvNeXt-V2-Base. CNNs capture local features such as texture, edges, and lesion patterns, which are important for fine-grained disease symptoms. EfficientNet-B0 is also used, but only as the compact student for the mobile model (Section 6.2.7.2), not as a server backbone.

6.2.4.2. Transformer-Based Models

The Swin Transformer family is evaluated at several capacities and input sizes: Swin-Tiny and Swin-Small at 224, and Swin-Small and Swin-Base at 384. Swin uses shifted-window self-attention to combine local detail with broader leaf-level context. The 384 input size was explored specifically for robustness when the diseased leaf is small in the frame, a scenario flagged for real use.

6.2.4.3. Architectural Differences

CNNs build spatial hierarchies through stacked convolutions and excel at local texture. Swin models share information across regions through windowed attention, which helps when symptoms are subtle or spread out. Larger input (384) and higher capacity (Base) generally improve the field slice at the cost of size and

latency. Comparing both families under identical conditions determines which is most suitable per deployment target.

6.2.5. Training Configuration

6.2.5.1. Optimization Strategy

Training is two-stage. Stage 1 is a short head-only warmup with the backbone frozen, letting the new classifier converge. Stage 2 fine-tunes the full network with AdamW (lower learning rate on the body than the head) for up to 50 epochs, with early stopping on the composite score. The loss is cross-entropy with a 0.5 weight on lab samples, so field samples receive proportionally more gradient. An exponential moving average (EMA) of the weights is tracked, and the EMA checkpoint is often selected.

6.2.5.2. Sampling Strategy

Because field data is only 2.4% of training, balancing is done on the loss side rather than by resampling. Real-world samples are upweighted (×5), and one experiment upweighted PlantDoc (×8) to raise its effective share. Explicit per-class weighting was tested but consistently hurt results, so it was not used in the final models.

6.2.5.3. Hyperparameters

All backbones share one configuration so differences reflect architecture, not tuning. Input resolution is 224 or 384 depending on the variant, batch size is set by hardware limits, and Stage 2 runs to convergence with patience-based early stopping. Horizontal-flip test-time augmentation is applied at evaluation.

6.2.6. Evaluation Metrics

6.2.6.1. Classification Metrics

The module is evaluated with accuracy, macro F1, and weighted F1, reported on the full test set and on the field and lab slices separately. Macro F1 is the primary metric because it weights every class equally and exposes weak minority classes that accuracy would hide. Accuracy and weighted F1 are reported as secondary contexts.

6.2.6.2. Metric Justification

Scores are always separated into the field slice (PlantDoc, the 11 field-present classes) and the lab slice (PlantVillage). The field slice is the deployment benchmark; the lab slice is a saturated control. Keeping them separate prevents the large, easy lab set from masking real-world performance in a single blended number.

6.2.6.3. Validation Selection Metric

Checkpoints and ensembles are selected on validation, not test, to keep reported numbers honest. Selection maximizes 0.5 × validation macro F1 + 0.5 × field macro F1. Tuning on the test set was measured to inflate the field score by roughly 2 points, so it is avoided for all reported figures.

6.2.7. Experimental Results

6.2.7.1. Single-Backbone Benchmark

All backbones were trained under the identical recipe and compared on the test set (n = 1,941). Lab F1 is near-saturated for every model (~0.99); the field slice is the real discriminator. Higher input resolution (384) and field oversampling give the best single-model field F1.

Backbone	Input	Macro F1	Field F1	Lab F1	Params
Swin-Base (field-oversampled)	224	0.9678	0.8393	0.9935	88M
Swin-Small	384	0.9678	0.8348	0.9945	49M
Swin-Base	384	0.9696	0.8332	0.9968	87M
ConvNeXt-V1-Base	224	0.9678	0.8290	0.9964	88M
Swin-Small	224	0.9649	0.8224	0.9943	49M
Swin-Base	224	0.9665	0.8204	0.9963	88M
ConvNeXt-V2-Base	224	0.9594	0.8042	0.9900	88M

6.2.7.2. Deployed Models

- **Server ensemble:** The deployed cloud model is a three-way, equal-weight ensemble of ConvNeXt-V2-Base, Swin-Small (384), and the field-oversampled Swin-Base (224), chosen by a brute-force subset search on validation. It reaches field F1 0.8488, macro F1 0.9697, lab F1 0.9944, and a worst-class F1 of 0.6486, with temperature calibration applied at inference (expected calibration error reduced from roughly 8-10% to under 2%). Notably, the ensemble outperforms every single backbone on the field slice and on worst-class F1, and the subset search beats a

larger 5-way ensemble while being smaller and faster, because the third member (ConvNeXt-V2-Base) adds diversity rather than raw single-model strength.

Metric	Value
Accuracy	0.9701
Macro F1	0.9697
Field F1	0.8488
Lab F1	0.9944
Worst-class F1	0.6486

The mobile model trails the server ensemble on the field slice (0.79 vs 0.85), which is the expected cost of compressing a large teacher into a 7.7 MB student.

6.2.7.3. Per-Class Performance Analysis

Per-class field F1 for the server ensemble is shown below, over the 11 classes present in the field slice. The three classes without field-test images (mosaic_virus, spider_mites, target_spot) are evaluated only on the lab slice and are excluded here.

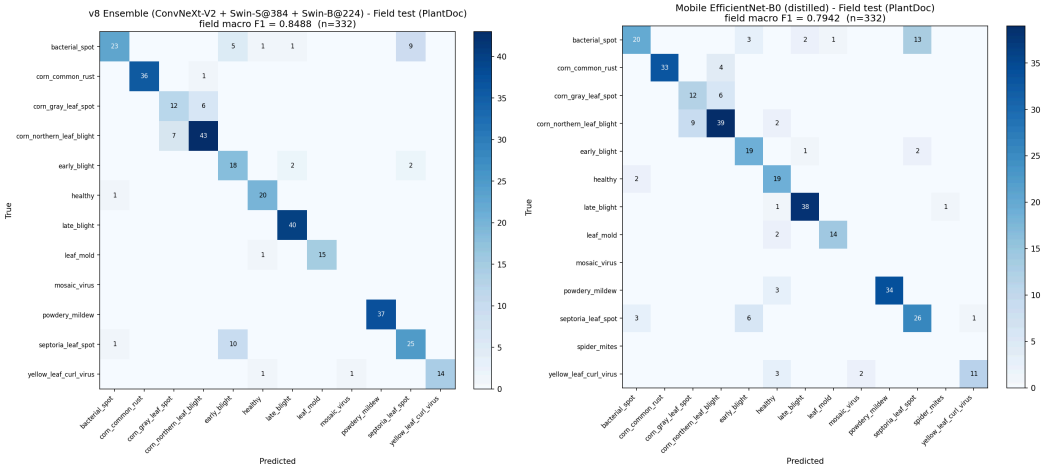
Class	Field images	Field F1
powdery_mildew	37	1.000
corn_common_rust	37	0.986
leaf_mold	16	0.968
late_blight	40	0.964
yellow_leaf_curl_virus	16	0.933
healthy	21	0.909
corn_northern_leaf_blight	50	0.860
bacterial_spot	39	0.719
septoria_leaf_spot	36	0.694
early_blight	22	0.655

Class	Field images	Field F1
corn_gray_leaf_spot	18	0.649

Classes with distinctive symptoms (powdery_mildew, rust, leaf_mold) are essentially solved on field images. The weak classes are the visually overlapping leaf-spot and blight groups: bacterial_spot, septoria_leaf_spot, early_blight, and corn_gray_leaf_spot, the last of which also has the fewest field-test images (18).

6.2.7.4. Confusion Patterns

Errors concentrate on visually similar classes rather than spreading randomly. The dominant confusion is bacterial_spot with septoria_leaf_spot, two tomato leaf-spot diseases with near-identical lesions. Early_blight and late_blight are confused with each other, as are the two corn diseases, corn_northern_leaf_blight and corn_gray_leaf_spot. These pairings explain the low field F1 of those classes and match the per-class results above.



6.2.8. Generalization and Robustness Analysis

6.2.8.1. Lab-to-Field Gap

The clearest robustness signal is the gap between the lab and field slices. Both deployed models score about 0.99 macro F1 on lab images but drop to 0.85 (server ensemble) and 0.79 (mobile) on field images. The gap is driven by the training distribution, where only 2.4% of images are field photos, and by the genuine difficulty of field conditions. The field score is therefore the honest estimate of deployment performance, and the lab score should not be read as real-world accuracy.

6.2.8.2. Class Stability

Stability across slices is uneven. Distinctive-symptom classes (powdery_mildew, rust, leaf_mold, late_blight) hold high F1 on both lab and field. The fragile classes are the leaf-spot and blight groups (bacterial_spot, septoria_leaf_spot, early_blight, corn_gray_leaf_spot), which are strong in the lab but weak in the field. Mobile-model checkpoint selection deliberately favored the worst-class floor over average F1 to keep these fragile classes from collapsing.

6.2.8.3. Failure Case Analysis

Most failures come from the visual-similarity confusions described in Section 6.2.7.4. Two structural limitations underlie them. First, field data is scarce (2.4% of training) and absent entirely for three classes, so the model has little real-world signal for the hardest cases. Second, several disease pairs are genuinely ambiguous at the leaf level, especially in early stages or poor lighting. The dataset is at its practical ceiling; further gains would require more field photographs, additional label cleaning, or hard-negative mining focused on the bacterial_spot and septoria_leaf_spot pair.

6.3. LLM Module Evaluation

6.3.1. Evaluation Methodology

The advisory module is evaluated qualitatively, not with the classification metrics used for the disease module. An advisory reply is open-ended text with no single correct answer, so accuracy and F1 do not apply.

Each response is judged on five points: factual correctness against live sensor data, correct tool selection, brevity, language fidelity (Turkish or English in Latin script), and no fabrication.

The module is agentic. For each question the model receives a cached system prompt, the farm and field inventory, and the last ten messages of the session, then runs a tool-use loop (reason, call a tool, read the result, reason again) for up to eight iterations within a 30-second budget. Evaluation exercises this full production loop, not raw model output.

Because the loop is model-portable, several candidate models were tested under identical conditions before one was chosen. Three procedures were used

- Functional validation of the tool loop against representative scenarios.
- A 25-run stability test per model, using a set of test prompts we wrote.

- A 7-prompt multi-turn quality test across a realistic conversation.

6.3.2. Scenario-Based Testing

Testing covered the four question types the assistant must handle in the field:

- **Live-data queries ("how is my field?", "should I irrigate Zone 2?"):** call a data tool, then answer in text leading with the number.
- **Agronomic-theory questions (crop coefficients, soil types, irrigation efficiency):** answered from the built-in reference, with a knowledge-base search only when the topic is not covered.
- **Navigation requests ("show me the moisture chart"):** a single navigation call followed by a one-sentence prompt on the target screen.
- **Out-of-scope or data-absent questions:** decline in one sentence without inventing a value.

Several language models were evaluated against these scenarios under the identical production loop before one was chosen. Claude Haiku 4.5 (Anthropic) was selected as the production model. As a frontier model it gave the most reliable agentic behavior: native structured tool calls, consistent adherence to the brevity and language rules, and none of the failure modes that affected the open candidates.

In parallel, a set of open models hosted on Groq was evaluated as a zero-cost alternative. Each candidate was run 25 times against a set of test prompts we wrote to check stability and quality, and scored on the 7-prompt multi-turn test. gpt-oss-120b was clearly the strongest: 93.5% multi-turn quality, the best open-model reasoning (MMLU-Pro 90), and zero language drift, channel leakage, or empty responses across the 25 runs. It is kept as a configurable alternative for cost-sensitive deployment.

The other open candidates were rejected for concrete, reproducible reasons:

- qwen3-32b: leaked non-Latin script, mixing Turkish, Chinese, and Japanese in three of five refusals.
- llama-4-scout: ignored the built-in agronomy reference and over-used the knowledge search.

6.3.3. Tool Usage Analysis

The assistant exposes eleven tools in three groups:

- **Navigation (1):** opens a specific section of a screen.
- **Data (9):** live PostgreSQL reads for field overview; zone latest readings, history, and configuration; irrigation history; sensor diagnostics; carbon

summary; disease history; and active alerts.

- **Knowledge (1):** a full-text search over the agricultural knowledge base.

Tool selection follows an explicit decision tree in the system prompt with a text-first policy. The assistant answers with numbers from a data tool whenever possible, and uses navigation only when seeing a chart, the 3D field, or a camera workflow beats a text reply.

Three behaviors were checked:

- **Correct selection:** A data tool for status questions, not a navigation "for completeness".
- **Parameter discipline:** the exact UUIDs from the per-request context are reused, since a truncated or invented ID breaks the query.
- **Loop efficiency:** the loop is capped at eight iterations, with one tool call per turn unless several zones or ranges are genuinely needed, and never a repeated identical call.

Tool handling was a decisive factor in model selection. Several open candidates failed here specifically, which ruled them out regardless of language quality.

6.3.4. Latency Analysis

Latency is driven by the agentic loop, not a single call. Each turn can issue up to eight sequential model calls within a 30-second timeout, though the decision tree keeps most queries to one or two iterations.

Two mechanisms reduce perceived latency:

- The system prompt is served through Anthropic's ephemeral prompt cache, so its large token block is not reprocessed or re-billed every turn.
- The final answer is streamed to the client over Server-Sent Events, and each tool phase emits an "analyzing" status so the interface can show progress.

Request latency and iteration counts are logged on the server. Typical end-to-end response time is three to six seconds. A lighter Groq model (gpt-oss-20b) was observed around 4 seconds end-to-end and can be swapped in when latency matters more than reasoning depth.

6.3.5. Reliability and Grounding

Grounding is enforced at three levels:

- **Live data:** all readings come from the nine data tools, which query PostgreSQL directly. Stale results are not reused; the assistant re-fetches when asked again.
- **Built-in reference:** agronomic theory is answered from a condensed reference (crop coefficients by stage, soil field-capacity and wilting-point

values, irrigation efficiencies, ET formulas) before any external lookup.
- **No fabrication:** when data is missing or a tool fails, the assistant says so in one sentence instead of guessing.

Operational reliability comes from several safeguards. Timeouts and the maximum-iteration limit return a graceful message rather than a partial answer. Advisory requests are rate-limited to three per minute per user for cost control. Model output is sanitized to strip reasoning tags, legacy tool-call text, and reasoning-channel tokens before display.

The tool loop is model-portable. It runs on Claude Haiku 4.5 in production but can switch to a Groq-hosted open model without code changes, so the service is not tied to one provider.

Language safety was a specific concern, since the chat is rendered in Latin script for Turkish users. Across the 25-run test the open alternative (gpt-oss-120b) produced no language drift and no non-Latin output, unlike qwen3-32b, which mixed three scripts in a single refusal. The production model showed no such issues.

6.3.6. Failure Cases

Evaluation surfaced several failure modes that shaped model selection and design.

The most serious was script mixing. One candidate produced blended Turkish, Chinese, and Japanese text in about three of five refusals, which is unacceptable for a Turkish-facing product.

Tool-protocol failures were also disqualifying: one model bypassed the built-in reference and over-searched, another emitted a deprecated text-based tool-call format, and the compound models did not accept custom tools.

A lower-severity issue affected every open model: a markdown bias, with about 60% of replies opening in a list despite the brevity rules. This is handled by post-processing rather than trusting the model.

Two failure modes remain by design. A query that exceeds eight iterations or the 30-second budget returns a graceful fallback message instead of an incomplete answer. Claude Haiku 4.5, the production model, showed none of the script or tool-format failures seen in the open candidates.

7. Discussion

7.1. Introduction

7.1.1. Purpose of the Document

This chapter discusses the AI and ML components of the TARAS precision-agriculture platform. Rather than repeating implementation details from the System Architecture and Methodology report, it interprets and justifies the design decisions behind the two ML-related modules, reflects on their results, and compares them with relevant literature. It explains why the selected approaches fit the problem, what the experimental evidence shows, and where the current design has limitations.

The Disease Detection and LLM-based Advisory modules are treated as separate subsystems. Although both support the same decision-support workflow, they solve different problems: one is visual and discriminative, while the other is linguistic and generative. Therefore, they are evaluated with different criteria and compared against different related work.

When result tables, performance metrics, or architectural diagrams are mentioned, the relevant sections of the main report are referenced. If the current system differs from the originally documented version, those differences are stated explicitly.

7.1.2. AI/ML Scope of the Project

TARAS includes several data-driven subsystems, but only two are machine-learning components in the strict sense: the vision-based plant disease detection module and the LLM-based advisory module. The disease detection module classifies leaf images using an ensemble of CNN and transformer backbones (ConvNeXt and Swin), while the advisory module interprets sensor readings, irrigation outputs, disease results, and user input to generate natural-language guidance.

The irrigation recommendation subsystem is excluded from this chapter. Although it is central to TARAS, it is based on calibrated agronomic calculations using reference evapotranspiration (ET_0), crop coefficient (Kc), and field sensor observations, following the FAO-56 methodology. Since this pipeline is deterministic and rule-based rather than learned from data, it is not evaluated here using classification metrics or learning-based arguments.

These two ML modules serve different purposes and are therefore discussed separately. Disease detection is a discriminative image-classification task, while the LLM advisory module is a generative, context-aware task that

combines structured system outputs with user text. Because they differ in input type, output form, and evaluation criteria, they are compared against different bodies of related work.

7.2. Disease Detection Module

7.2.1. Problem Definition

The disease detection module identifies plant diseases from a single RGB leaf photo and returns one disease-or-health label with a confidence score. The current model covers fourteen treatment-distinct, cross-crop-merged classes for bell pepper, corn, potato, squash, and tomato workflows, with additional healthy cherry and peach samples used during training. The deployed labels include bacterial_spot, early_blight, late_blight, healthy, powdery_mildew, target_spot, yellow_leaf_curl_virus, and other crop-specific disease classes.

This module is important because early diagnosis can prevent serious yield loss, especially for fast-spreading diseases such as late blight, bacterial spot, early blight, and leaf mold. It is designed to help small-scale farmers who may not have regular access to an agronomist.

The main challenge is that field images vary in lighting, angle, occlusion, and background complexity. Some diseases also look visually similar in early stages, so the model must distinguish subtle cues such as lesion shape, margins, patterns, and small structures.

Within TARAS, the module works both as a standalone diagnostic tool and as an input to the LLM-based advisory module, which combines disease results with humidity, irrigation, and field history to generate contextual guidance.

7.2.2. Dataset and Data Preparation

The disease detection module uses a hybrid dataset combining clean laboratory images and real-world field images. PlantVillage provides a large, controlled lab dataset with healthy and diseased leaves on plain backgrounds, but it does not fully represent field conditions.

To reduce this gap, TARAS also uses real-world sources: a curated RW dataset collected from permissively licensed sources such as iNaturalist, Wikimedia Commons, extension services, and open repositories, plus PlantDoc, which includes natural backgrounds, varied lighting, multiple leaves, and field-like conditions. Each RW image is tracked with metadata such as source URL, licence, label confidence, and visual difficulty.

The datasets are combined to address domain shift. PlantVillage is large but less realistic, while RW and PlantDoc images are more realistic but smaller

and noisier. The hybrid dataset benefits from both scale and field variability.

The real-world blend is small, about 40–74 images, and focuses on poorly covered classes, especially spider_mites. Images are reviewed for symptom clarity, background complexity, leaf visibility, and label reliability. Scarce classes are prioritized, and duplicates or near-duplicates are removed using perceptual hashing.

All sources use the same preprocessing pipeline: format and resolution checks, resizing, color standardization, and backbone-specific normalization. Augmentation, including flips, small rotations, brightness/contrast changes, and mild crop or zoom, is applied only to the training split. Validation and test images are not augmented. The held-out test set is tagged by source, with PlantDoc used as the field benchmark and PlantVillage as the clean reference.

The final training dataset used for the current production candidate contains 28,600 training images across 14 classes. The held-out test set (n=1,941) is composed by source as follows:

Class	Field (PD)	Lab (PV)	RW	Total
bacterial_spot	39	110	0	149
corn_common_rust	37	113	0	150
corn_gray_leaf_spot	18	77	0	95
corn_northern_leaf_blight	50	100	0	150
early_blight	22	126	0	148
healthy	21	129	0	150
late_blight	40	107	0	147
leaf_mold	16	133	0	149
mosaic_virus	0	56	0	56
powdery_mildew	37	113	0	150
septoria_leaf_spot	36	113	0	149
spider_mites	0	133	15	148
target_spot	0	150	0	150
yellow_leaf_curl_virus	16	134	0	150
Total	332	1594	15	1941

A validation set of 5,383 images is used for checkpoint selection. During model selection, TARAS maximizes a composite score of 0.5 x validation macro F1 plus 0.5 x field (PlantDoc) macro F1, so that real-world behaviour and clean-data stability are

weighted equally.

A single held-out test set of 1,941 images is maintained and tagged by image source. Its field slice (PlantDoc photographs, 332 images) is the principal real-world evaluation used for deployment decisions, while its lab slice (PlantVillage, 1,594 images) provides a clean, controlled reference; a small real-world blend (15 images) is also included.

7.2.3. Model Selection

The final deployment configuration is a three-member softmax-averaging ensemble, ConvNeXt-V2-Base, Swin-Small (384×384, window 12), and a field-oversampled Swin-Base, selected by a brute-force search over all three-member subsets of a 14-model candidate pool, ranked by a validation composite of 0.5·macro-F1 + 0.5·field-F1 (analogous to Caruana et al.'s ensemble-selection method [47]). The winning equal-weight trio is a strict Pareto improvement over the earlier five-member ensemble: equal or better on every test metric while being ~40% faster and ~38% smaller. It reaches 0.8488 field macro-F1 and 0.9697 full-test macro-F1, with temperature calibration cutting ECE from ~10% to ~1.4%. Single-backbone benchmarking (in which Swin-Small was the strongest individual model) is what justified building the ensemble; no single model is deployed on its own. Each member contributes orthogonal signal, ConvNeXt-V2's FCMAE pretraining[21], Swin-S's high-resolution input, and Swin-B's field-biased oversampling. These results suggest that the ensemble generalizes well across both real-world and balanced evaluation conditions.

Swin-Small is also suitable for the task from a modelling perspective. After label merging, the main challenge is less about broad class separation and more about fine-grained visual differences between similar diseases. Windowed self-attention allows the model to capture local lesion details while still considering broader leaf structure, which is useful for difficult pairs such as bacterial_spot vs. septoria_leaf_spot and early_blight vs. late_blight.

For deployment, the trained PyTorch model is converted to ONNX and served on AWS Lambda using ONNX Runtime. This replaced the earlier TensorFlow/Keras and TFLite path because it supports smaller container images, simpler CPU-only inference, and a clearer separation between training and serving. Before promotion, PyTorch and ONNX outputs are checked on fixture images, and a parity gate rejects any export whose maximum absolute softmax difference exceeds 0.005.

7.2.4. Training Strategy

The current system uses a unified, single-stage training strategy instead of the earlier curriculum-based approach. This choice makes the pipeline simpler, more stable, and easier to reproduce while still allowing the model to learn from both laboratory and real-world images.

The final training set combines PlantVillage (PV) images with the curated real-world (RW) dataset. This exposes the model to both clean, label-consistent laboratory images and more variable field-style images during the same training process. Instead of applying a staged transition from PV to RW data, domain balance is handled through sampling and loss weighting.

Class and domain imbalance are handled on the loss side rather than by resampling. Lab (PlantVillage) samples are down-weighted in the loss (composite weight 0.5) so the scarcer field samples contribute more gradient, and real-world sources are up-weighted (real-world x5, and PlantDoc x8 in the field-oversampled run). Per-class weighting was tested but it consistently hurt the hardest class, so it is not used in the final model.

Data augmentation is applied only to the training split. The pipeline includes random resized cropping, horizontal flipping, rotation, and mild brightness and contrast changes. These augmentations improve robustness to differences in viewpoint, lighting, and background while preserving the disease-specific visual patterns needed for classification.

The evaluation protocol is explicitly designed to reflect deployment conditions. Two separate test sets are maintained:

- **field slice: PlantDoc photographs only, representing the primary benchmark for deployment performance.**
- **lab slice: clean PlantVillage images, used to analyse per-class behaviour under a controlled distribution.** Model selection is based on a composite score that weights the full validation set and the real-world field (PlantDoc) slice equally:
- **the full validation set macro F1, weighted 0.5 ● the real-world field (PlantDoc) macro F1, weighted 0.5**

This weighting ensures that the selected model prioritises real-world generalisation while still retaining sensitivity to clean, canonical visual patterns.

7.2.5. Experimental Results

The most important numerical result of this work is the distinction between the final deployment candidate and the broader comparison set. The deployed three-member ensemble achieves 0.9701 accuracy and 0.9697 macro F1 on the full held-out test set, and 0.8488 macro F1 on the real-world field slice (with 0.9944 macro F1 on the clean lab slice).

Metric	Value
Test Macro F1 (overall)	0.9697
Test Accuracy	0.9701
Field Macro F1 (PlantDoc n=332)	0.8488

Lab Macro F1 (PlantVillage n=1594)	0.9944
Validation composite (selection)	0.9273
Minimum per-class field F1	0.6486

The single-backbone benchmark on the revised dataset is what motivated building the ensemble: no individual backbone matched the three-member ensemble's balance of field and overall macro F1, so the deployed system averages the calibrated softmax of ConvNeXt-V2-Base, Swin-Small (384), and Swin-Base rather than shipping any single backbone.

Backbone	Params	Input	Val composite	Test Macro F1	Test Field F1	Role / verdict
ConvNeXt-V2-Base (FCMAE)	88M	224	0.8474	0.9594	0.8042	Top ensemble contributor
ConvNeXt-V1-Base	88M	224	0.8651	0.9678	0.8290	Strong single backbone
Swin-Base @384	87M	384	0.8701	0.9696	0.8332	Distant-leaf robust
Swin-Base @224	88M	224	0.8747	0.9665	0.8204	Core member
Swin-Small @224	49M	224	0.8662	0.9649	0.8224	Efficient peer
Swin-Small @384	49M	384	—	—	0.8348	Ensemble member (highest single field F1)
Swin-Tiny @224	28M	224	0.8433	0.9648	0.8204	Redundant with Swin-S/B
Swin-Base + class weights	88M	224	0.8377	0.9637	0.8053	Class weights hurt hardest class
CAFormer-S36	37M	224	0.8704	0.9623	0.8207	Screened; no test gain
Hiera-Base	51M	224	0.831	0.9605	0.7776	Screened
MaxViT-Small	68M	224	—	—	—	Training failed (NaN on ROCm)

Per-class results of the deployed ensemble on the real-world field slice show:

Class	Field-test n	Field F1
powdery_mildew	37	1
corn_common_rust	37	0.986
leaf_mold	16	0.968
late_blight	40	0.964

yellow_leaf_curl_virus	16	0.933
healthy	21	0.909
corn_northern_leaf_blight	50	0.86
bacterial_spot	39	0.719
septoria_leaf_spot	36	0.694
early_blight	22	0.655
corn_gray_leaf_spot	18	0.649
Field Macro F1 (11 classes)	332	0.849

7.2.6. Interpretation of Results

The revised results support a balanced conclusion: no single architecture is inherently superior. Final performance depends strongly on label design, dataset quality, and model selection.

Candidate models were trained under the same setup, and the final classifier was selected based on measured performance rather than architectural assumptions. The real-world field slice was treated as the main deployment-readiness benchmark, while the lab-heavy full test set served as a secondary sanity check.

The deployed ensemble achieves 0.8488 macro F1 on the real-world field slice, 0.9697 macro F1 on the full test set, and 0.9944 on the clean lab slice. This performance gap reflects the difficulty of uncontrolled field imagery rather than weakness on clean data.

Remaining errors mainly occur between visually similar leaf-spot and blight classes, such as bacterial_spot vs. septoria_leaf_spot or early_blight vs. late_blight. This shows the limits of single-image diagnosis without extra crop metadata or multiple images.

Overall, the final TARAS classifier is a deployment-oriented model selected after dataset revision, label-schema redesign, model comparison, and error analysis, not only a laboratory benchmark model.

7.2.7. Comparison with Related Studies

The classic reference point remains Mohanty, Hughes and Salathe (2016), who trained deep CNNs on 54,306 PlantVillage images and reported 99.35% accuracy on a held-out test set[7]. That result established the feasibility of deep learning for plant disease detection, but it also exemplifies the main limitation that later studies exposed: excellent laboratory accuracy does not guarantee strong field performance[48]. TARAS deliberately accepts lower headline numbers in exchange for evaluation on more realistic imagery and therefore

addresses a different question than PlantVillage-only studies.

A useful field-oriented comparison is PlantDoc and later work built around it. Introduced PlantDoc as a challenging real-world benchmark with cluttered backgrounds and cross-domain variability [49], and later object-detection and classification studies continued to use it precisely because it is harder than PlantVillage. Relative to this line of work, TARAS is similar in spirit because it prioritises natural images, but it differs in being a task-specific hybrid corpus with label redesign guided by error analysis rather than a fixed public benchmark alone.

For comparison with modern transformer-based approaches,recent studies based on Swin Transformer architecture have demonstrated strong performance under natural and noisy image conditions[22], [50]. TARAS is aligned with these findings: the final selection of a transformer-based ensemble over the revised 14-class dataset supports the claim that transformer-style feature modelling can be advantageous once the label schema and data preparation are appropriate.

The main point of difference is methodological. Most published papers compare architectures on a fixed benchmark, whereas TARAS also redesigns the label taxonomy when the benchmarked error patterns show that a finer distinction is not reliable enough for deployment. This is why the final document reports not only architecture comparison but also the decision to merge classes by treatment across crops, for example collapsing early blight and late blight on both tomato and potato into single, crop-agnostic early_blight and late_blight classes. For a live agricultural system, this decision is more meaningful than preserving a more granular taxonomy that produces unstable advice.

Taken together, the literature suggests that TARAS occupies an intermediate position. It does not chase PlantVillage-only accuracy numbers, and it does not depend exclusively on one public field dataset either. Instead, it uses a curated hybrid dataset, a deployment-oriented test split and a model-selection process that is directly tied to the target application

7.2.8. Limitations of the Module

Several limitations follow from the current design and should be acknowledged clearly. First, the classifier is limited to fourteen categories across five crops. Images outside this scope, such as different crops, unseen diseases, nutrient deficiencies, pest damage, or abiotic stress, may still be forced into one of the known classes. Since the model does not currently include an out-of-distribution rejection class, the backend confidence threshold is the only mechanism for reducing unreliable predictions, and this is not a perfect solution.

Second, the real-world dataset is still relatively small. Although source diversity was considered during collection, the dataset reflects the regions,

seasons, and crop conditions available in permissively licensed image sources. As a result, the model is likely to perform better in contexts similar to the training data and less reliably in underrepresented conditions, such as specific greenhouse cases in Türkiye. Expanding the real-world dataset remains an important future improvement.

Third, some classes remain weaker because of dataset imbalance. PlantVillage contains fewer examples for classes such as mosaic_virus and healthy, and augmentation cannot fully replace real image diversity. The curated real-world data helps reduce this issue, but these classes still represent weak points in the current model.

Fourth, the module works at the image level. It classifies one leaf image at a time and does not reason across multiple images of the same plant or track symptom progression over time. This limits performance in early-stage disease cases, where a single leaf may be ambiguous but several images over time could make the diagnosis clearer.

Finally, the module is only a visual diagnostic aid. Some plant-health problems, such as over-irrigation, nutrient deficiency, or root disease, may produce visible symptoms without being identifiable from the image alone. The LLM-based advisory module partially addresses this by combining disease predictions with sensor and irrigation history, but the image classifier should not be treated as a standalone diagnostic authority. Its role is to narrow down possible causes and support further investigation.

7.3. LLM Module

7.3.1. Problem Definition

The LLM-based advisory module addresses a different but complementary problem to disease detection: it turns system outputs into explanations that users can understand and act on. Soil moisture values, irrigation recommendations, and disease confidence scores are useful only if the user can interpret what they mean for their field. Since TARAS is designed for non-expert users, this translation layer is an important part of the platform.

The module answers questions such as “What do these values mean for my field today?” and “What should I do next?” using the context of the user’s farm, field, zone, and crop. Its responses are generated from structured inputs such as live sensor readings, recent irrigation decisions, disease detection results, and user-provided information. The goal is not to create an open-ended chatbot, but a decision-support assistant whose answers remain grounded in the data available in the system.

The main design challenge is balancing usefulness and reliability. The module needs to be flexible enough to answer different user questions, but

constrained enough to avoid common LLM problems such as hallucinated details, overconfident statements, or plausible but incorrect recommendations. Therefore, the advisory system is designed to produce practical guidance while keeping its reasoning tied to concrete data sources and curated agricultural knowledge.

7.3.2. System Design

The advisory module is implemented as an agentic pipeline instead of a single-shot prompt. When the user asks a question, the model does not answer only from its general training knowledge. It first decides whether it needs to call one or more read-only tools to ground the answer. These tools can retrieve live database records, search the curated agricultural knowledge base, or direct the mobile app to a relevant screen when visual information is more useful than text. The final response is generated only after the tool results are returned.

The agent currently has access to eleven tools: one navigation tool, nine live-data read tools (field overview, latest zone reading, zone history, zone configuration, irrigation history, sensor diagnostics, carbon summary, disease history, and active alerts), and one knowledge-base search tool. To keep the system controlled, the tool-use loop is capped at eight iterations with a thirty-second wall-clock timeout, plus per-tool input schemas, user-scoped data access, and capped result sizes.

Domain grounding is mainly provided through the search_knowledge tool. This tool uses PostgreSQL full-text search over a curated knowledge base of about 146 fragments covering crop guides, diseases, pests, soils, fertilizers, irrigation practices, climate considerations, and system usage. Instead of relying only on the LLM's internal knowledge for agronomic advice, the system retrieves relevant project-controlled guidance and lets the model reason over it. This also allows the advisory knowledge to be updated without retraining the model.

The second grounding mechanism is the system prompt. It is compact, cached, and designed to enforce a few key behaviours: answer in the user's language, stay short and direct unless more detail is needed, use tools for user-specific data, and use navigation only when a visual screen would genuinely help. This text-first design was chosen because earlier versions overused navigation and interrupted the user's flow.

7.3.3. Model Selection and Explanation

The production model is Anthropic Claude Haiku 4.5 (claude-haiku-4-5-20251001), accessed through the Anthropic TypeScript SDK. Haiku 4.5 was chosen because, as a frontier model, it gave the most reliable agentic behaviour: native structured tool calls, consistent adherence to the brevity and language rules, and none of the failure modes such as mixed-script

output or malformed tool calls seen in the open-weight candidates, while remaining fast and inexpensive enough for interactive use. The system also includes a fallback path that runs the same agentic tool loop through Groq, using openai/gpt-oss-120b by default (Qwen3 was evaluated and rejected because it produced mixed-script Turkish output). The fallback uses a separately tuned, structurally simpler system prompt adapted to open-weight models, but it exposes the same eleven tools and the same eight-iteration loop.

The agentic design was selected over simpler alternatives such as a single-shot prompt or fixed retrieval pipeline. Different user questions require different parts of the system state: soil-moisture questions may need zone history, disease-related questions may need irrigation history, and explanation questions may need the knowledge base. Instead of inlining all possible context into every request, the tool-loop lets the model retrieve only what is relevant. This keeps responses cheaper, more focused, and easier to trace through the selected tool calls.

7.3.4. Benefits and Challenges

The main benefit of the advisory module is accessibility. Raw values such as soil moisture, crop coefficient, or disease confidence scores are difficult for non-expert users to interpret on their own. The advisory module turns these outputs into practical explanations, such as why a specific zone needs irrigation today while another does not. In this sense, it makes the platform's technical outputs usable for the intended audience.

The module also acts as an integration layer across TARAS subsystems. Disease results, irrigation recommendations, sensor readings, and field history are stored separately, but user questions often combine them. For example, a question such as "Is this leaf spot related to overwatering?" requires disease output, irrigation history, and humidity data together. The tool-loop design allows the module to retrieve and connect these pieces without requiring the user to understand the underlying database structure.

Another benefit is explainability. Because responses are grounded in tool outputs and retrieved knowledge, the module can refer to the specific factors behind an answer, such as current soil moisture, growth stage, irrigation history, or a relevant disease guide. This makes the response more trustworthy than a generic chatbot answer.

The main challenges are accuracy, latency, cost, reliability, and trust boundaries. The grounding mechanisms reduce the risk of hallucination, but they do not eliminate it, especially for edge cases outside the curated knowledge base. The agentic tool loop also adds latency because some questions require multiple tool calls, although streaming improves perceived response time. Cost is controlled through rate limits and result-size caps, while provider outages are

handled through the Anthropic/Groq fallback setup. Finally, the advisory output must be framed as decision support, not as a replacement for expert diagnosis, especially for high-risk cases such as possible late blight.

7.3.5. Comparison with Related Studies

Recent studies on LLMs in agriculture, such as Wang et al. (2024), identify the same problem addressed by the TARAS advisory module: smart-farming systems can produce useful analytical outputs, but farmers may struggle to interpret them. LLMs can help by translating sensor readings, model outputs, and agronomic recommendations into natural-language explanations. TARAS follows the retrieval-augmented generation (RAG) approach rather than fine-tuning: Claude Haiku 4.5 is grounded with a curated agricultural knowledge base retrieved through PostgreSQL full-text search.

Most agricultural chatbot systems fall into two broad groups. Rule-based systems are predictable and explainable, but they are limited to predefined questions. Free-form LLM chatbots are more flexible, but they can produce unsupported or incorrect advice. TARAS is designed to sit between these two approaches. Users can ask questions naturally, but the model is constrained by tool use, scoped live data access, and retrieval grounding.

The TARAS advisory module differs from many agricultural LLM systems in three main ways. First, it connects to live sensor, irrigation, disease, diagnostic, and carbon data through scoped and audit-logged tools. Second, it remains text-first and uses navigation only when a screen view would help the user more than a written answer. Third, it includes a provider fallback path, so the advisory function can continue in a simpler form if the primary LLM provider is unavailable.

7.3.6. Limitations of the Module

The advisory module depends heavily on the quality of its upstream data. If a zone has faulty sensors, stale readings, or inconsistent metadata, the generated response may also be unreliable, even if it sounds fluent. The get_sensor_diagnostics tool helps reduce this risk by allowing the agent to check sensor-health information and add caveats when needed, but the module cannot be more accurate than the data it receives.

Retrieval quality is another limitation. The curated knowledge base contains about 146 fragments across eight categories, so it is useful but not encyclopedic. Questions that match its coverage can be grounded well, while questions outside its scope may rely more on the model's general knowledge. For this reason, expanding and maintaining the knowledge base remains an important future task.

The module should also not be treated as a replacement for expert

agronomic judgment. In complex or high-risk cases, such as unusual disease symptoms, multiple stress factors, or decisions with serious economic consequences, the advisory response should be used as a starting point rather than a final authority.

The agentic tool loop introduces some tool-specific risks, such as selecting the wrong tool, misreading a tool result, or answering with incomplete context. These risks are reduced through iteration limits, result-size caps, scope checks, and audit logging, but they cannot be removed completely.

Finally, the module requires an internet connection. This may be a limitation in rural areas with unstable connectivity. At present, the mobile client can show the last-known advisory state, but it cannot generate new advisory responses offline. An offline-capable lightweight model could be considered as future work.

7.4. Future Work

Both ML modules are at a point where the core design is stable and the remaining work is largely incremental, extending coverage, improving robustness, and closing specific gaps identified during development. The items below are grouped by module and ordered loosely by estimated impact on the user experience.

7.4.1. Disease Detection Improvements

Future improvements for the disease-detection module should build on the revised 14-class ensemble system rather than returning to a broader but less reliable label set. The first priority is targeted dataset expansion for the hardest categories, especially the bacterial_spot/septoria_leaf_spot and corn leaf-spot pairs. Adding more real-world images from Turkish greenhouse and open-field conditions would likely improve performance more than switching to a new backbone.

A second priority is confidence calibration and rejection handling. Although the current model gives strong top-1 predictions, production use would benefit from calibrated thresholds and a fallback path for uncertain cases. For example, the system could return “low confidence, please capture another image or use the advisory module” instead of always forcing a class.

Future architecture experiments should also be selective. The current results already identify Swin-Small as a strong deployment candidate, with credible alternatives such as ConvNeXt-Small. Therefore, the next step should focus on improving the deployed ensemble pipeline and occasionally testing it against a small number of strong baselines.

7.4.2. LLM Module Improvements

The LLM advisory module has four main future improvements.

First, the knowledge base should be expanded. The current 146 curated fragments cover common cases well, but unusual pest–disease interactions, region-specific crop guidance, and less common soil types need more coverage. New content should remain curated and attributable rather than scraped from the open web.

Second, the tool set can be extended. Useful additions include a weather-forecast tool, a historical-analogue tool for comparing current conditions with similar past cases, and a push-notification tool for proactive alerts.

Third, offline and low-bandwidth support should be considered. A small on-device fallback model could answer a narrow set of common questions using cached field data, such as irrigation status, recent disease results, and basic crop-stage guidance.

Fourth, the module needs a feedback loop. A simple thumbs-up/thumbs-down mechanism would help identify weak answers, improve the system prompt, and guide future knowledge-base updates.

Additional improvements could include conversation summarisation, voice input, what-if irrigation scenarios, and more detailed per-tool authorization.

7.5. Conclusion

This chapter discussed the two AI/ML components of TARAS: the vision-based disease detection module and the LLM-based advisory module. Although they solve different problems, both support the same goal: helping users interpret field data and make better agricultural decisions.

For disease detection, the project converged on a revised 14-class label schema and, after benchmarking individual backbones, selected a three-member ensemble (ConvNeXt-V2-Base, Swin-Small at 384, and Swin-Base) as the final deployment configuration. On the real-world field test slice the ensemble reaches 0.8488 macro F1, and 0.9697 macro F1 on the full balanced test set. These results show that the strongest improvements came not only from model architecture, but from aligning the label schema, dataset, and evaluation protocol with real deployment conditions.

The LLM advisory module addresses a different need: turning structured system outputs into understandable guidance. Its agentic tool-loop design, scoped read-only tools, curated knowledge base, and provider fallback were chosen to balance usefulness with reliability. The module has been demonstrated end-to-end in production, with typical response times of three to six seconds, but it has not yet been evaluated through a large-scale quantitative study. Its current evaluation is therefore mainly qualitative, based on coherence, grounding, and consistency with system outputs.

The current limitations are clear: the classifier is limited to fourteen classes, the advisory knowledge base is compact, the LLM module depends on cloud connectivity,

and the classifier does not yet provide visual explanation maps. However, these are incremental improvement areas rather than reasons to redesign the system from scratch.

Overall, the AI/ML work in TARAS follows a consistent design philosophy: both modules should be useful without overclaiming, flexible without becoming ungrounded, and supportive without replacing expert judgment. The disease classifier and advisory agent are therefore best understood as decision-support tools that help non-expert users make better use of the data produced by their own fields.

8. Conclusion & Future Work

8.1. Overview

This document brings together the work presented across the preceding ones and reflects on what the TARAS project has achieved as a whole. TARAS (Tarım Analizi Sistemi) was developed as an end-to-end precision-agriculture decision-support platform that integrates IoT-based field sensing, physics-grounded irrigation analysis, computer-vision-based disease detection, an LLM-driven advisory layer, and a mobile application built around a digital-twin visualization of the field. The intent was never to automate farming, but to give farmers a single, accessible interface that turns raw sensor and image data into clear, explainable guidance. The conclusions below summarize what was built, where it currently stands, and the limitations that shaped both the design decisions and the direction of future work.

The project began with a broad survey of smart-agriculture features and was deliberately narrowed, based on impact, feasibility, cost, and sustainability, to the components that could be delivered to a defensible standard within the project timeline. What follows is a synthesis of the outcomes from each subsystem, an honest account of the limitations that remain, and a structured plan for the work that should come next.

8.2. Summary of Contributions

8.2.1. Hardware and Field Sensing

The hardware layer was designed around an ESP32-C6 sensor node paired with a mains-powered ESP32 WROOM-32 gateway. Each sensor node samples soil moisture through a capacitive probe and temperature and humidity through an SHT31 over I²C, running on a 3.7 V Li-Po battery, deep-sleep cycling, and a per-unit calibration step that absorbs probe-to-probe variation. The node-to-gateway link uses ESP-NOW with HMAC signing and replay protection, while the gateway-to-backend channel was migrated to HTTPS:443 with full certificate-chain validation against the Let's Encrypt root CA. Across 52 defined hardware test cases, 42 passed, 3 were partial, and 7 remain pending; nothing reached a FAIL state. The deployed gateway has been running continuously on the final firmware since the upgrade with no observed regressions, which serves

as a second source of evidence alongside the targeted bench and integration tests.

8.2.2. Backend, Cloud, and Data Layer

The backend was implemented as a Node.js API service on AWS, fronted by a PostgreSQL database that holds users, farms, fields, zones, plantings, time-series telemetry, irrigation recommendations, carbon-footprint records, and disease diagnoses. Disease inference was deployed as a Lambda function backed by a three-model classifier ensemble (a ConvNeXt-V2-Base, a Swin-Small, and a field-tuned Swin-Base), including a queue-and-resilience layer that survives transient failures by reverting requests to a QUEUED state and retrying through a cron-driven scheduler. Cloud-side optimization work brought the disease-inference cold-start down from roughly sixty seconds to approximately fourteen seconds across multiple iterations, with subsequent effort redirected toward cost per invocation rather than further latency reduction. The result is a backend that behaves well under intermittent connectivity, retries gracefully when downstream services are unavailable, and exposes a clean REST surface for the mobile client.

8.2.3. Irrigation Recommendation Module

The irrigation recommendation module was implemented as a decision-support feature that uses real-time soil moisture readings, crop information, field configuration and planting data to determine whether irrigation is needed. Instead of following a fixed irrigation schedule, the system evaluates the current condition of each zone and generates recommendations such as whether the crop should be irrigated, the suggested irrigation time, urgency level, target soil moisture and estimated water amount.

Each recommendation is stored in the database as an irrigation job, allowing the system to track pending, executed, skipped, and analyzed irrigation decisions. After irrigation is performed, follow-up sensor readings can be used to compare the predicted soil moisture response with the actual result. This creates a feedback loop that supports calibration and improves the reliability of future recommendations.

The main contribution of this module is that it turns raw sensor data into practical, zone-specific irrigation guidance. By helping farmers avoid both over-irrigation and under-irrigation, the module supports water efficiency, crop health, and more informed field management.

8.2.4. Disease Detection Module

The vision-based classifier converged on a revised fourteen-class label schema covering the most relevant disease and healthy categories across the supported crops. A three-model ensemble was selected as the final deployment

configuration after a controlled benchmark in which a pool of backbones, including ConvNeXt-V2-Base, Swin-Small, Swin-Tiny, and Swin-Base at both 224 px and 384 px, was trained under identical conditions on the same revised dataset, and the best-performing subset was combined. On the held-out test set (1,941 images), the deployed ensemble reached an overall accuracy of 0.9701 and a macro F1 of 0.9697. The more informative figure is the real-world field slice (332 images), where the macro F1 was 0.8488, compared with 0.9944 on the cleaner laboratory (PlantVillage-style) slice (1,594 images). This field-slice result is notable because it was measured under deployment-oriented conditions rather than on a clean PlantVillage-style benchmark, which makes it a stronger predictor of behaviour in actual field use.

The most consequential lesson from this subsystem was that the largest improvements did not come from chasing a stronger backbone, but from aligning the label schema, the dataset composition, and the evaluation protocol with the conditions the model would actually encounter. The composite validation score (0.5 × overall val macro F1 + 0.5 × field-slice macro F1) operationalized this principle during model selection, and the project treats the field (real-world) slice as the primary indicator of deployment readiness.

8.2.5. LLM Advisory Module

The advisory layer was built as an agentic tool-loop running on top of an LLM with a small, scoped set of read-only tools and a curated knowledge base of around 146 fragments across eight categories. Its role is to translate the structured outputs of the rest of the system, sensor readings, irrigation recommendations, disease predictions, and field metadata, into context-aware natural-language guidance that the farmer can act on. Provider fallback, iteration limits, result-size caps, scope checks, and audit logging were added to keep the agent grounded and to bound the risks introduced by autonomous tool selection. End-to-end response times in production typically fall between three and six seconds. Evaluation of this module remains qualitative, based on coherence, grounding, and consistency with upstream system outputs, since a large-scale quantitative study would require a labelled corpus that does not yet exist for this specific task.

8.2.6. Mobile Application and Digital Twin

The mobile application was implemented in React Native with Expo, chosen for cross-platform deployment and for the iteration speed it permits on a small team. Its main surfaces are the dashboard with sensor readings and a 2D/3D moisture visualization, the disease-detection flow with camera capture and history-folder organization, the irrigation timetable, the carbon-footprint logger, and a global advisory assistant overlay that exposes the LLM module to the user. The digital twin uses low-poly geometry (rendered with three.js / React-Three-Fiber) and an SVG overlay strategy over a standard mobile map so that spatial moisture

visualization runs smoothly on a standard smartphone, rather than requiring the desktop-class hardware that high-end agronomic twins typically assume. Throughout the project, the design defaulted to decision support rather than full automation: every recommendation is presented in a form the farmer can override, and the AI assistant is consistently framed as a guide rather than as an authority.

8.2.7. Sustainability and Carbon Footprint

Carbon-footprint estimation was implemented as a user-facing feature that allows farmers to log fuel, electricity, and fertilizer usage. Each input is converted into an estimated CO_2 equivalent value using the standard emission-factor approach:

$$\textbf{CO}_2\textbf{e Emission = Activity Amount} \times \textbf{Emission Factor}$$

Here, the activity amount represents the user entered value, such as liters of fuel, kilowatt-hours of electricity, or kilograms of fertilizer. The emission factors were selected from commonly used international references such as IPCC and UN-related sustainability guidelines. Each calculated result is saved as a carbon record, allowing the system to show total emissions and category-based emission history. This supports the project's connection with SDG 2, SDG 9, SDG 12, and SDG 13.

8.3. Reflections on the Design Philosophy

Three design commitments shaped the project from the literature-review stage onward, and they held up under implementation pressure. The first was decision support rather than automation: every analytical output is presented as a recommendation that the farmer can act on, ignore, or adjust, rather than as an instruction issued to an actuator. This kept the trust model simple and avoided the safety and accountability problems that come with closed-loop control of irrigation hardware. The second was real-world evaluation over laboratory-style benchmarks. The disease classifier in particular benefited from this commitment, since the strongest gains came from improving the realism of the evaluation pipeline rather than from architectural changes. The third was a mobile-first orientation, which was reflected in the choice of React Native, the low-poly digital twin, and the deliberate avoidance of features that would require desktop-class hardware in the field.

Each of these design commitments introduced certain trade-offs. Because TARAS is framed as a decision-support system rather than a fully automated control system, it cannot independently prevent a farmer from over-irrigating; instead, it provides evidence-based explanations that help the farmer understand why over-irrigation would be a poor decision. Similarly, evaluating the disease model on real-world test data resulted in lower headline accuracy than a PlantVillage-only benchmark might have produced, but these results offer a more realistic and honest indication of deployment performance. The mobile-first design also limited the use of more complex visualization techniques that would be easier to render on workstation-level hardware. These

trade-offs were intentional, and preserving this balance between practicality, transparency, and deployability remains important as the system evolves.

8.4. Limitations

The limitations identified during development can be grouped into four main areas, each of which defines an important direction for future work.

First, there are limitations related to scope. The disease classifier is currently limited to fourteen categories across a small number of crops, and it does not yet include an out-of-distribution rejection class. As a result, images outside the supported scope, such as unseen diseases, nutrient deficiencies, pest damage, or abiotic stress symptoms, may still be forced into one of the known classes. At this stage, the backend confidence threshold is the main mechanism used to reduce unreliable predictions. Similarly, the LLM knowledge base covers common agricultural cases, but it is not encyclopedic. Questions outside its retrieval scope may therefore rely more heavily on the model's general knowledge.

Second, the evidence base is limited in terms of coverage and duration. Hardware reliability data currently spans hours to days rather than weeks or months, so long-term battery life and multi-week stability are still extrapolated from shorter tests instead of being measured directly. Outdoor testing has also been limited to controlled damp-exposure tests and bench conditions that approximate field use, rather than full diurnal cycles involving sun, rain, wind, and dust. In addition, enclosure ingress testing remains pending, and scale-related claims have not yet been validated through multi-gateway deployments with tens of sensors per gateway.

Third, connectivity remains a practical constraint. Both the LLM advisory module and the disease-detection Lambda require an internet connection to generate new outputs, which may be problematic in rural areas with unstable connectivity. Although the mobile client can display the last-known system state while offline, it cannot generate new advisory responses or perform new disease classifications without a connection.

Fourth, the advisory module still requires deeper evaluation. The LLM module has been demonstrated end-to-end in production, but it has not yet undergone a large-scale quantitative evaluation using labelled rubrics for groundedness, helpfulness, and safety. The current evidence base is therefore qualitative. A more formal evaluation will be necessary before the module can be presented as anything beyond a decision-support aid.

8.5. Future Work

The remaining work is largely incremental: the core architecture is stable, and what remains is to extend coverage, improve robustness, and close specific gaps identified during development. The items below are grouped by subsystem and ordered loosely by estimated impact on the user experience.

8.5.1. Hardware and Field Deployment

The most valuable next step on the hardware side is a real outdoor deployment of at least one gateway and several sensors for two weeks or longer across changing weather, with continuous diagnostic capture. This would convert the current short-run reliability evidence into observed long-term behaviour and would expose any failure modes that bench testing cannot reach. Alongside this, the partial and pending test rows should be closed out, including formal deep-sleep and active-current measurements with a precision current analyser, the formal enclosure ingress test, the remaining adversarial security cases, and the outstanding OTA fault-injection variants.

Beyond coverage, two scalability questions remain open. The first is multi-sensor scalability on a single gateway, which would surface any concurrency issues in ESP-NOW packet interleaving and backend ingestion. The second is synthetic backend load: replaying sensor traffic at multiples of the normal rate and measuring backend latency and database throughput, which is needed before any claim about farm-scale rather than test-field-scale deployment. An automated regression harness wrapping the main integration tests (HMAC, pairing, end-to-end, OTA) would turn firmware releases from one-off events into a routine.

8.5.2. Disease Detection Module

Future improvements should build on the revised fourteen-class system rather than returning to a broader but less reliable label set. The first priority is targeted dataset expansion for the hardest categories, particularly the corn and tomato leaf-spot classes (bacterial_spot, septoria_leaf_spot, corn_gray_leaf_spot, and early_blight) which remain the weakest on the real-world slice. Adding more real-world images from Turkish greenhouse and open-field conditions would likely produce a larger gain than switching to a stronger backbone.

The second priority is confidence calibration and explicit rejection handling. A calibrated threshold and an out-of-distribution rejection path would let the system respond with "low confidence, please capture another image or use the advisory module" rather than forcing a class. Visual explanation maps (for example, attention or Grad-CAM-style overlays) would also improve user trust by showing which regions of the leaf drove the prediction. Further architecture experiments should be selective: the current results already identify the selected three-model ensemble (with Swin-Small as its strongest single member) as a strong deployment configuration, and the next round of work should improve this pipeline rather than restart the comparison.

8.5.3. Irrigation Recommendation Module

The irrigation recommendation module should be improved through longer-term field validation, stronger calibration, and better use of environmental context.

Although the system can currently generate zone-specific irrigation jobs using soil-moisture readings, crop information, field configuration, and planting data, its reliability still needs to be tested across different soil types, crop stages, and outdoor conditions.

A key next step is to collect more post-irrigation follow-up data. By comparing predicted soil-moisture changes with actual sensor readings after irrigation, the system can refine its calibration parameters and reduce prediction error over time. Future versions should also incorporate weather-aware reasoning, including rainfall forecasts, temperature, humidity, and evapotranspiration estimates, so that recommendations can adapt to changing environmental conditions.

User feedback should also be integrated into the irrigation history. Whether farmers accept, modify, delay, or ignore a recommendation can provide useful evidence for improving personalization and practical usability. Finally, a controlled pilot study comparing

TARAS-assisted irrigation with unassisted irrigation would help measure real outcomes such as water savings, soil-moisture stability, crop-health effects, and user trust.

8.5.4. LLM Advisory Module

Four improvements stand out. First, the curated knowledge base should be expanded to cover unusual pest-disease interactions, region-specific crop guidance, and less common soil types, while remaining curated and attributable rather than scraped from the open web. Second, the tool set should be extended with a weather-forecast tool, a historical-analogue tool for comparing current conditions with similar past cases, and a push-notification tool for proactive alerts. Third, offline and low-bandwidth support should be considered: a small on-device fallback model could answer a narrow set of common questions using cached field data such as irrigation status, recent disease results, and basic crop-stage guidance. Fourth, the module needs a feedback loop. A thumbs-up/thumbs-down mechanism on advisory responses would identify weak answers, inform system-prompt revisions, and guide knowledge-base updates. Additional improvements that fit naturally into this roadmap include conversation summarization, voice input, what-if irrigation scenarios, and finer-grained per-tool authorization.

8.5.5. Platform and User Experience

On the platform side, the next round of work should consolidate features that already exist into a more cohesive user experience. This includes deeper integration between the carbon-footprint tracker and the irrigation timetable so that water and energy savings are surfaced in the same view as the recommendations that produced them, a stakeholder-facing dashboard that

aggregates anonymized field outcomes for agricultural cooperatives and advisory bodies, and improved offline behaviour in the mobile client so that the digital twin and the most recent recommendations remain useful even when connectivity is intermittent. Internationalization beyond Turkish, particularly for languages spoken in agriculturally significant regions, would also broaden the platform's reach.

8.5.6. Longer-term Directions

Looking further ahead, the natural extensions of the current platform are multi-image and time-series reasoning for disease progression rather than single-leaf classification, tighter coupling between weather forecasts and the irrigation engine so that recommendations explicitly account for predicted rainfall and evapotranspiration windows, and a controlled pilot study with a small number of cooperating farms to measure real water and energy savings against an unassisted baseline. A pilot of this kind would also generate the labelled production data that the advisory module would need for a more rigorous quantitative evaluation, closing the loop between deployment and continued improvement.

8.6. Closing Remarks

TARAS set out to combine sensing, prediction, visualization, and advisory functions into a single mobile-first platform that small and mid-scale farmers could actually use, and to do so without overstating what any individual component can deliver. The hardware is deployed, the backend is in production, the disease classifier reaches roughly 97 percent overall accuracy on the held-out test set, with about 0.85 macro F1 on the real-world field slice, the advisory agent runs end-to-end with grounded responses in a few seconds, and the digital twin renders smoothly on commodity smartphones. Each of these components has limitations, and those limitations have been documented honestly rather than minimized.

What makes the project worth continuing is precisely the fact that the remaining gaps are addressable. Real outdoor deployments, dataset expansion in the harder disease categories, knowledge-base growth and feedback loops in the advisory layer, offline fallbacks, and a controlled pilot study would together move the platform from a working prototype to a system whose value can be measured in saved water, saved energy, and earlier disease intervention. The architecture, the design philosophy, and the evidence accumulated over the course of this project all point in the same direction: the next steps are clear, they are incremental, and they are within reach.

9. References

[1] H. Turral, J. Burke, and J.-M. Faurès, Climate Change, Water and Food Security. Rome, Italy: Food and Agriculture Organization of the United Nations, 2011. [Online]. Available: https://www.fao.org/4/i2096e/i2096e.pdf
[2] G. Ganjegunte and J. Clark, “Improved irrigation scheduling for freshwater conservation in the desert southwest U.S.,” Irrigation Science, vol. 35, no. 4, pp. 315–326, Jul. 2017, doi: 10.1007/s00271-017-0546-8.
[3] X. Chen, Z. Qi, D. Gui, Z. Gu, L. Ma, F. Zeng, L. Li, and M. W. Sima, “A model-based real-time decision support system for irrigation scheduling to improve water productivity,” Agronomy, vol. 9, no. 11, Art. no. 686, Nov. 2019, doi: 10.3390/agronomy9110686.
[4] A. Soussi, E. Zero, R. Sacile, D. Trinchero, and M. Fossa, "Smart Sensors and Smart Data for Precision Agriculture: A Review," Sensors, vol. 24, no. 8, p. 2647, 2024.
[5] R. Zhang, H. Zhu, Q. Chang, and Q. Mao, “A Comprehensive Review of Digital Twins Technology in Agriculture,” Agriculture, vol. 15, no. 9, Art. no. 903, Apr. 2025, doi: 10.3390/agriculture15090903.
[6] T. Y. Melesse, “Digital twin-based applications in crop monitoring,” Heliyon, vol. 11, no. 2, Art. no. e42137, Jan. 2025, doi: 10.1016/j.heliyon.2025.e42137.
[7] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," Frontiers in Plant Science, vol. 7, p. 1419, 2016.
[8] Republic of Türkiye, "Twelfth Development Plan (2024–2028)," Presidency of Strategy and Budget, Ankara, 2023.
[9] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," A/RES/70/1, 2015.
[10] Food and Agriculture Organization of the United Nations, “The digital revolution and its potential to transform the use of machinery and agricultural practices,” in The State of Food and Agriculture 2022. Rome, Italy: FAO, 2022. [Online]. Available: https://www.fao.org/3/cb9479en/online/sofa-2022/digital-revolution-agricultural-practices.html
[11] T. Ojha, S. Misra, and N. S. Raghuwanshi, "Wireless sensor networks for agriculture: The state-of-the-art in practice and future challenges," Computers and Electronics in Agriculture, vol. 118, pp. 66–84, 2015.
[12] B. Zhang and L. Meng, "Energy Efficiency Analysis of Wireless Sensor Networks in Precision Agriculture Economy," Scientific Programming, vol. 2021, Article ID 8346708, 2021.
[13] I. Irga, F. R. I. Mariati, and A. Zuchriadi, “Analysis of Power Consumption Efficiency of ESP Microcontroller Operation in the Implementation of IoT-Based Agricultural Monitoring Systems,” in Proc. 2023 10th International Conference on ICT for Smart Society (ICISS), Bandung, Indonesia, 2023, pp. 1–5, doi: 10.1109/ICISS59129.2023.10291944.
[14] N. K. M. Khalid, N. Jamal, F. Mustafa, N. A. Zambri, and M. S. R. Abdullah, “Solar Power IoT Based Smart Agriculture System Using NodeMCU ESP32,” Progress in Engineering Application and Technology, vol. 5, no. 1, pp. 95–102, Jun. 2024, doi: 10.30880/peat.2024.05.01.010.
[15] Espressif Systems, "Inter-Integrated Circuit (I2C)," ESP-IDF API Reference. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2c.html

[16] E. T. Engman, "Soil Moisture," in Remote Sensing in Hydrology and Water Management, G. A. Schultz and E. T. Engman, Eds. Berlin, Heidelberg: Springer, 2000, pp. 197–216.
[17] H. Koyuncu, B. Gunduz, and B. Koyuncu, “Construction of 3D Soil Moisture Maps in Agricultural Fields by Using Wireless Sensor Communication,” Gazi University Journal of Science, vol. 34, no. 1, pp. 84–98, Mar. 2021, doi: 10.35378/gujs.720778.
[18] E. A. A. D. Nagahage, I. S. P. Nagahage, and T. Fujino, “Calibration and Validation of a Low-Cost Capacitive Moisture Sensor to Integrate the Automated Soil Moisture Monitoring System,” Agriculture, vol. 9, no. 7, Art. no. 141, Jul. 2019, doi: 10.3390/agriculture9070141.
[19] Sensirion AG, "Datasheet SHT3x-DIS: Humidity and Temperature Sensors," Version 6.0, Sensirion AG, Stäfa, Switzerland, 2023.
[20] M. Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. Sebastopol, CA, USA: O’Reilly Media, 2017.
[21] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), Vancouver, BC, Canada, 2023, pp. 16133–16142.
[22] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” in Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV), Montreal, QC, Canada, 2021, pp. 10012–10022, doi: 10.1109/ICCV48922.2021.00986.
[23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in Proc. NIPS Deep Learning and Representation Learning Workshop, 2015.
[24] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. 36th Int. Conf. Machine Learning (ICML), 2019, pp. 6105–6114.
[25] T. A. Shaikh, T. Rasool, K. Veningston, and S. M. Yaseen, “The role of large language models in agriculture: harvesting the future with LLM intelligence,” Progress in Artificial Intelligence, vol. 14, pp. 117–164, 2025, doi: 10.1007/s13748-024-00359-4.
[26] S. A. L. S. Kummar, F. S. Al-Aani, H. Kahtan, M. J. Darr, and H. Al-Bashiri, “Data Visualisation for Smart Farming Using Mobile Application,” International Journal of Computer Science and Network Security, vol. 19, no. 11, pp. 1–7, Nov. 2019. [Online]. Available: http://paper.ijcsns.org/07_book/201911/20191101.pdf
[27] C. Verdouw et al., "Digital twins in smart farming," Agricultural Systems, vol. 189, p. 103046, 2021.
[28] S. Mandal, A. Yadav, F. A. Panme, K. M. Devi, and S. Kumar S. M., “Adaption of smart applications in agriculture to enhance production,” Smart Agricultural Technology, vol. 7, Art. no. 100431, Mar. 2024, doi: 10.1016/j.atech.2024.100431.
[29] Meta Platforms, Inc., “React Native,” React Native Documentation. Accessed: May 21, 2026. [Online]. Available: https://reactnative.dev/
[30] Expo, “EAS Update,” Expo Documentation. Accessed: May 21, 2026. [Online]. Available: https://docs.expo.dev/eas-update/introduction/
[31] pmndrs, “React Three Fiber Documentation.” Accessed: May 21, 2026. [Online]. Available: https://r3f.docs.pmnd.rs/
[32] H. Tyagi, S. Samant, A. Singh, P. K. Goel, V. Balyan, and R. Devi, “IoT-Based Real-Time Carbon Monitoring in Urban and Agricultural areas,” in Advanced Systems for Monitoring

Carbon Sequestration, H. M. Pandey, P. K. Goel, V. Balyan, and S. P. Yadav, Eds. Hershey, PA, USA: IGI Global Scientific Publishing, 2025, pp. 375–410, doi: 10.4018/979-8-3373-2091-5.ch016.
[33] A. Mahato, "Greenhouse gas emission from agriculture and their mitigation strategies," International Journal of Agriculture Extension and Social Development, vol. 7, no. 7, pp. 188–198, 2024.
[34] E. A. Abioye et al., "Precision irrigation management using machine learning and digital farming solutions," AgriEngineering, vol. 4, no. 1, pp. 70–103, 2022.
[35] D. Neocleous, G. Kotsiras, and G. K. Georgiou, "Applying IoT sensor‐based ferti-irrigation practices to enhance water and nutrient use efficiency in intensive potato cropping systems," Frontiers in Agronomy, vol. 7, 2025.
[36] S. Getahun, H. Kefale, and Y. Gelaye, “Application of precision agriculture technologies for sustainable crop production and environmental sustainability: A systematic review,” The Scientific World Journal, vol. 2024, no. 1, Art. no. 2126734, 2024, doi: 10.1155/2024/2126734.
[37] N. K. Nawandar and V. R. Satpute, “IoT based low cost and intelligent module for smart irrigation system,” Computers and Electronics in Agriculture, vol. 162, pp. 979–990, Jul. 2019, doi: 10.1016/j.compag.2019.05.027.
[38] C. Pylianidis, S. Osinga, and I. N. Athanasiadis, “Introducing digital twins to agriculture,” Computers and Electronics in Agriculture, vol. 184, Art. no. 105942, May 2021, doi: 10.1016/j.compag.2020.105942.
[39] Espressif Systems, “ESP32-C6 Technical Reference Manual,” 2023.
[40] “Systematic review of machine learning applications in sustainable agriculture: Insights on soil health and crop improvement,” PHYTON, vol. 94, 61375, 2025.
[41] PostgreSQL Global Development Group, PostgreSQL Documentation
[42] Timescale Inc., “TimescaleDB Documentation – Continuous Aggregates,” 2024.
[43] PostGIS Project, PostGIS Manual
[44] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements, FAO Irrigation and Drainage Paper No. 56. Rome, Italy: Food and Agriculture Organization of the United Nations, 1998. Available: https://www.fao.org/4/x0490e/x0490e00.htm
[45] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in Proc. 2022 IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA, 2022, pp. 11966–11976, doi: 10.1109/CVPR52688.2022.01167.
[46] S. J. Czaja and C. C. Lee, “The Impact of Aging on Access to Technology,” Universal Access in the Information Society, vol. 5, no. 4, pp. 341–349, 2007. doi: 10.1007/s10209-006-0060-x. Available: https://doi.org/10.1007/s10209-006-0060-x
[47] R. Caruana, A. Niculescu-Mizil, G. Crew, and A. Ksikes, “Ensemble selection from libraries of models,” in Proc. 21st International Conference on Machine Learning (ICML), Banff, AB, Canada, 2004, Art. no. 18, doi: 10.1145/1015330.1015432.
[48] J. G. A. Barbedo, “Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification,” Computers and Electronics in Agriculture, vol. 153, pp. 46–53, 2018, doi: 10.1016/j.compag.2018.08.013.

[49] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, “PlantDoc: A dataset for visual plant disease detection,” in Proc. 7th ACM IKDD CoDS and 25th COMAD, Hyderabad, India, 2020, pp. 249–253, doi: 10.1145/3371158.3371196.
[50] Y. Guo, Y. Lan, and X. Chen, “CST: Convolutional Swin Transformer for detecting the degree and types of plant diseases,” Computers and Electronics in Agriculture, vol. 202, Art. no. 107407, 2022, doi: 10.1016/j.compag.2022.107407.
[51] ISO, ISO 14064-1: Greenhouse Gases - Carbon Footprint Quantification, International Organization for Standardization, 2018.
[52] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How Transferable Are Features in Deep Neural Networks?” in Advances in Neural Information Processing Systems, 2014. Available: https://arxiv.org/abs/1411.1792